



NURTURING POTENTIAL

SAKET GYANPEETH'S
SAKET COLLEGE OF ARTS, SCIENCE AND COMMERCE
KALYAN (EAST)

ACADEMIC YEAR [2022 – 2023]

MSc. Information Technology

SEMESTER – I

- 1. Research in Computing***
 - 2. Data Science***
 - 3. Cloud Computing***
 - 4. Soft Computing Techniques***
-

SUBMITTED BY

ANSHUMAN

GAURISHANKAR SINGH

AS PRESCRIBED BY

UNIVERSITY OF MUMBAI



Seat No.: 3254185

RESEARCH IN COMPUTING



NURTURING POTENTIAL

SAKET GYANPEETH'S
SAKET COLLEGE OF ARTS, SCIENCE, AND COMMERCE
(Permanently Affiliated to University of Mumbai)
NAAC Accredited B Grade

Saket Vidyanagari Marg, Chinchpada Road, Katemanivali, Kalyan (East) – 421306, Dist.

Department of Information Technology

Thane (MAH)

CERTIFICATE

This is to certify that

ANSHUMAN GAURISHANKAR SINGH of MSc Information Technology Part-I

Class has satisfactorily carried out the required practical in the subject.

RESEARCH IN COMPUTING

For the Academic year 2022 – 2023

Practical In-Charge

Head of Department

External Examiner

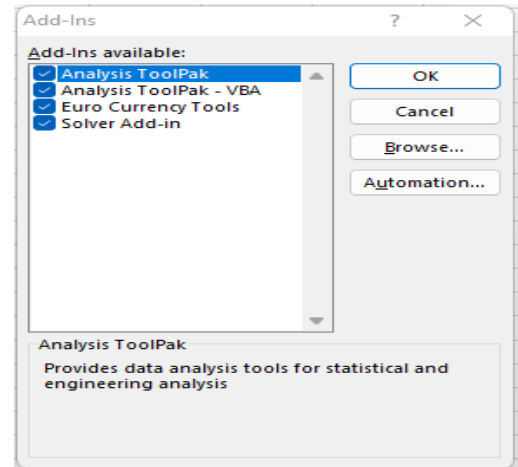
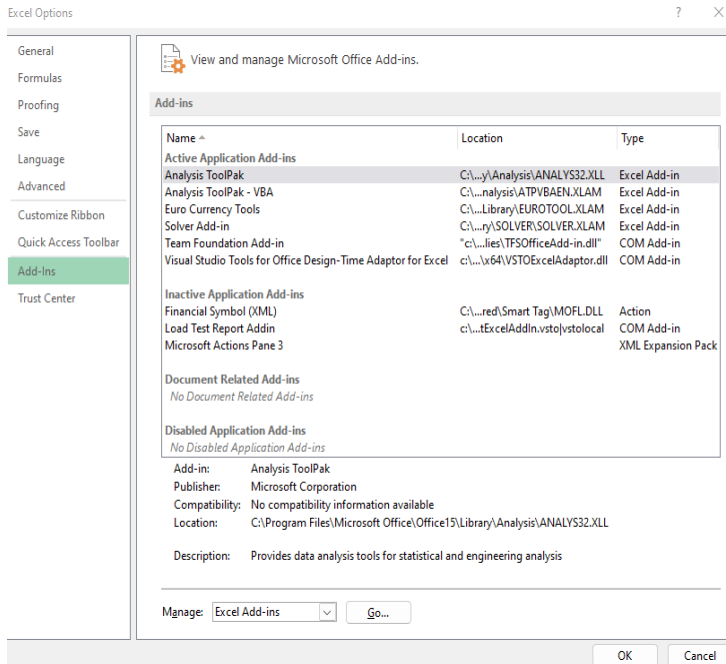
INDEX

Sr. No.	Aim	Sign
1	Write a program for obtaining descriptive statistics of data	
2	Import data from different data source (from Excel, csv, mysql, sql, server, oracle to R/python/Excel)	
3	Design a survey from for a given case study, collect the primary data and analyse it	
4	Perform analysis of given secondary data	
5	Perform testing of hypothesis using one sample t-test	
6	Write a program for t-test comparing two means for independent samples	
7	Perform testing of hypothesis using paired t-test	
8	Perform testing of hypothesis using chi-squared godness-of-fit test	

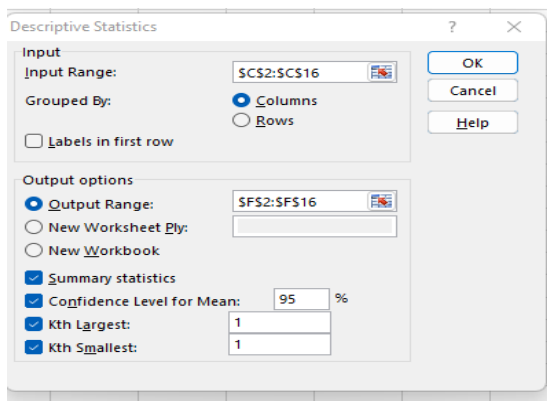
PRACTICAL-1

Aim: Write a program for obtaining descriptive statistics of data.

Step1: Program/Steps to obtain descriptive statistics of data. Using Excel Go to File Menu -> Options -> Add-Ins -> Select Analysis Tool Pak -> Press OK



Step2: Go to Data Menu -> Data Analysis -> Descriptive Statistics ->and Select the Age Range



OUTPUT:

FILE

HOME

INSERT

PAGE LAYOUT

FORMULAS

DATA

REVIEW

VIEW

Load Test

Team

From Access

From Web

From Text

From Other Sources

Get External Data

Existing Connections

Refresh All

Connections

Properties

Edit Links

Sort

Filter

Clear

Reapply

Advanced

Text to Columns

Flash Fill

Remove Duplicates

Data Validation

Consolidate

What-If Analysis

DATA TOOLS

J11

	A	B	C	D	E	F	G	H	I	J	K	L
1	Sr.No	Name	Age	Rating								
2	1	Mansi	18	6.5		Column1						
3	2	Priyanka	20	3.5								
4	3	Tejashree	22	4.5	Mean		35.26666667					
5	4	Gudiya	50	3.2	Standard Error		3.606167556					
6	5	Sejal	23	2.2	Median		31					
7	6	Akash	43	2.7	Mode		50					
8	7	Hiten	50	2.87	Standard Deviation		13.96662689					
9	8	Suraj	34	3.56	Sample Variance		195.0666667					
10	9	Sneha	25	4.78	Kurtosis		-1.300536327					
11	10	Shruti	25	5.56	Skewness		0.456220211					
12	11	Vidya	31	7.78	Range		42					
13	12	Sujata	28	8.2	Minimum		18					
14	13	Nikhil	60	9.1	Maximum		60					
15	14	Nikita	55	6.9	Sum		529					
16	15	Ramesh	45	1.9	Count		15					
17					Largest(1)		60					
18					Smallest(1)		18					
19					Confidence Level(95.0%)		7.734460169					
20												
21												

PRACTICAL -2

Aim: Import data from different data sources (from Excel, csv, mysql, sql server, oracle to R/Python/Excel)

➤ **READ DATA FROM MYSQL**

#Install Python-MySQL connector(Download from Resource folder)

Click Start> MySQL CLI> Login > Create database > create table > Insert records in table

```
mysql> Create database mydb1;
Query OK, 1 row affected (0.00 sec)

mysql> use mydb1;
Database changed
mysql> Create table test(name varchar(10), eid int);
Query OK, 0 rows affected (0.02 sec)

mysql> insert into test values('Hiten','1');
Query OK, 1 row affected (0.00 sec)

mysql> insert into test values('Sky','2');
Query OK, 1 row affected (0.02 sec)

mysql> insert into test values('Abhishek','3');
Query OK, 1 row affected (0.00 sec)

mysql> insert into test values('Chiku','4');
Query OK, 1 row affected (0.00 sec)

mysql> insert into test values('Rustom','5');
Query OK, 1 row affected (0.00 sec)

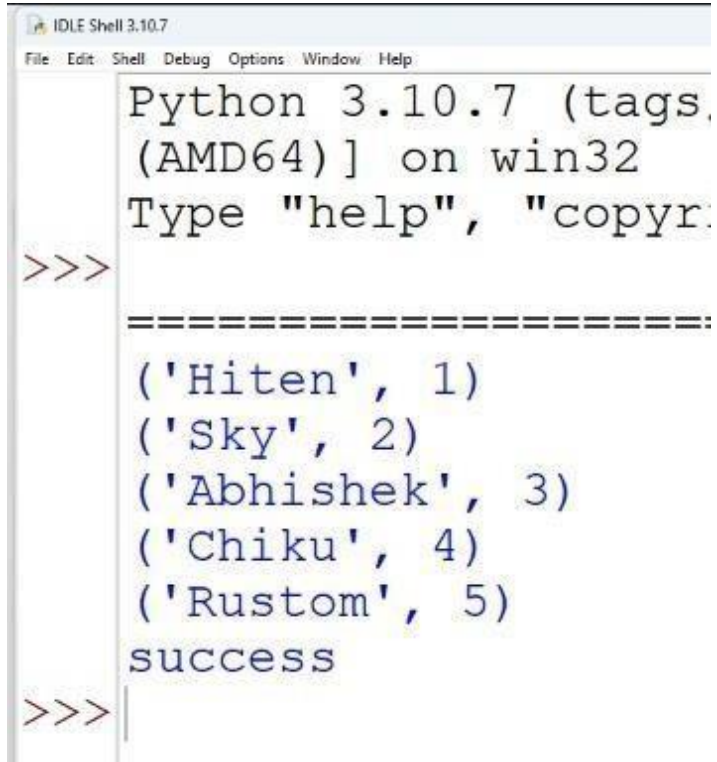
mysql> |
```

#To install pymysql package run pip command on cmd
#pip3 install pymysql

CODE:

```
import pymysql
db = pymysql.connect(host='localhost',user='root',password='root',database='mydb1')
cursor = db.cursor()
sql = """SELECT * from test"""
try:
    cursor.execute(sql)
    results = cursor.fetchall()
    for row in results:
        print(row)
```

```
print("success")
db.commit()
except:
    db.rollback()
db.close()
```

A screenshot of the IDLE Shell 3.10.7 window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell displays the following text: 'Python 3.10.7 (tags, (AMD64)] on win32', 'Type "help", "copyr:', and a prompt '>>>'. Below the prompt, there is a separator line '=====', followed by a list of tuples: ('Hiten', 1), ('Sky', 2), ('Abhishek', 3), ('Chiku', 4), and ('Rustom', 5). Below this list, the word 'success' is printed. The prompt '>>>' appears again at the bottom of the shell.

```
IDLE Shell 3.10.7
File Edit Shell Debug Options Window Help
Python 3.10.7 (tags,
(AMD64)] on win32
Type "help", "copyr:
>>>
=====
('Hiten', 1)
('Sky', 2)
('Abhishek', 3)
('Chiku', 4)
('Rustom', 5)
success
>>>
```

PRACTICAL-3

Aim: Design a survey form for a given case study, collect the primary data, and analyse it

Case 1:

A researcher wants to conduct a Survey in colleges on Use of ICT in higher education from Mumbai, Thane and Navi Mumbai. The survey focuses on access to and use of ICT in teaching and learning, as well as on attitudes towards the use of ICT in teaching and learning. Design questionnaire addressed to teachers seeks information about the target class, his experience using ICT for teaching, access to ICT infrastructure, support available, ICT based activities and material used, obstacles to the use of ICT in teaching, learning activities with the target class, your skills and attitudes to ICT, and some personal background information. Arrange question in following groups:

1. Information about the target class you teach
2. Experience with ICT for teaching
3. ICT access for teaching
4. Support to teachers for ICT use
5. ICT based activities and material used for teaching
6. Obstacles to using ICT in teaching and learning
7. Learning activities with the target class
8. Teacher skills
9. Teacher opinions and attitudes
10. Personal background information

Using the collected data for analysis

Seat No.: 3254185

	A	B	C	D	E	F	G	H	I
1	Timestamp	Username	Name	Phone number	1.Age	2.Gender	3.Subject	4.Teaching hours per week	1. Including this year, how long have you using ICT?
2	2022/11/24 7:07:34 PM GMT+5:30	siddhibhosale08@gmail.com	Siddhi Vinayak Bhosale	8208228407	21-30	Female	Science	Fewer than 10hours	1-3 years
3	2022/11/24 7:09:48 PM GMT+5:30	pawarprerana9702@gmail.com	PRERANA TANAJI PAWAR	8108876282	21-30	Female	Technology	10-15 hours	3-6 years
4	2022/11/24 7:21:26 PM GMT+5:30	snehapasi2001@gmail.com	Pasi sneha ramkevel	9372622367	21-30	Female	Science	Fewer than 10hours	1-3 years
5	2022/11/24 9:41:07 PM GMT+5:30	pratikarolikar2.model@gmail.com	Pratik Arolkar	9.1808E+11	21-30	Male	Technology	Fewer than 10hours	1-3 years
6	2022/11/24 10:00:30 PM GMT+5:30	sandipdhangar1997@gmail.com	Sandip Dhangar	7776969640	21-30	Male	Technology	Fewer than 10hours	1-3 years
7	2022/11/24 10:01:59 PM GMT+5:30	jayshreenaidu67@gmail.com	Jayshree	9930092706	40-50	Female	Science	Fewer than 10hours	3-6 years
8	2022/11/25 7:51:07 AM GMT+5:30	jaydeppagar37617@gmail.com	Jaydeep Pagar	8369747784	21-30	Male	Science	Fewer than 10hours	1-3 years
9	2022/11/25 10:38:10 AM GMT+5:30	sharvarigavhane19@gmail.com	Sharvari Gavhane	9820188487	21-30	Female	Technology	Fewer than 10hours	1-3 years
10	2022/11/25 11:17:06 AM GMT+5:30	anujfivezero@gmail.com	Anuj Gandhat	7977936516	21-30	Male	Science	Fewer than 10hours	1-3 years
11	2022/11/25 11:25:00 AM GMT+5:30	shardulika@gmail.com	Shardulika Agrahari	9137519993	21-30	Female	Other	Fewer than 10hours	1-3 years
12	2022/11/25 11:38:06 AM GMT+5:30	priyankakondhavl01@gmail.com	Priyanka Kondhavl	9152698805	21-30	Female	Science	Fewer than 10hours	1-3 years
13	2022/11/25 11:54:23 AM GMT+5:30	durguderaska10@gmail.com	Rasika	865241852	21-30	Female	Other	Fewer than 10hours	1-3 years
14	2022/11/25 12:55:53 PM GMT+5:30	aakashchale20@gmail.com	Aakash Achale	7666453334	21-30	Male	Technology	Fewer than 10hours	1-3 years
15	2022/11/25 3:15:47 PM GMT+5:30	yashu020@icloud.com	Yashu	9136288011	21-30	Male	Science	16-20 hours	3-6 years
16	2022/11/25 4:22:16 PM GMT+5:30	qfaizal755@gmail.com	Faizal	9702982558	21-30	Male	Other	Fewer than 10hours	1-3 years
17	2022/11/25 4:31:56 PM GMT+5:30	hrcot07@gmail.com	Hiten	7385890876	21-30	Male	Science	Fewer than 10hours	1-3 years
18	2022/11/25 8:39:16 PM GMT+5:30	nbalajinadar@gmail.com	Balaji Nadar	9820945498	21-30	Male	Other	Fewer than 10hours	1-3 years
19	2022/11/25 9:34:56 PM GMT+5:30	pallanaidu01@gmail.com	Eshika Naidu	9819356960	30-40	Female	Mathematics	10-15 hours	3-6 years
20	2022/11/25 9:39:42 PM GMT+5:30	naiduhemakshi1111@gmail.com	Hemakshi M Naidu	7875754291	40-50	Female	Mathematics	10-15 hours	6-10 years
21	2022/11/26 3:06:08 PM GMT+5:30	malitejashri46@gmail.com	Tejashri prakash mali	7304350488	21-30	Female	Technology	Fewer than 10hours	1-3 years
22	2022/11/27 10:19:20 AM GMT+5:30	abhishekdas2222@gmail.com	Abhishek Das	8878219561	21-30	Male	Technology	10-15 hours	1-3 years
23	2022/11/27 10:30:28 AM GMT+5:30	cvishakha018@gmail.com	Vishakha Vijay Chavan	9011653459	21-30	Female	Technology	Fewer than 10hours	1-3 years
24	2022/11/27 10:31:13 AM GMT+5:30	pranitwaingankar20@gmail.com	Shruti Waingankar	8369521630	21-30	Female	Science	Fewer than 10hours	1-3 years
25	2022/11/27 10:36:41 AM GMT+5:30	surajgupta37@gmail.com	Suraj Gupta	8108899101	21-30	Male	Science	Fewer than 10hours	1-3 years
26	2022/11/27 10:42:18 AM GMT+5:30	santoshyadav2397@gmail.com	Santosh Yadav	9005063716	21-30	Male	Technology	Fewer than 10hours	1-3 years
27									

J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA
2. How often do you use computers or the internet in your classes?	1. What a	2. Does y	1. Is part	2. Who pr	3. Have y	1. Which	2. What a	1. What o	1. To what extent are you confident using ICT?								
All of the time	Desktop c	Yes	Yes	A more ex	Introduct	Electronic	Communi	Insuffici	Use a spreadsheet to plot a graph								
All of the time	Digital car	Yes	Yes	A more ex	Advanced	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Download								
All of the time	Computer	Yes	Yes	A more ex	Introduct	Existing or	Create yo	Insuffici	Create a presentation with video or audio clips movies;Capture and edit digital photos, movies or								
Sometimes	Digital car	No	Yes	A more ex	Advanced	Existing or	Use ICT to	Insuffici	Create a presentation with video or audio clips movies;Capture and edit digital photos, movies or								
Sometimes	Digital car	Yes	Yes	A more ex	Introduct	Material c	Post home	Insuffici	Use a spreadsheet to plot a graph								
Sometimes	Computer	No	Yes	A more ex	Introduct	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Create								
Sometimes	Digital car	No	No	An online	Other pro	Existing or	Communi	Most of th	Create a presentation with video or audio clips movies;Capture and edit digital photos, movies or								
All of the time	Computer	Yes	Yes	A more ex	Advanced	Existing or	Use ICT to	Insuffici	Create a presentation with video or audio clips movies								
All of the time	Desktop c	Yes	Yes	A more ex	Subject-sp	Existing or	Communi	Insuffici	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Capture								
All of the time	Desktop c	Yes	Yes	School ICT	Introduct	Material c	Create yo	Using ICT	Create a presentation with video or audio clips movies								
Sometimes	Desktop c	Yes	Yes	School ICT	Introduct	Existing or	Use ICT to	Insuffici	Create a presentation with video or audio clips movies;Download and install software on a system								
All of the time	Digital car	Yes	Yes	A more ex	Introduct	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Capture								
Rarely	Computer	Yes	Yes	A more ex	Introduct	Material c	Use ICT to	Insuffici	Create a presentation with video or audio clips movies;Capture and edit digital photos, movies or								
All of the time	Computer	No	Yes	A more ex	Advanced	Existing or	Browse/s	Most of th	Use a spreadsheet to plot a graph								
Never	Digital car	No	No	Expert fro	Other pro	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Capture and edit digital photos, movies or other images								
Sometimes	Computer	No	Yes	School ICT	Introduct	Existing or	Communi	Insuffici	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Capture								
Rarely	Computer	No	No	A more ex	Course on	Material c	Create yo	Insuffici	Create a presentation with video or audio clips movies;Capture and edit digital photos, movies or								
All of the time	Digital car	Yes	Yes	School ICT	Advanced	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Capture								
Sometimes	Computer	Yes	Yes	A more ex	Subject-sp	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Download and install software on a system								
All of the time	Desktop c	Yes	No	A more ex	Introduct	Existing or	Browse/s	Insuffici	Use a spreadsheet to plot a graph								
All of the time	Desktop c	Yes	No	A more ex	Introduct	Existing or	Create yo	Most of th	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Capture								
Sometimes	Computer	Yes	Yes	School ICT	Introduct	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Create a presentation with video or audio clips movies;Capture								
Rarely	Computer	Yes	Yes	School ICT	Introduct	Material c	Communi	Insuffici	Use a spreadsheet to plot a graph;Capture and edit digital photos, movies or other images								
Sometimes	Computer	No	No	School ICT	Introduct	Existing or	Communi	Insuffici	number of system;Using ICT in teaching and learning not being a goal in your school								
Rarely	Computer	Yes	Yes	A more ex	Advanced	Material c	Post home	Most of th	Create a presentation with video or audio clips movies								

	Question1	Question2	Question3	Question4	Question5	Question6	Question7	Question8	Question9	Question10	Question11	Question12
COUNT(N)	25	25	25	25	25	25	25	25	25	25	25	24
Not Answer	0	0	0	0	0	0	0	0	0	0	0	0
TOTAL:	25	25	25	25	25	25	25	25	25	25	25	24
DISAGREE:	8	6										
Agree:	17	19										
TOTAL:	25	25										
PERCENTAGE:												
NO(%):	133%											
YES(%):	89%											
TOTAL:	223%											

FORMULA FOR OPERATION:

count(N) = =COUNTA(H2:H26)

Not Answer = =COUNT(H2:H26)

Total =SUM(E29:E30)

YES =COUNTIF(L\$2:L\$26,\$M24)

NO= COUNTIF(L\$2:L\$26,\$M25)

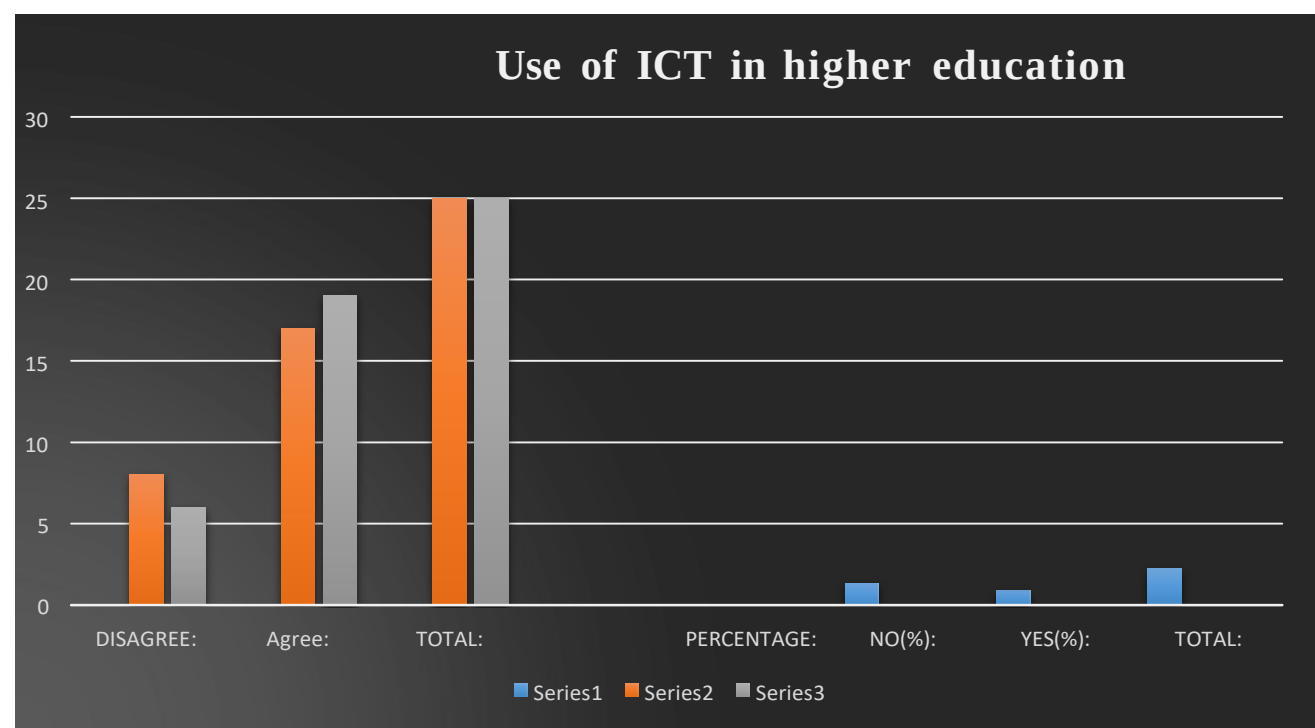
PERCENTAGE:

YES= E34/F34

NO= E33/F33

TOTAL=SUM(D38:D39)

OUTPUT:



PRACTICAL 4

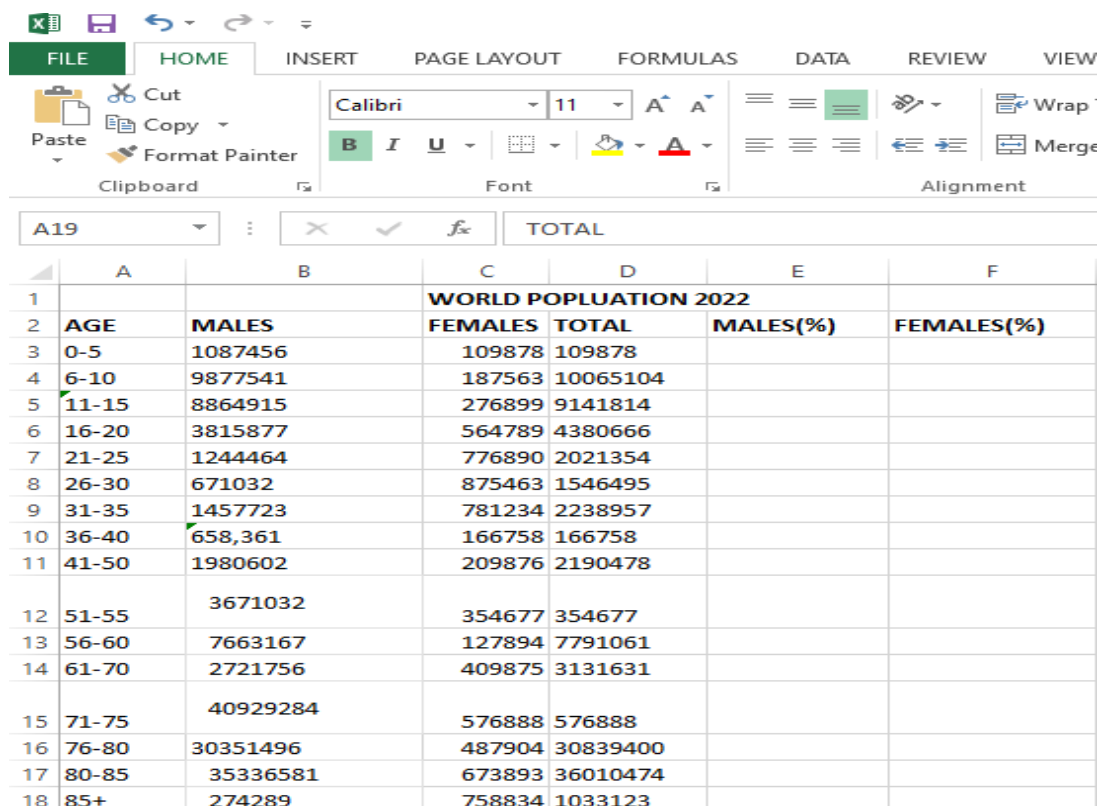
Aim: Perform analysis of given secondary data.

Steps in Secondary Data Analysis

1. **Determine your research question** – Knowing exactly what you are looking for.
2. **Locating data**– Knowing what is out there and whether you can gain access to it. A quick Internet search, possibly with the help of a librarian, will reveal a wealth of options.
3. **Evaluating relevance of the data** – Considering things like the data's original purpose, when it was collected, population, sampling strategy/sample, data collection protocols, operationalization of concepts, questions asked, and form/shape of the data.
4. **Assessing credibility of the data** – Establishing the credentials of the original researchers, searching for full explication of methods including any problems encountered, determining how consistent the data is with data from other sources, and discovering whether the data has been used in any credible published research.
5. **Analysis** – This will generally involve a range of statistical processes. Example: Analyse the given Population Census Data for Planning and Decision Making by using the size and composition of populations

Example: Analyse the given Population Census Data for Planning and Decision Making by using the size and composition of populations

OUTPUT:



	A	B	C	D	E	F
1			WORLD POPULATION 2022			
2	AGE	MALES	FEMALES	TOTAL	MALES(%)	FEMALES(%)
3	0-5	1087456	109878	109878		
4	6-10	9877541	187563	10065104		
5	11-15	8864915	276899	9141814		
6	16-20	3815877	564789	4380666		
7	21-25	1244464	776890	2021354		
8	26-30	671032	875463	1546495		
9	31-35	1457723	781234	2238957		
10	36-40	658,361	166758	166758		
11	41-50	1980602	209876	2190478		
12	51-55	3671032	354677	354677		
13	56-60	7663167	127894	7791061		
14	61-70	2721756	409875	3131631		
15	71-75	40929284	576888	576888		
16	76-80	30351496	487904	30839400		
17	80-85	35336581	673893	36010474		
18	85+	274289	758834	1033123		

Put the cursor in cell **B19** and click on the **AutoSum** and then click **Enter**. This will calculate the total population. Then copy the formula in cell **D19** across the row **19**. (**Total_population**)

To calculate the percent of males in cell **E3**, enter the formula

[$-1*100*Male_count*Total_population$]
 $=-1*100*B3/D\$19$

And copy the formula in cell **E3** down to cell **E18**.

To calculate the percent of females in cell **F3**, enter the formula

[$1*100*Female_count*Total_population$]
 $=100*C3/D\$19$. Copy the formula in cell **F3** down to cell **F18**.

This gives percentage in +ve for female and –ve for male.

To build the population pyramid, we need to choose a horizontal bar chart with two series of data (% male and % female) and the age labels in column A as the **Category X-axis** labels. Highlight the range **A2:A18**, hold down the CTRL key and highlight the range **E2:F19**

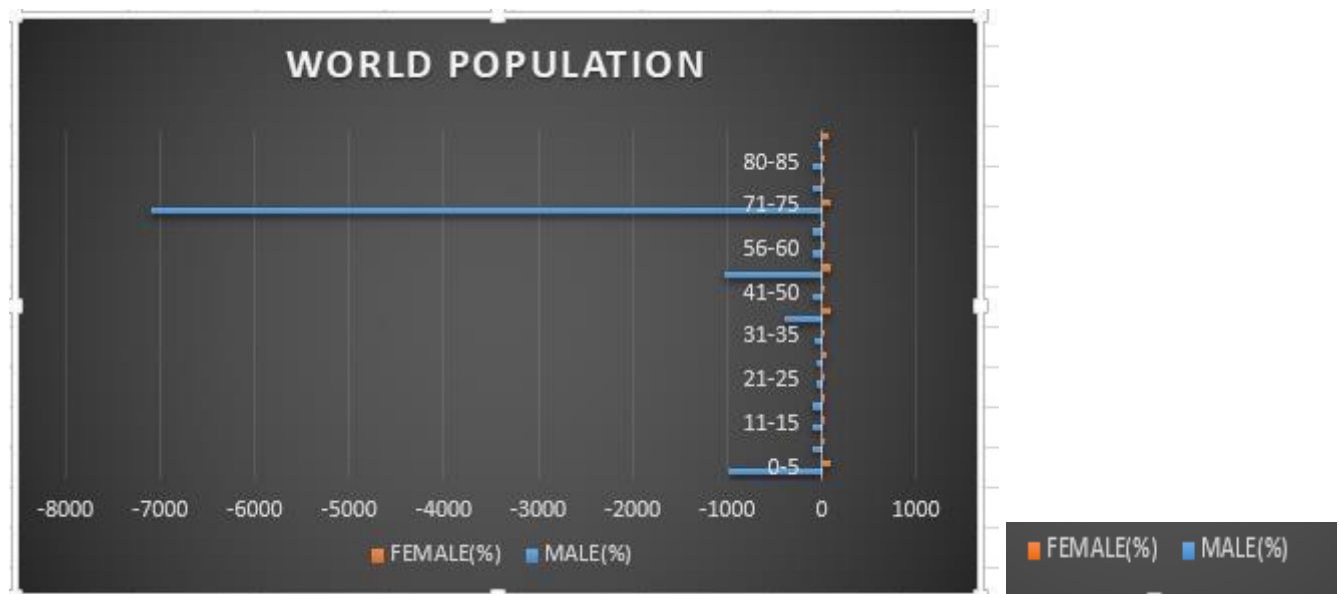
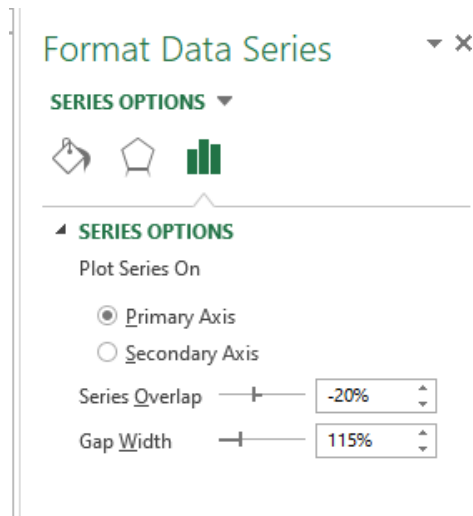
K10						
	A	B	C	D	E	F
1			WORLD POPLUATION 2022			
2	AGE	MALES	FEMALES	TOTAL	MALE(%)	FEMALE(%)
3	0-5	1087456	109878	109878	-989.6940243	100
4	6-10	9877541	187563	10065104	-98.13650212	1.863497883
5	11-15	8864915	276899	9141814	-96.97107161	3.028928394
6	16-20	3815877	564789	4380666	-87.10723438	12.89276562
7	21-25	1244464	776890	2021354	-61.5658613	38.4341387
8	26-30	671032	875463	1546495	-43.39050563	56.60949437
9	31-35	1457723	781234	2238957	-65.1072352	34.8927648
10	36-40	6,58,361	166758	166758	-394.8002495	100
11	41-50	1980602	209876	2190478	-90.41871226	9.581287737
12	51-55	3671032	354677	354677	-1035.035257	100
13	56-60	7663167	127894	7791061	-98.35845208	1.641547923
14	61-70	2721756	409875	3131631	-86.91177217	13.08822783
15	71-75	40929284	576888	576888	-7094.840593	100
16	76-80	30351496	487904	30839400	-98.41791993	1.582080066
17	80-85	35336581	673893	36010474	-98.12861947	1.871380532
18	85+	274289	758834	1033123	-26.54950088	73.45049912
19	TOTAL	104259443	7339315	111598758	-10465.43351	648.936613
20						

Under **inset** tab, under horizontal bar charts select **clustered bar chart**

Put the tip of your mouse arrow on the **Y-axis** (vertical axis) so it says “Category Axis”, right click and chose **Format Axis**

Choose **Axis options** tab and set the major and minor tick mark type to **None**, Axis labels to **Low**, and click **OK**.

Click on any of the bars in your pyramid, click right and select “format data series”. Set the **Overlap** to **100** and **Gap Width** to **0**. Click **OK**.



PRACTICAL 5

Aim: Perform testing of hypothesis using one-sample t-test.

One sample t-test : The One Sample t Test determines whether the sample mean is statistically different from a known or hypothesized population mean. The One Sample t Test is a parametric test.

H0: Mean age of given sample is 30.

H1: Mean age of given sample is not 30

#pip3 install scipy

#pip3 install numpy

```
from scipy.stats import ttest_1samp
import numpy as np
ages = np.genfromtxt('ages.csv')
print(ages)
ages_mean = np.mean(ages)
print("Mean age:",ages_mean)
print("Test 1: m=30")
tset, pval = ttest_1samp(ages, 30)
print('p-values - ',pval)
if pval< 0.05:

    print("we reject null hypothesis")

else:

    print("we fail to reject null hypothesis")
```

OUTPUT:

```
===== RESTART: C:\Mansi\RIC PRACTICAL 5.PY =====
[20. 30. 25. 13. 16. 17. 34. 35. 38. 43. 45. 48. 49. 50. 51. 54. 55. 56.
 59. 61. 62. 18. 22. 29.]
Mean age: 38.75
Test 1: m=30
p-values - 0.01333239479255858
we reject null hypothesis
```

#TEST 2

H0: Mean age of given sample is 38.

H1: Mean age of given sample not 38

```
from scipy.stats import ttest_1samp
import numpy as np
ages = np.genfromtxt('ages.csv')
print(ages)

ages_mean = np.mean(ages)
print("Mean
age:",ages_mean)print("Test
2: m=38")
tset, pval = ttest_1samp(ages,
38)print('p-values - ',pval)
if pval< 0.05:

    print("we reject null hypothesis")
else:

    print("we fail to reject null hypothesis")
```

OUTPUT:

```
===== RESTART: C:\Mansi\RIC PRACTICAL 5.PY =====
[20. 30. 25. 13. 16. 17. 34. 35. 38. 43. 45. 48. 49. 50. 51. 54. 55. 56.
 59. 61. 62. 18. 22. 29.]
Mean age: 38.75
Test 1: m=38
p-values - 0.8202593087020069
we fail to reject null hypothesis
```

Practical 6

Aim: Write a program for t-test comparing two means for independent samples.

The t distribution provides a good way to perform one sample tests on the mean when the population variance is not known provided the population is normal or the sample is sufficiently large so that the Central Limit Theorem applies.

Two Sample t Test

Example: A college Principal informed classroom teachers that some of their students showed unusual potential for intellectual gains. One months later the students identified to teachers as having potential for unusual intellectual gains showed significantly greater gains performance on a test said to measure IQ than did students who were not so identified. Below are the data for the students:

	A	B	C
1	Experiment	Comaparison	
2	35	2	
3	40	27	
4	12	38	
5	15	31	
6	21	1	
7	14	19	
8	46	1	
9	10	34	
10	28	3	
11	48	1	
12	16	2	
13	30	3	
14	32	2	
15	48	1	
16	31	2	
17	22	1	
18	12	3	
19	39	29	
20	19	37	
21	25	2	
22	27.15	11.95	Mean
23	12.50799744	14.24245414	SD

Experimental Data

To calculate Standard Mean go to cell A22 and type =SUM(A2:A21)/20

To calculate Standard Deviation go to cell A23 and type =STDEV(A2:A21)

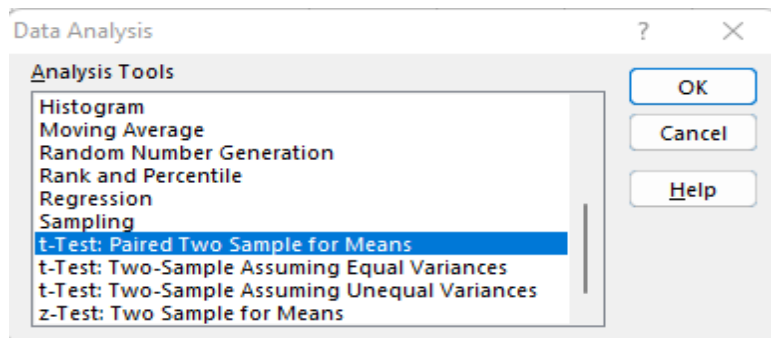
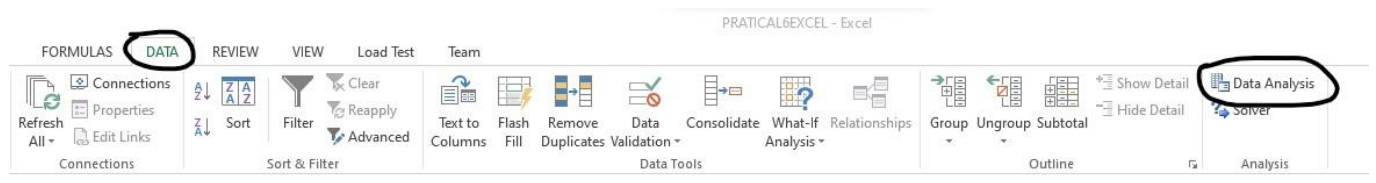
Comparison Data

To calculate Standard Mean go to cell B22 and type =SUM(B2:B21)/20

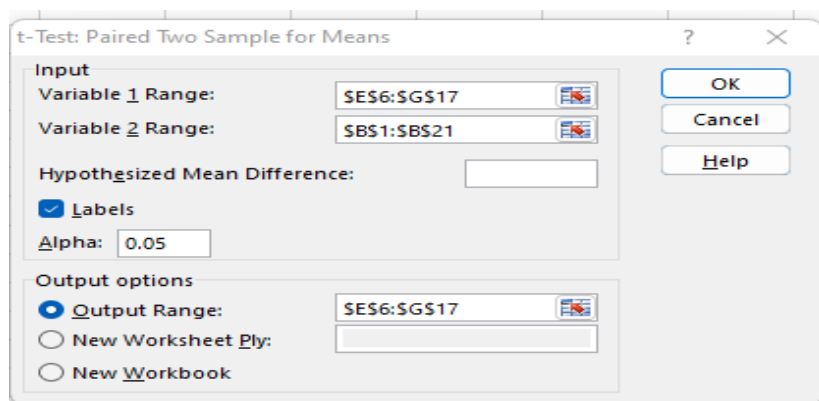
To calculate Standard Deviation go to cell B23 and type =STDEV(B2:B21)

To find T-Test Statistics go to data Data Analysis

OUTPUT:



	A	B	C	D	E	F
1	Experiment	Comaparison				
2	35	2				
3	40	27				
4	12	38				
5	15	31				
6	21	1				
7	14	19				
8	46	1				
9	10	34				
10	28	3				
11	48	1				
12	16	2				
13	30	3				
14	32	2				
15	48	1				
16	31	2				
17	22	1				
18	12	3				
19	39	29				
20	19	37				
21	25	2				



To calculate the T-Test square value go to cell E20 and type

$$=(A22-B22)/SQRT((A23*A23)/COUNT(A2:A21)+(B23*B23)/COUNT(A2:A21))$$

Now go to cell E20 and type

=IF(E20<E12,"H0 is Accepted", "H0 is Rejected and H1 is Accepted")

Our calculated value is larger than the tabled value at $\alpha = .01$, so we reject the null hypothesis and accept the alternative hypothesis, namely, that the difference in gain scores is likely the result of the experimental treatment and not the result of chance variation.

	<i>Experiment</i>	<i>Comaparison</i>
Mean	27.15	11.95
Variance	156.45	213.5236842
Observations	20	20
Pearson Correlation	-0.395904927	
Hypothesized Mean Difference	0	
df	19	
t Stat	2.996289153	
P(T<=t) one-tail	0.003711226	
t Critical one-tail	1.729132812	
P(T<=t) two-tail	0.007422452	
t Critical two-tail	2.093024054	

OUTPUT:

	A	B	C	D	E	F	G
1	Experiment	Comaparison					
2	35	2			H0 -Difference in gain score is not likely the result of experiment		
3	40	27			H1-Difference in gain score is likely the result of experimental		
4	12	38			t-Test: Paired Two Sample for Means		
5	15	31			t-Test: Paired Two Sample for Means		
6	21	1				<i>Experiment</i>	<i>Comaparison</i>
7	14	19			Mean	27.15	11.95
8	46	1			Variance	156.45	213.5236842
9	10	34			Observations	20	20
10	28	3			Pearson Correlation	-0.395904927	
11	48	1			Hypothesized Mean Difference	0	
12	16	2			df	19	
13	30	3			t Stat	2.996289153	
14	32	2			P(T<=t) one-tail	0.003711226	
15	48	1			t Critical one-tail	1.729132812	
16	31	2			P(T<=t) two-tail	0.007422452	
17	22	1			t Critical two-tail	2.093024054	
18	12	3					
19	39	29					
20	19	37			3.586175078	Calculated Value	
21	25	2					
22	27.15	11.95	Mean			H0 is Rejected and H1 is accepted	
23	12.50799744	14.24245414	SD				
24							

Practical 7

Aim: Perform testing of hypothesis using paired t-test.

The paired sample t-test is also called the dependent sample t-test. It is an univariate test that tests for a significant difference between 2 related variables. An example of this is if you were to collect the blood pressure for an individual before and after some treatment, condition, or time point. The data set contains blood pressure readings before and after an intervention. These are variables "bp_before" and "bp_after".

The hypothesis being tested is:

- **H₀** - The mean difference between sample 1 and sample 2 is equal to 0.
- **H₁** - The mean difference between sample 1 and sample 2 is not equal to 0

```
from scipy import stats
import matplotlib.pyplot as plt
import pandas as pd
df = pd.read_csv("blood_pressure.csv")
print(df[['bp_before', 'bp_after']].describe())
tst, pval = stats.ttest_rel(df['bp_before'], df['bp_after'])
if pval < 0.05:
    print("we reject null hypothesis")
else:
    print("we fail to reject null hypothesis")
```

OUTPUT:

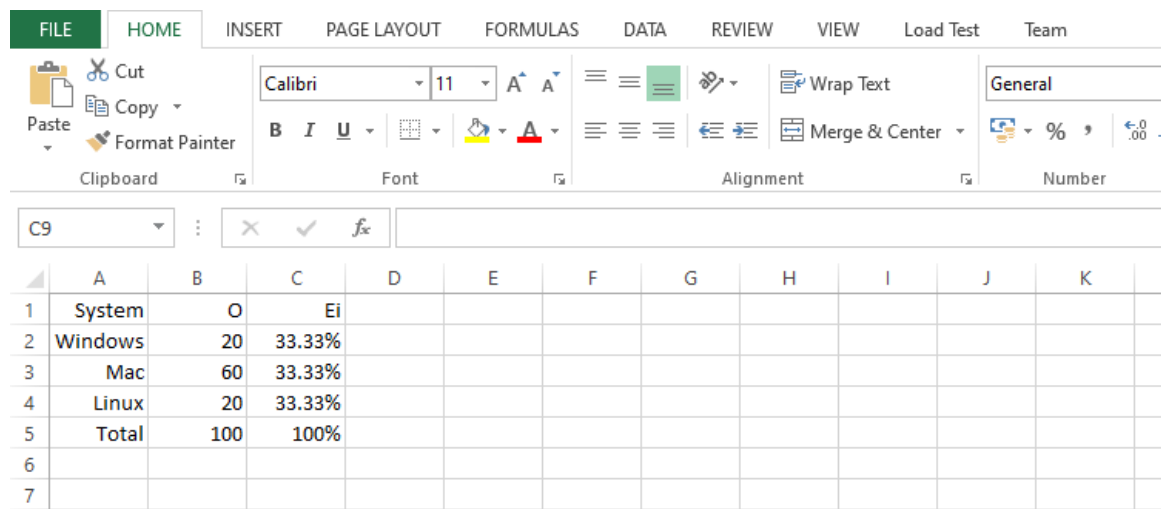
```
===== RESTART: C:\Mansi
count      bp_before      bp_after
count      5.000000      5.000000
mean       156.000000     137.800000
std         33.615473      13.198485
min         120.000000     120.000000
25%         130.000000     129.000000
50%         150.000000     140.000000
75%         180.000000     150.000000
max         200.000000     150.000000
we fail to reject null hypothesis
>>>
```

Practical 8

Aim: Perform testing of hypothesis using the chi-squared goodness-of-fit test.

Problem in the system administrator needs to upgrade the computers for his division. He wants to know what sort of computer system his workers prefer. He gives three choices: Windows, Mac, or Linux. Test the hypothesis or theory that an equal percentage of the population prefers each type of computer system.

Step 1: Creation of the data.



	A	B	C	D	E	F	G	H	I	J	K
1	System	O	Ei								
2	Windows	20	33.33%								
3	Mac	60	33.33%								
4	Linux	20	33.33%								
5	Total	100	100%								
6											
7											

Step 2: Setting the value of H0 and HA

H0: The population distribution of the variable is the same as the proposed distribut

HA: The distributions are different to calculate the Chi – Square value for Windows go to cell D2 and type

Step 3:

To calculate the Chi – Square value for Windows go to cell D2 and type = ((B2-C2)*(B2-C2))/C2

Step: 4

To calculate the Chi – Square value for Mac go to cell D3 and type = ((B3-C3)*(B3-C3))/C3

Step: 5

To calculate the Chi – Square value for Mac go to cell D3 and type

= ((B4-C4)*(B4-C4))/C4

Step: 6

Go to Cell D5 for and type =SUM(D2:D4) To get the table value for Chi-Square for $\alpha = 0.05$ and dof = 2,

Step: 7

Go to cell D7 and type =CHIINV(0.05,2) At cell D8 type =IF(D5>D7, "H0

Accepted", "H0 Rejected"

OUTPUT:

FILE HOME INSERT PAGE LAYOUT FORMULAS DATA REVIEW

Clipboard: Paste, Copy, Format Painter

Font: Calibri, 11, Bold, Italic, Underline, Paragraph, Styles

Formulas: L29, X, Checkmark, fx

	A	B	C	D	E
1	System	O	Ei		
2	Windows	20	33.33%	1160.453312	
3	Mac	60	33.33%	10681.41341	
4	Linux	20	33.33%	1160.453312	
5	Total	100	100%	13002.32003	
6					
7			Total Values	5.991464547	
8				H0 Accepted	
9					
10					
11					

DATA SCIENCE



SAKET GYANPEETH'S
SAKET COLLEGE OF ARTS, SCIENCE AND COMMERCE
(Permanently Affiliated to University of Mumbai)

NAAC Accredited B Grade

Saket Vidyanagari Marg, Chinchpada Road, Katemanivali, Kalyan (East) – 421306, Dist.

Department of Information Technology

Thane (MAH)

CERTIFICATE

This is to certify that

ANSHUMAN GAURISHANKAR SINGH of MSc Information Technology Part-I

Class has satisfactory carried out the required practical in the subject.

DATA SCIENCE

For the Academic year 2022 – 2023

Practical In-Charge

Head of Department

External Examiner

INDEX

Sr.No.	Aim	Sign
1	A) Csv To Hours B) Xml To Hours C) Json To Hours D) Mysql Database To Hours E) Picture (Jpeg) To Hours F) Video To Hours G) Audio To Hours	
2	A) Utilities And Auditing B) Data Binning And Bucketing C) Averaging Of Data D) Outlier Detection Audit	
3	A) Retrieve Superstep B) Program To Retrieve Different Attributes Of Data C) Data Pattern D) Loading Ip Data All	
4	Assessing Data A. Perform Error Management On The Given Data Using the Pandas Package. B. Write Python / R Program To Create The Network Routing Diagram From The Given Data on routers. C. Write A Python / R Program To Build a Directed Acyclic Graph.	
5	Processing Data A. Build The Time Hub, Links, And Satellites.	
6	Transforming Data	
7	Organizing Data	

PRACTICAL 1

A) CSV TO HOURS

CODE:

```
# Utility Start CSV to HORUS =====
# Standard Tools
import pandas as pd
# Input Agreement =====
sInputFileName='C:/VKHCG/05-DS/9999-Data/Country_Code.csv'
InputData=pd.read_csv(sInputFileName,encoding="latin-1")
print('Input Data Values =====')
print(InputData)
print('=====')
# Processing Rules =====
ProcessData=InputData
# Remove columns ISO-2-Code and ISO-3-CODE
ProcessData.drop('ISO-2-CODE', axis=1,inplace=True)
ProcessData.drop('ISO-3-Code', axis=1,inplace=True)
# Rename Country and ISO-M49
ProcessData.rename(columns={'Country': 'CountryName'}, inplace=True)
ProcessData.rename(columns={'ISO-M49': 'CountryNumber'}, inplace=True)
# Set new Index
ProcessData.set_index('CountryNumber', inplace=True)
# Sort data by CurrencyNumber
ProcessData.sort_values('CountryName', axis=0, ascending=False, inplace=True)
print('Process Data Values =====')
print(ProcessData)
print('=====')
# Output Agreement =====
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-CSV-Country.csv'
OutputData.to_csv(sOutputFileName, index = False)
print('CSV to HORUS - Done')
# Utility done =====
```

OUTPUT:

```

IDLE Shell 3.11.0
File Edit Shell Debug Options Window Help
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\VKHCG\05-DS\9999-Data\CSV2HORUS.py =====
Input Data Values
=====
Country ISO-2-CODE ISO-3-Code ISO-M49
0 Afghanistan AF AFG 4
1 Aland Islands AX ALA 248
2 Albania AL ALB 8
3 Algeria DZ DZA 12
4 American Samoa AS ASM 16
...
242 Wallis and Futuna Islands WF WLF 876
243 Western Sahara EH ESH 732
244 Yemen YE YEM 887
245 Zambia ZM ZMB 894
246 Zimbabwe ZW ZWE 716

[247 rows x 4 columns]
=====
Process Data Values
=====
CountryNumber CountryName
716 Zimbabwe
894 Zambia
887 Yemen
732 Western Sahara
876 Wallis and Futuna Islands
...
16 American Samoa
12 Algeria
8 Albania
248 Aland Islands
4 Afghanistan

[247 rows x 1 columns]
=====
CSV to HORUS - Done
>>>

```

B) XML TO HOURS

CODE:

```

# Utility Start XML to HORUS =====
# Standard Tools
import pandas as pd
import xml.etree.ElementTree as ET
def df2xml(data):
    header = data.columns
    root = ET.Element('root')
    for row in range(data.shape[0]):
        entry = ET.SubElement(root,'entry')
        for index in range(data.shape[1]):
            schild=str(header[index])
            child = ET.SubElement(entry, schild)
            if str(data[schild][row]) != 'nan':
                child.text = str(data[schild][row])
            else:
                child.text = 'n/a'
        entry.append(child)
    result = ET.tostring(root)
    return result
def xml2df(xml_data):
    root = ET.XML(xml_data)
    all_records = []
    for i, child in enumerate(root):
        record = {}

```

```

    for subchild in child:
        record[subchild.tag] = subchild.text
    all_records.append(record)
    return pd.DataFrame(all_records)

# Input Agreement =====
sInputFileName='C:/VKHCG/05-DS/9999-Data/Country_Code.xml'
InputData = open(sInputFileName).read()
print('=====')
print('Input Data Values =====')
print('=====')
print(InputData)
print('=====')
# Processing Rules =====
ProcessDataXML=InputData
# XML to Data Frame
ProcessData=xml2df(ProcessDataXML)
# Remove columns ISO-2-Code and ISO-3-CODE
ProcessData.drop('ISO-2-CODE', axis=1,inplace=True)
ProcessData.drop('ISO-3-Code', axis=1,inplace=True)
# Rename Country and ISO-M49
ProcessData.rename(columns={'Country': 'CountryName'}, inplace=True)
ProcessData.rename(columns={'ISO-M49': 'CountryNumber'}, inplace=True)
# Set new Index
ProcessData.set_index('CountryNumber', inplace=True)
# Sort data by CurrencyNumber
ProcessData.sort_values('CountryName', axis=0, ascending=False, inplace=True)
print('=====')
print('Process Data Values =====')
print('=====')
print(ProcessData)
print('=====')
# Output Agreement =====
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-XML-Country.csv'
OutputData.to_csv(sOutputFileName, index = False)
print('=====')
print('XML to HORUS - Done')
print('=====')
# Utility done =====

```

OUTPUT:

Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

```
===== RESTART: C:\VKHCG\05-DS\9999-Data\XML2HORUS.py =====
Input Data Values =====
Squeezed text (707 lines).
=====
Process Data Values =====
=====
CountryNumber      CountryName
716                Zimbabwe
894                Zambia
887                Yemen
732                Western Sahara
876                Wallis and Futuna Islands
...
16                American Samoa
12                Algeria
8                Albania
248               Aland Islands
4                Afghanistan

[247 rows x 1 columns]
=====
XML to HORUS - Done
=====
```

C) JSON TO HOURS

CODE:

```
# Utility Start JSON to HORUS =====
# Standard Tools
import pandas as pd
# Input Agreement =====
sInputFileName='C:/VKHCG/05-DS/9999-Data/Country_Code.json'
InputData=pd.read_json(sInputFileName,
                        orient='index',
                        encoding="latin-1")
print('Input Data Values =====')
print(InputData)
print('=====')
# Processing Rules =====
ProcessData=InputData
# Remove columns ISO-2-Code and ISO-3-CODE
ProcessData.drop('ISO-2-CODE', axis=1,inplace=True)
ProcessData.drop('ISO-3-Code', axis=1,inplace=True)
# Rename Country and ISO-M49
ProcessData.rename(columns={'Country': 'CountryName'}, inplace=True)
ProcessData.rename(columns={'ISO-M49': 'CountryNumber'}, inplace=True)
# Set new Index
ProcessData.set_index('CountryNumber', inplace=True)
# Sort data by CurrencyNumber
ProcessData.sort_values('CountryName', axis=0, ascending=False, inplace=True)
print('Process Data Values =====')
print(ProcessData)
print('=====')
```

```
# Output Agreement =====
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-JSON-Country.csv'
OutputData.to_csv(sOutputFileName, index = False)
print('JSON to HORUS - Done')
# Utility done =====
```

OUTPUT:

```
===== RESTART: C:\VKHCG\05-DS\9999-Data\JSON2HORUS.py =====
Input Data Values =====
Country ISO-2-CODE ISO-3-Code ISO-M49
0 Afghanistan AF AFG 4
1 Aland Islands AX ALA 248
2 Albania AL ALB 8
3 Algeria DZ DZA 12
4 American Samoa AS ASM 16
...
242 Wallis and Futuna Islands WF WLF 876
243 Western Sahara EH ESH 732
244 Yemen YE YEM 887
245 Zambia ZM ZMB 894
246 Zimbabwe ZW ZWE 716

[247 rows x 4 columns]
=====
Process Data Values =====
CountryNumber CountryName
716 Zimbabwe
894 Zambia
887 Yemen
732 Western Sahara
876 Wallis and Futuna Islands
...
16 American Samoa
12 Algeria
8 Albania
248 Aland Islands
4 Afghanistan

[247 rows x 1 columns]
=====
JSON to HORUS - Done
```

D) MYSQL DATABASE TO HOURS

CODE:

```
# Utility Start Database to HORUS =====
# Standard Tools
import pandas as pd
import sqlite3 as sq
# Input Agreement =====
sInputFileName='C:/VKHCG/05-DS/9999-Data/utility.db'
sInputTable='Country_Code'
conn = sq.connect(sInputFileName)
sSQL='select * FROM ' + sInputTable + ';'
InputData=pd.read_sql_query(sSQL, conn)
print('Input Data Values =====')
print(InputData)
print('=====')
# Processing Rules =====
ProcessData=InputData
# Remove columns ISO-2-Code and ISO-3-CODE
ProcessData.drop('ISO-2-CODE', axis=1,inplace=True)
ProcessData.drop('ISO-3-Code', axis=1,inplace=True)
# Rename Country and ISO-M49
ProcessData.rename(columns={'Country': 'CountryName'}, inplace=True)
```

Seat No.: 3254185

```
ProcessData.rename(columns={'ISO-M49': 'CountryNumber'}, inplace=True)
# Set new Index
ProcessData.set_index('CountryNumber', inplace=True)
# Sort data by CurrencyNumber
ProcessData.sort_values('CountryName', axis=0, ascending=False, inplace=True)
print('Process Data Values =====')
print(ProcessData)
print('=====')
# Output Agreement =====
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-CSV-Country.csv'
OutputData.to_csv(sOutputFileName, index = False)
print('Database to HORUS - Done')
# Utility done =====
```

OUTPUT:

```
===== RESTART: C:\VKHCG\05-DS\9999-Data\DATABASE2HORUS.py =====
Input Data Values =====
index      Country ISO-2-CODE ISO-3-Code ISO-M49
0          0      Afghanistan      AF      AFG      4
1          1      Aland Islands      AX      ALA     248
2          2      Albania          AL      ALB      8
3          3      Algeria          DZ      DZA     12
4          4      American Samoa    AS      ASM     16
...      ...
242      242      Wallis and Futuna Islands      WF      WLF     876
243      243      Western Sahara      EH      ESH     732
244      244      Yemen              YE      YEM     887
245      245      Zambia              ZM      ZMB     894
246      246      Zimbabwe            ZW      ZWE     716

[247 rows x 5 columns]
=====
Process Data Values =====
index      CountryName
CountryNumber
716      246      Zimbabwe
894      245      Zambia
887      244      Yemen
732      243      Western Sahara
876      242      Wallis and Futuna Islands
...      ...
16       4      American Samoa
12       3      Algeria
8        2      Albania
248     1      Aland Islands
4        0      Afghanistan

[247 rows x 2 columns]
=====
Database to HORUS - Done
```

E) PICTURE (JPEG) TO HOURS

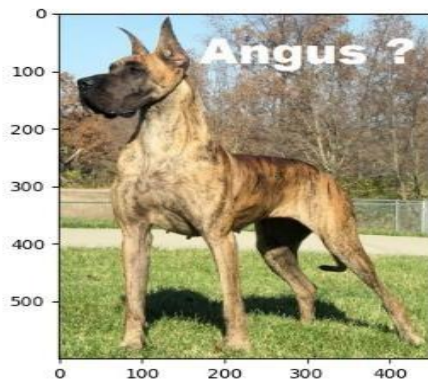
CODE:

```
# Utility Start Picture to HORUS =====
# Standard Tools
import sys
import os
#from scipy.misc import imread
from skimage import io
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
if sys.platform == 'linux':
    Base=os.path.expanduser('~') + '/VKHCG'
```

else:

```
    Base='C:/VKHCG'
    print('#####')
    print('Working Base :',Base, ' using ', sys.platform)
    print('#####')
    # Input Agreement =====
    sInputFileName=Base + '/05-DS/9999-Data/Angus.jpg'
    InputData = io.imread(sInputFileName, as_gray=False, pilmode='RGBA')
    print('Input Data Values =====')
    print('X: ',InputData.shape[0])
    print('Y: ',InputData.shape[1])
    print('RGBA: ', InputData.shape[2])
    print('=====')
    # Processing Rules =====
    ProcessRawData=InputData.flatten()
    y=InputData.shape[2] + 2
    x=int(ProcessRawData.shape[0]/y)
    ProcessData=pd.DataFrame(np.reshape(ProcessRawData, (x, y)))
    sColumns= ['XAxis','YAxis','Red', 'Green', 'Blue','Alpha']
    ProcessData.columns=sColumns
    ProcessData.index.names =['ID']
    print('Rows: ',ProcessData.shape[0])
    print('Columns :',ProcessData.shape[1])
    print('=====')
    print('Process Data Values =====')
    print('=====')
    plt.imshow(InputData)
    plt.show()
    print('=====')
    # Output Agreement =====
    OutputData=ProcessData
    print('Storing File')
    sOutputFileName=Base + '/05-DS/9999-Data/HORUS-Picture.csv'
    OutputData.to_csv(sOutputFileName, index = False)
    print('=====')
    print('Picture to HORUS - Done')
    print('=====')
    # Utility done =====
```

OUTPUT:



F) VIDEO TO HOURS

CODE:

MOVIE TO FRAMES

```
# Utility Start Movie to HORUS (Part 1) =====
# Standard Tools
import os
import shutil
import cv2
#=====
sInputFileName='C:/VKHCG/05-DS/9999-Data/dog.mp4'
sDataBaseDir='C:/VKHCG/05-DS/9999-Data/temp'
if os.path.exists(sDataBaseDir):
    shutil.rmtree(sDataBaseDir)
if not os.path.exists(sDataBaseDir):
    os.makedirs(sDataBaseDir)
print('=====')
print('Start Movie to Frames')
print('=====')
vidcap = cv2.VideoCapture(sInputFileName)
success,image = vidcap.read()
count = 0
while success:
    success,image = vidcap.read()
    sFrame=sDataBaseDir + str('/dog-frame-' + str(format(count, '04d')) + '.jpg')
    print('Extracted: ', sFrame)
    cv2.imwrite(sFrame, image)
    if os.path.getsize(sFrame) == 0:
        count += -1
        os.remove(sFrame)
        print('Removed: ', sFrame)
    if cv2.waitKey(10) == 27: # exit if Escape is hit
```

```

break
if count > 100: # exit
    break
count += 1
print('=====')
print('Generated : ', count, ' Frames')
print('=====')
print('Movie to Frames HORUS - Done')
print('=====')
# Utility done =====

```

OUTPUT:

```

Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0067.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0068.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0069.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0070.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0071.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0072.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0073.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0074.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0075.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0076.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0077.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0078.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0079.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0080.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0081.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0082.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0083.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0084.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0085.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0086.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0087.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0088.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0089.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0090.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0091.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0092.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0093.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0094.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0095.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0096.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0097.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0098.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0099.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0100.jpg
Extracted: C:/VKHCG/05-DS/9999-Data/temp/dog-frame-0101.jpg
=====
Generated : 101 Frames
=====
Movie to Frames HORUS - Done
=====

```

FRAMES TO HOURS:

CODE:

```

# Utility Start Movie to HORUS (Part 2)
===== # Standard Tools

from scipy.misc import imread
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os

# Input Agreement =====
sDataBaseDir='C:/VKHCG/05-DS/9999-Data/temp'
f=0
for file in os.listdir(sDataBaseDir):
    if file.endswith(".jpg"):
        f += 1
sInputFileName=os.path.join(sDataBaseDir, file)

```

```

print('Process : ', sInputFileName)
InputData = imread(sInputFileName, flatten=False, mode='RGBA')
print('Input Data Values=====')
print('X: ',InputData.shape[0])
print('Y:',InputData.shape[1])
print('RGBA: ', InputData.shape[2])
print('=====')
# Processing Rules =====
ProcessRawData=InputData.flatten()
y=InputData.shape[2] + 2
x=int(ProcessRawData.shape[0]/y)
ProcessFrameData=pd.DataFrame(np.reshape(ProcessRawData, (x, y)))
ProcessFrameData['Frame']=file
print('=====')
print('Process Data Values =====')
print('=====')
plt.imshow(InputData) plt.show()
if f == 1:
    ProcessData=ProcessFrameData
else:
    ProcessData=ProcessData.append(ProcessFrameData)
if f > 0:
    sColumns= ['XAxis','YAxis','Red', 'Green', 'Blue','Alpha','FrameName']
    ProcessData.columns=sColumns
    print('=====')
    ProcessFrameData.index.names =['ID']
    print('Rows: ',ProcessData.shape[0])
    print('Columns :',ProcessData.shape[1])
    print('=====')
# Output Agreement
=====
    OutputData=ProcessData
    print('Storing File')
    sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-Movie-Frame.csv'
    OutputData.to_csv(sOutputFileName, index = False)
    print('=====')
    print('Processed ; ', f,' frames')

    print('=====')
    print('Movie to HORUS - Done')
    print('=====')

```

OUTPUT:

```
=====
Rows: 15667200
Columns : 7
=====
Storing File
=====
Processed ; 102 frames
=====
Movie to HORUS - Done
=====
```



dog-frame-0000.jpeg



dog-frame-0001.jpeg



dog-frame-0100.jpeg



dog-frame-0101.jpeg

G) AUDIO TO HOURS

CODE:

```
# Utility Start Audio to HORUS =====
# Standard Tools
#=====

from scipy.io import wavfile
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
#=====

def show_info(aname, a,r):
    print('.....')
    print("Audio:", aname)
    print('.....')
    print("Rate:", r)
    print('.....')
    print("shape:", a.shape)
    print("dtype:", a.dtype)
    print("min, max:", a.min(), a.max())
    print('.....')
    plot_info(aname, a,r)
#=====

def plot_info(aname, a,r):
```

```

sTitle= 'Signal Wave - '+ aname + ' at ' + str(r) + 'hz'
plt.title(sTitle)
sLegend=[]
for c in range(a.shape[1]):
    sLabel = 'Ch' + str(c+1)
    sLegend=sLegend+[str(c+1)]
    plt.plot(a[:,c], label=sLabel)
plt.legend(sLegend)
plt.show()

#=====
sInputFileName='C:/VKHCG/05-DS/9999-Data/2ch-sound.wav'
print('=====')
print('Processing : ', sInputFileName)
print('=====')
InputRate, InputData = wavfile.read(sInputFileName)
show_info("2 channel", InputData,InputRate)
ProcessData=pd.DataFrame(InputData)
sColumns= ['Ch1','Ch2']
ProcessData.columns=sColumns
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-Audio-2ch.csv'
OutputData.to_csv(sOutputFileName, index = False)

#=====
sInputFileName='C:/VKHCG/05-DS/9999-Data/4ch-sound.wav'
print('=====')
print('Processing : ', sInputFileName)
print('=====')
InputRate, InputData = wavfile.read(sInputFileName)
show_info("4 channel", InputData,InputRate)
ProcessData=pd.DataFrame(InputData)
sColumns= ['Ch1','Ch2','Ch3', 'Ch4']
ProcessData.columns=sColumns
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-Audio-4ch.csv'
OutputData.to_csv(sOutputFileName, index = False)

#=====
sInputFileName='C:/VKHCG/05-DS/9999-Data/6ch-sound.wav'
print('=====')
print('Processing : ', sInputFileName)
print('=====')
InputRate, InputData = wavfile.read(sInputFileName)
show_info("6 channel", InputData,InputRate)
ProcessData=pd.DataFrame(InputData)
sColumns= ['Ch1','Ch2','Ch3', 'Ch4', 'Ch5','Ch6']

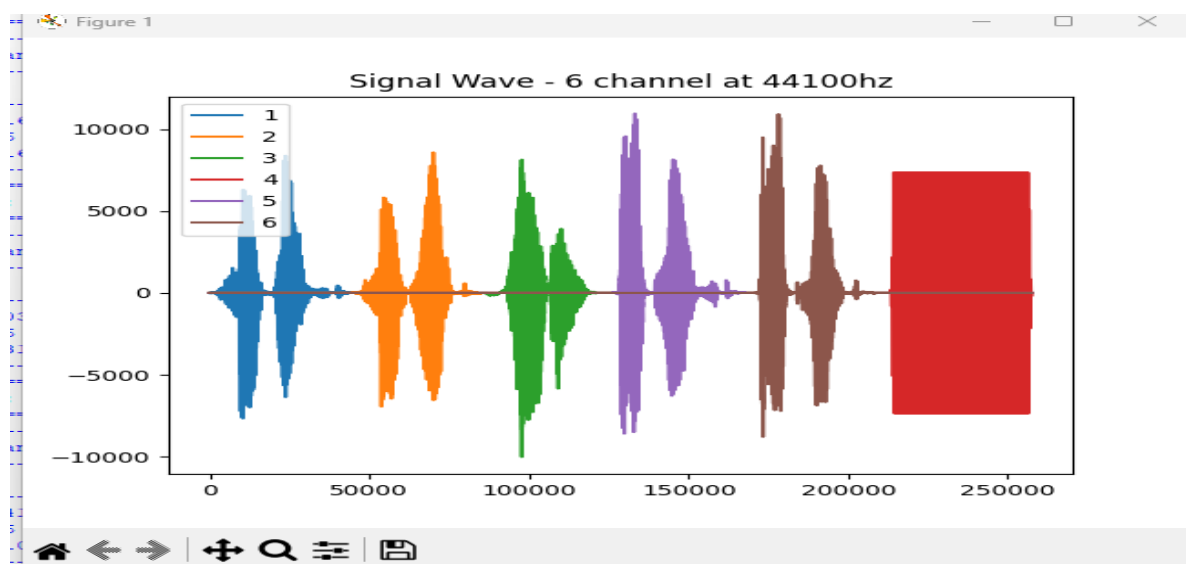
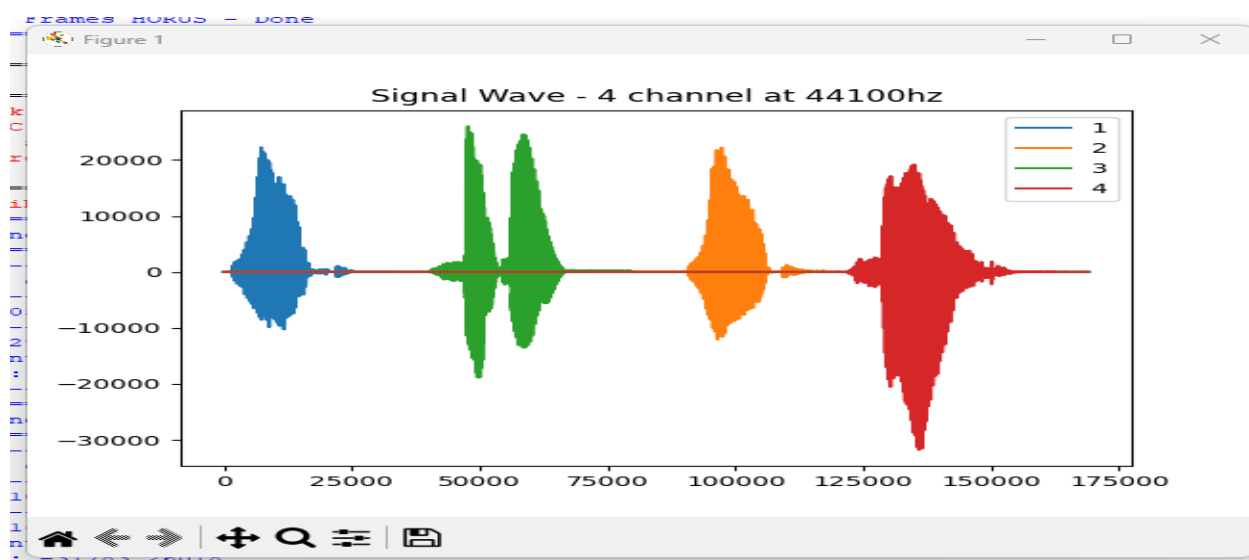
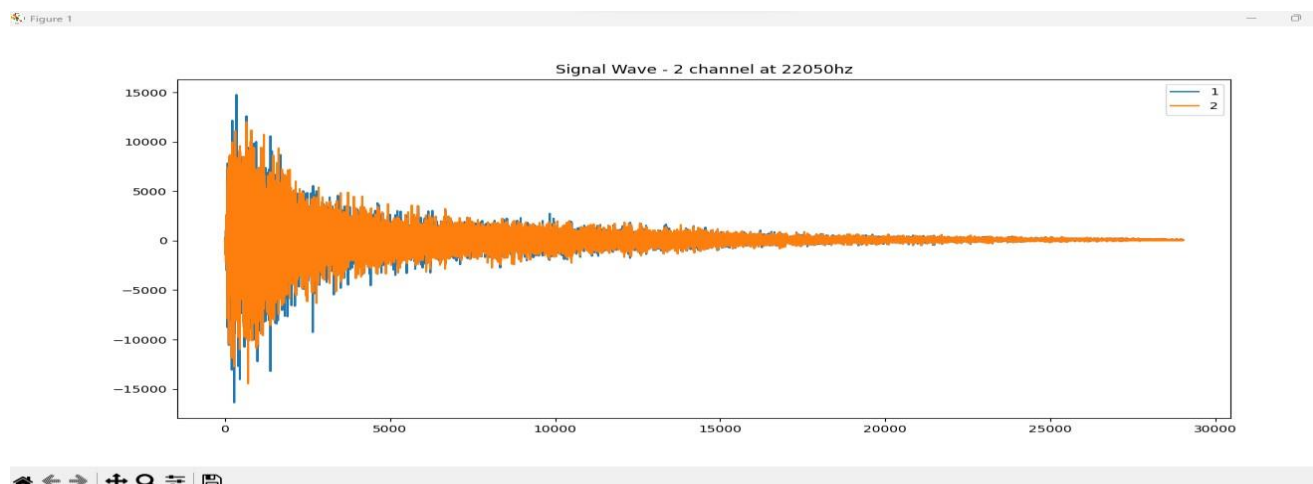
```

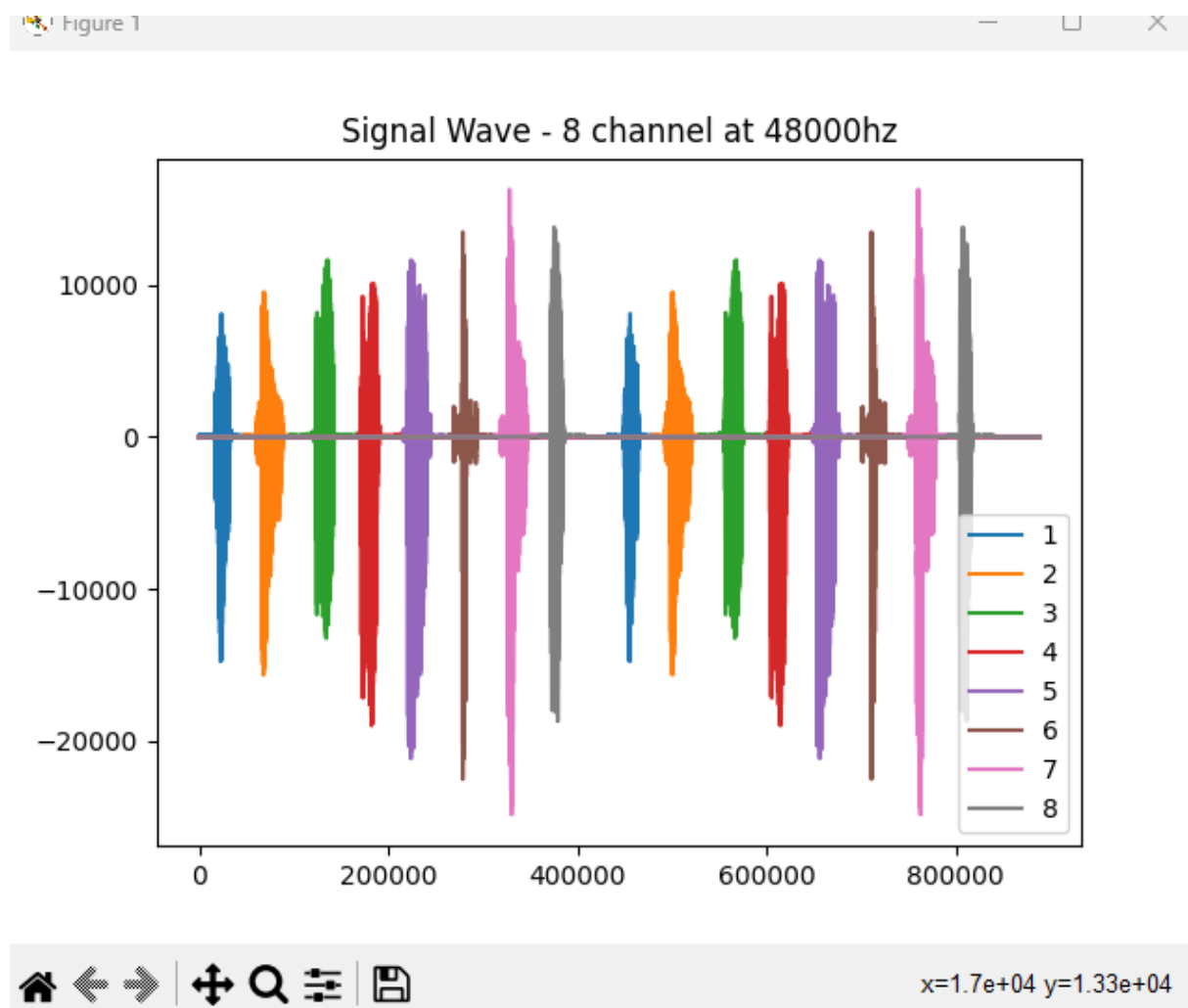
Seat No.: 3254185

```
ProcessData.columns=sColumns
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-Audio-6ch.csv'
OutputData.to_csv(sOutputFileName, index = False)
#=====
sInputFileName='C:/VKHCG/05-DS/9999-Data/8ch-sound.wav'
print('=====')
print('Processing : ', sInputFileName)
print('=====')
InputRate, InputData = wavfile.read(sInputFileName)
show_info("8 channel", InputData,InputRate)
ProcessData=pd.DataFrame(InputData)
sColumns= ['Ch1','Ch2','Ch3', 'Ch4', 'Ch5','Ch6','Ch7','Ch8']
ProcessData.columns=sColumns
OutputData=ProcessData
sOutputFileName='C:/VKHCG/05-DS/9999-Data/HORUS-Audio-8ch.csv'
OutputData.to_csv(sOutputFileName, index = False)
print('=====')
print('Audio to HORUS - Done')
print('=====')
#=====
# Utility done =====
#=====
```

OUTPUT:

```
===== RESTART: C:\VKHCG\05-DS\9999-Data\AUDIO2HORUS.py =====
Matplotlib is building the font cache; this may take a moment.
=====
Processing : C:/VKHCG/05-DS/9999-Data/2ch-sound.wav
=====
-----
Audio: 2 channel
-----
Rate: 22050
-----
shape: (29016, 2)
dtype: int16
min, max: -16384 14767
-----
```





Audio to HORUS – Done

PRACTICAL 2**A) UTILITIES AND AUDITING****CODE:**

```
#----- Program to Demonstrate Fixers utilities -----
import string
import datetime as dt
# 1 Removing leading or lagging spaces from a data entry
print('#1 Removing leading or lagging spaces from a data entry');
baddata = " Data Science with too many spaces is bad!!! "
print('>',baddata,<')
cleandata=baddata.strip()
print('>',cleandata,<')
# 2 Removing nonprintable characters from a data entry
print('#2 Removing nonprintable characters from a data entry')
printable = set(string.printable)
baddata = "Data\x00Science with\x02 funny characters is \x10bad!!!"
cleandata="".join(filter(lambda x: x in string.printable,baddata))
print('Bad Data : ',baddata);
print('Clean Data : ',cleandata)
# 3 Reformatting data entry to match specific formatting criteria.
# Convert YYYY/MM/DD to DD Month YYYY
print('# 3 Reformatting data entry to match specific formatting criteria.')
baddate = dt.date(2019, 10, 31)
baddata=format(baddate,'%Y-%m-%d')
gooddate = dt.datetime.strptime(baddata,'%Y-%m-%d')
gooddata=format(gooddate,'%d %B %Y')
print('Bad Data : ',baddata)
print('Good Data : ',gooddata)
```

OUTPUT:

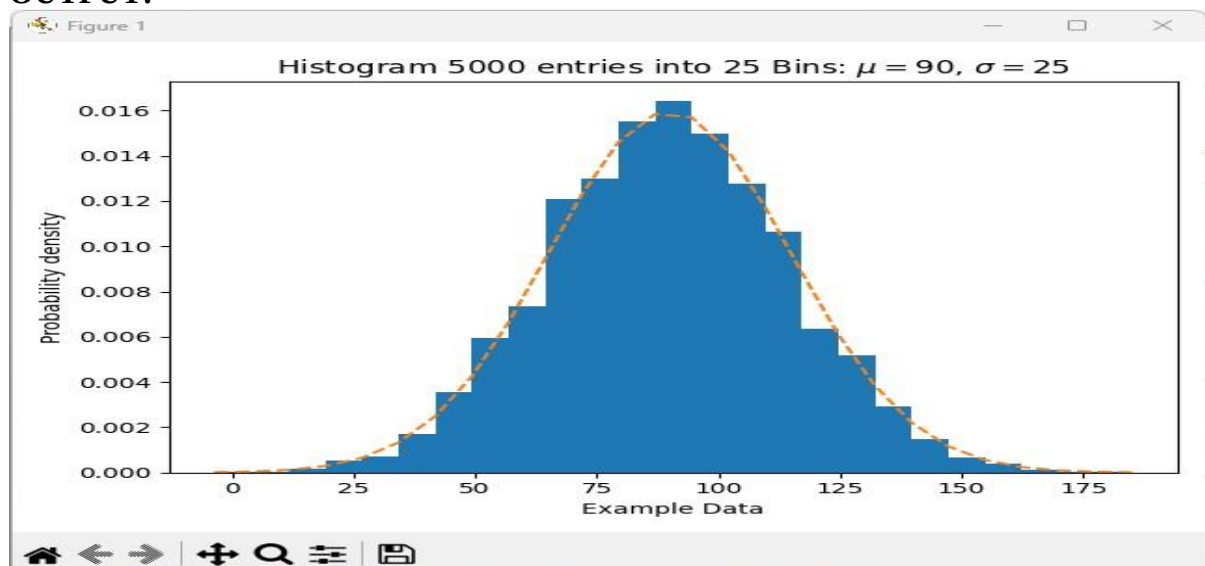
```
===== RESTART: C:\VKHCG\05-DS\4000-UL\0200-DU\Fixers Utilities.py =====
#1 Removing leading or lagging spaces from a data entry
> Data Science with too many spaces is bad!!! <
> Data Science with too many spaces is bad!!! <
#2 Removing nonprintable characters from a data entry
Bad Data : Data
Clean Data : DataScience with funny characters is bad!!!
# 3 Reformatting data entry to match specific formatting criteria.
Bad Data : 2019-10-31
Good Data : 31 October 2019
```

B) Data Binning and Bucketing**CODE:**

```

import numpy as np
import matplotlib.mlab as mlab
import matplotlib.pyplot as plt
import scipy.stats as stats
np.random.seed(0)
# example data
mu = 90 # mean of distribution
sigma = 25 # standard deviation of distribution
x = mu + sigma * np.random.randn(5000)
num_bins = 25
fig, ax = plt.subplots()
# the histogram of the data
n, bins, patches = ax.hist(x, num_bins, density=1)
# add a 'best fit' line
y = stats.norm.pdf(bins, mu, sigma)
# mlab.normpdf(bins, mu, sigma)
ax.plot(bins, y, '--')
ax.set_xlabel('Example Data')
ax.set_ylabel('Probability density')
sTitle=r'Histogram ' + str(len(x)) + ' entries into ' + str(num_bins) + ' Bins:  $\mu =$  ' + str(mu) + '$,
 $\sigma =$  ' + str(sigma) + '$'
ax.set_title(sTitle)
fig.tight_layout()
sPathFig='C:/VKHCG/05-DS/4000-UL/0200-DU/DU-Histogram.png'
fig.savefig(sPathFig)
plt.show()

```

OUTPUT:

C. Averaging Of Data**CODE:**

```
import pandas as pd
InputFileName='IP_DATA_CORE.csv'
OutputFileName='Retrieve_Router_Location.csv'
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ')
print('#####')
sFileName=Base + '/01-Vermeulen/00-RawData/' + InputFileName
print('Loading :',sFileName)
IP_DATA_ALL=pd.read_csv(sFileName,header=0,low_memory=False,
usecols=['Country','Place Name','Latitude','Longitude'], encoding="latin-1")
IP_DATA_ALL.rename(columns={'Place Name': 'Place_Name'}, inplace=True)
AllData=IP_DATA_ALL[['Country','Place_Name','Latitude']]
print(AllData)
MeanData=AllData.groupby(['Country','Place_Name'])['Latitude'].mean()
print(MeanData)
```

OUTPUT:

```
#####
Working Base : C:/VKHCG using
#####
Loading : C:/VKHCG/01-Vermeulen/00-RawData/IP_DATA_CORE.csv
  Country Place_Name  Latitude
0      US    New York   40.7528
1      US    New York   40.7528
2      US    New York   40.7528
3      US    New York   40.7528
4      US    New York   40.7528
...      ...      ...
3557    DE      Munich   48.0915
3558    DE      Munich   48.1833
3559    DE      Munich   48.1000
3560    DE      Munich   48.1480
3561    DE      Munich   48.1480

[3562 rows x 3 columns]
Country Place_Name  Latitude
DE      Munich      48.143223
GB      London      51.509406
US      New York     40.747044
Name: Latitude, dtype: float64
```

D. Outlier Detection**CODE:**

```
import pandas as pd
InputFileName='IP_DATA_CORE.csv'
OutputFileName='Retrieve_Router_Location.csv'
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base)
print('#####')
sFileName=Base + '/01-Vermeulen/00-RawData/' + InputFileName
```

```

print('Loading :',sFileName)
IP_DATA_ALL=pd.read_csv(sFileName,header=0,low_memory=False,
usecols=['Country','Place Name','Latitude','Longitude'], encoding="latin-1")
IP_DATA_ALL.rename(columns={'Place Name': 'Place_Name'}, inplace=True)
LondonData=IP_DATA_ALL.loc[IP_DATA_ALL['Place_Name']=='London']
AllData=LondonData[['Country', 'Place_Name','Latitude']]
print('All Data')
print(AllData)
MeanData=AllData.groupby(['Country', 'Place_Name'])['Latitude'].mean()
StdData=AllData.groupby(['Country', 'Place_Name'])['Latitude'].std()
print('Outliers')
UpperBound=float(MeanData+StdData)
print('Higher than ', UpperBound)
OutliersHigher=AllData[AllData.Latitude>UpperBound]
print(OutliersHigher)
LowerBound=float(MeanData-StdData)
print('Lower than ', LowerBound)
OutliersLower=AllData[AllData.Latitude<LowerBound]
print(OutliersLower)
print('Not Outliers')
OutliersNot=AllData[(AllData.Latitude>=LowerBound) & (AllData.Latitude<=UpperBound)]
Print(OutliersNot)

```

OUTPUT:

```

#####
Loading : C:/VIGSCG/01-Vermeulen/00-RawData/IP_DATA_CORE.csv
All Data
  Country Place_Name Latitude
1910    GB    London    51.5130
1911    GB    London    51.5508
1912    GB    London    51.5649
1913    GB    London    51.5895
1914    GB    London    51.5232
...
3434    GB    London    51.5092
3435    GB    London    51.5092
3436    GB    London    51.5163
3437    GB    London    51.5085
3438    GB    London    51.5136

[1502 rows x 3 columns]
Outliers
Higher than 51.512635507867415
  Country Place_Name Latitude
1910    GB    London    51.5130
1911    GB    London    51.5508
1912    GB    London    51.5649
1913    GB    London    51.5895
1914    GB    London    51.5232
1916    GB    London    51.5491
1919    GB    London    51.5161
1920    GB    London    51.5198
1921    GB    London    51.5198
1923    GB    London    51.5237
1924    GB    London    51.5237
1925    GB    London    51.5237
1926    GB    London    51.5237
1927    GB    London    51.5232
3436    GB    London    51.5163
3438    GB    London    51.5136
Lower than 51.506176875621264
  Country Place_Name Latitude
1915    GB    London    51.4735
Not Outliers
  Country Place_Name Latitude
1917    GB    London    51.5085
1918    GB    London    51.5085
1922    GB    London    51.5085
1928    GB    London    51.5085
1929    GB    London    51.5085
...
3432    GB    London    51.5092
3433    GB    London    51.5092
3434    GB    London    51.5092
3435    GB    London    51.5092
3437    GB    London    51.5085

[1485 rows x 3 columns]

```

Audit

CODE:

```
import sys
import os
import logging
import uuid
import shutil
import time
Base='C:/VKHCG'
sCompanies=['01-Vermeulen','02-Krennwallner','03-Hillman','04-Clark']
sLayers=['01-Retrieve','02-Assess','03-Process','04-Transform','05-Organise','06-Report']
sLevels=['debug','info','warning','error']
for sCompany in sCompanies:
    sFileDir=Base + '/' + sCompany
    if not os.path.exists(sFileDir):
        os.makedirs(sFileDir)
    for sLayer in sLayers:
        log = logging.getLogger() # root logger
        for hdlr in log.handlers[:]: # remove all old handlers
            log.removeHandler(hdlr)
        sFileDir=Base + '/' + sCompany + '/' + sLayer + '/Logging'
        if os.path.exists(sFileDir):
            shutil.rmtree(sFileDir)
        time.sleep(2)
        if not os.path.exists(sFileDir):
            os.makedirs(sFileDir)
skey=str(uuid.uuid4())
sLogFile=Base + '/' + sCompany + '/' + sLayer + '/Logging/Logging_'+skey+'.log'
print('Set up:',sLogFile)
# set up logging to file - see previous section for more details
logging.basicConfig(level=logging.DEBUG,
    format='%(asctime)s %(name)-12s %(levelname)-8s %(message)s',
    datefmt='%m-%d %H:%M',
    filename=sLogFile,
    filemode='w')
# define a Handler which writes INFO messages or higher to the sys.stderr
console = logging.StreamHandler()
console.setLevel(logging.INFO)
# set a format which is simpler for console use
formatter = logging.Formatter('%(name)-12s: %(levelname)-8s %(message)s')
# tell the handler to use this format
console.setFormatter(formatter)
# add the handler to the root logger
logging.getLogger("").addHandler(console)
```

Now, we can log to the root logger, or any other logger. First the root...

```
logging.info('Practical Data Science is fun!')
```

```
for sLevel in sLevels:
```

```
    sApp='Appllication-'+ sCompany + '-' + sLayer + '-' + sLevel
```

```
    logger = logging.getLogger(sApp)
```

```
    if sLevel == 'debug':
```

```
        logger.debug('Practical Data Science logged a debugging message.')
```

```
    if sLevel == 'info':
```

```
        logger.info('Practical Data Science logged information message.')
```

```
    if sLevel == 'warning':
```

```
        logger.warning('Practical Data Science logged a warning message.')
```

```
    if sLevel == 'error':
```

```
        logger.error('Practical Data Science logged an error message.')
```

OUTPUT:

```
=====
Set up: C:/VKHCG/04-Clark/06-Report/Logging/Logging_5fcdd601-23ae-47fe-9f5a-3d1700d4f049.log
root      : INFO    Practical Data Science is fun!.
Appllication-04-Clark-06-Report-error: ERROR    Practical Data Science logged an error message.
>
```

PRACTICAL 3**A Retrieve Superstep**

The Retrieve superstep is a practical method for importing completely into the processing ecosystem a data lake consisting of various external data sources. The Retrieve superstep is the first contact between your data science and the source systems. I will guide you through a methodology of how to handle this discovery of the data up to the point you have all the data you need to evaluate the system you are working with, by deploying your data science skills. The successful retrieval of the data is a major stepping-stone to ensuring that you are performing good data science. Data lineage delivers the audit trail of the data elements at the lowest granular level, to ensure full data governance.

Data tagged in respective analytical models define the profile of the data that requires loading and guides the data scientist to what additional processing is required.

```
library(readr)
IP_DATA_ALL <- read_csv("C:/VKHCG/01-Vermeulen/00-RawData/IP_DATA_ALL.csv")
cols(
  ID = col_double(),
  Country = col_character(),
  `Place Name` = col_character(),
  `Post Code` = col_double(),
  Latitude = col_double(),
  Longitude = col_double(),
  `First IP Number` = col_double(),
  `Last IP Number` = col_double()
);
View(IP_DATA_ALL)
spec(IP_DATA_ALL)
cols(
  ID = col_double(),
  Country = col_character(),
  `Place Name` = col_character(),
  `Post Code` = col_double(),
  Latitude = col_double(),
  Longitude = col_double(),
  `First IP Number` = col_double(),
  `Last IP Number` = col_double()
);
library(tibble)
set_tidy_names(IP_DATA_ALL, syntactic = TRUE, quiet = FALSE)
Place Name -> Place.Name
Post Code -> Post.Code
First IP Number -> First.IP.Number
Last IP Number -> Last.IP.Number
supply(IP_DATA_ALL_FIX, typeof)
```

```

library(data.table)
hist_country=data.table(Country=unique(IP_DATA_ALL_FIX[is.na(IP_DATA_ALL_FIX
['Country'])==0,]$Country))
setorder(hist_country,'Country')
hist_country_with_id=rowid_to_column(hist_country, var = "RowIDCountry")
View(hist_country_fix)
IP_DATA_COUNTRY_FREQ=data.table(with(IP_DATA_ALL_FIX, table(Country)))
View(IP_DATA_COUNTRY_FREQ)

```

OUTPUT:

ID	Country	Place.Name	Post.Code	Latitude	Longitude	First.IP.Number	Last.IP.Number
1	BW	Gaborone	NA	-24.6464	25.9119	692781056	692781567
2	BW	Gaborone	NA	-24.6464	25.9119	692781824	692783103
3	BW	Gaborone	NA	-24.6464	25.9119	692909056	692909311
4	BW	Gaborone	NA	-24.6464	25.9119	692909568	692910079
5	BW	Gaborone	NA	-24.6464	25.9119	693051392	693052415
6	BW	Gaborone	NA	-24.6464	25.9119	693078272	693078527
7	BW	Gaborone	NA	-24.6464	25.9119	693608448	693616639
8	BW	Gaborone	NA	-24.6464	25.9119	696929792	696930047
9	BW	Gaborone	NA	-24.6464	25.9119	700438784	700439039
10	BW	Gaborone	NA	-24.6464	25.9119	702075904	702076927

B: Program to retrieve different attributes of data

```

import sys
import os
import pandas as pd
Base='C:/VKHCG'
sFileName=Base + '/01-Vermeulen/00-RawData/IP_DATA_ALL.csv'
print('Loading:',sFileName)
IP_DATA_ALL=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
sFileDir=Base + '/01-Vermeulen/01-Retrieve/01-EDS/02-Python'
if not os.path.exists(sFileDir):
    os.makedirs(sFileDir)
print('Rows:', IP_DATA_ALL.shape[0])
print('Columns:', IP_DATA_ALL.shape[1])
print('### Raw Data Set #####')
for i in range(0,len(IP_DATA_ALL.columns)):
    print(IP_DATA_ALL.columns[i],type(IP_DATA_ALL.columns[i]))
print('### Fixed Data Set #####')
IP_DATA_ALL_FIX=IP_DATA_ALL
for i in range(0,len(IP_DATA_ALL.columns)):
    cNameOld=IP_DATA_ALL_FIX.columns[i] + ''
    cNameNew=cNameOld.strip().replace(" ", ".")
    IP_DATA_ALL_FIX.columns.values[i] = cNameNew

```

```

print(IP_DATA_ALL.columns[i],type(IP_DATA_ALL.columns[i]))
#print(IP_DATA_ALL_FIX.head())
print('Fixed Data Set with ID')
IP_DATA_ALL_with_ID=IP_DATA_ALL_FIX
IP_DATA_ALL_with_ID.index.names = ['RowID']
#print(IP_DATA_ALL_with_ID.head())
sFileName2=sFileDir + '/Retrieve_IP_DATA.csv'
IP_DATA_ALL_with_ID.to_csv(sFileName2, index = True, encoding="latin-1")
Print('###Done!!#####')

```

OUTPUT:

```

===== REST
Loading : C:/VKHCG/01-Vermeulen/00-RawData/IP_DATA_ALL.csv
Rows: 1247502
Columns: 9
### Raw Data Set #####
Unnamed: 0 <class 'str'>
ID <class 'str'>
Country <class 'str'>
Place.Name <class 'str'>
Post.Code <class 'str'>
Latitude <class 'str'>
Longitude <class 'str'>
First.IP.Number <class 'str'>
Last.IP.Number <class 'str'>
### Fixed Data Set #####
Unnamed:.0 <class 'str'>
ID <class 'str'>
Country <class 'str'>
Place.Name <class 'str'>
Post.Code <class 'str'>
Latitude <class 'str'>
Longitude <class 'str'>
First.IP.Number <class 'str'>
Last.IP.Number <class 'str'>
Fixed Data Set with ID
### Done!! #####

```

C. DATA PATTERN

To determine a pattern of the data values, Replace all alphabet values with an uppercase case *A*, all numbers with an uppercase *N*, and replace any spaces with a lowercase letter *b* and all other unknown characters with a lowercase *u*. As a result, “Good Book 101” becomes AAAAbAAAAbNNNu”. This pattern creation is beneficial for designing any specific assess rules. This pattern view of data is a quick way to identify common patterns or determine standard layouts.

```

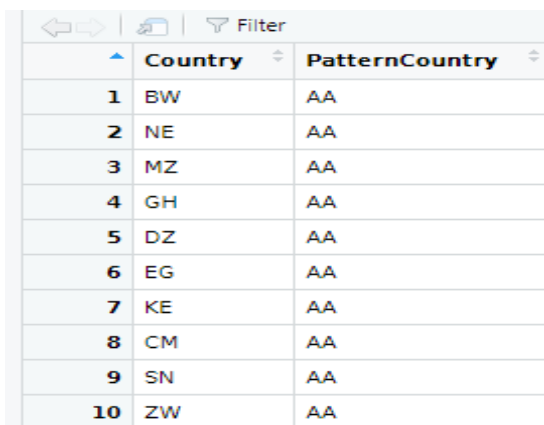
library(readr)
library(data.table)
FileName=paste0('c:/VKHCG/01-Vermeulen/00-RawData/IP_DATA_ALL.csv')
IP_DATA_ALL <- read_csv(FileName)
hist_country=data.table(Country=unique(IP_DATA_ALL$Country))
pattern_country=data.table(Country=hist_country$Country, PatternCountry=hist_country$Country)
oldchar=c(letters,LETTERS)
newchar=replicate(length(oldchar),"A")
for (r in seq(nrow(pattern_country))){
s=pattern_country[r,]$PatternCountry;
  for (c in seq(length(oldchar))){

```

```

s=chartr(oldchar[c],newchar[c],s)
};
for (n in seq(0,9,1)){
  s=chartr(as.character(n),"N",s)
};
s=chartr(" ", "b",s)
s=chartr(".", "u",s)
pattern_country[r,]$PatternCountry=s;
};
View(pattern_country)

```

OUTPUT:


	Country	PatternCountry
1	BW	AA
2	NE	AA
3	MZ	AA
4	GH	AA
5	DZ	AA
6	EG	AA
7	KE	AA
8	CM	AA
9	SN	AA
10	ZW	AA

Example 2: This is a common use of patterns to separate common standards and structures. Pattern can be loaded in separate retrieve procedures. If the same two patterns, NNNNuNNuNN and uuNNuNNuNN, are found, you can send NNNNuNNuNN directly to be converted into a date, while uuNNuNNuNN goes through a quality-improvement process to then route back to the same queue as NNNNuNNuNN, once it complies.

```

library(readr)
library(data.table)
Base='C:/VKHCG'
FileName=paste0(Base,'/01-Vermeulen/00-RawData/IP_DATA_ALL.csv')
IP_DATA_ALL <- read_csv(FileName)
hist_latitude=data.table(Latitude=unique(IP_DATA_ALL$Latitude))
pattern_latitude=data.table(latitude=hist_latitude$Latitude,
                             Patternlatitude=as.character(hist_latitude$Latitude))
oldchar=c(letters,LETTERS)
newchar=replicate(length(oldchar),"A")
for (r in seq(nrow(pattern_latitude))){
  s=pattern_latitude[r,]$Patternlatitude;
  for (c in seq(length(oldchar))){
    s=chartr(oldchar[c],newchar[c],s)
  };
}

```

Seat No.: 3254185

```
for (n in seq(0,9,1)){
  s=chartr(as.character(n),"N",s)
};
s=chartr(" ","b",s)
s=chartr("+","u",s)
s=chartr("-","u",s)
s=chartr(".", "u",s)
pattern_latitude[r,$Patternlatitude=s;
};
setorder(pattern_latitude,latitude)
View(pattern_latitude[1:3])
```

OUTPUT:

	latitude	Patternlatitude
1	-54.1561	uNNuNNNN
2	-51.7000	uNNuN
3	-49.3500	uNNuNN

D.Loading IP_DATA_ALL:

This data set contains all the IP address allocations in the world. It will help you to locate your customers when interacting with them online.

Create a new Python script file and save it as Retrieve-IP_DATA_ALL.py in directory

```
import sys
import os
import pandas as pd
Base='C:/VKHCG'
sFileName=Base + '/01-Vermeulen/00-RawData/IP_DATA_ALL.csv'
print('Loading:',sFileName)
IP_DATA_ALL=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
sFileDir=Base + '/01-Vermeulen/01-Retrieve/01-EDS/02-Python'
if not os.path.exists(sFileDir):
  os.makedirs(sFileDir)
print('Rows:', IP_DATA_ALL.shape[0])
print('Columns:', IP_DATA_ALL.shape[1])
print('### Raw Data Set #####')
for i in range(0,len(IP_DATA_ALL.columns)):
  print(IP_DATA_ALL.columns[i],type(IP_DATA_ALL.columns[i]))
```

```
print('### Fixed Data Set #####')
IP_DATA_ALL_FIX=IP_DATA_ALL
for i in range(0,len(IP_DATA_ALL.columns)):
    cNameOld=IP_DATA_ALL_FIX.columns[i] + ' '
    cNameNew=cNameOld.strip().replace(" ", ".")
    IP_DATA_ALL_FIX.columns.values[i] = cNameNew
print(IP_DATA_ALL.columns[i],type(IP_DATA_ALL.columns[i]))
print(IP_DATA_ALL_FIX.head())
print('Fixed Data Set with ID')
IP_DATA_ALL_with_ID=IP_DATA_ALL_FIX
IP_DATA_ALL_with_ID.index.names = ['RowID']
#print(IP_DATA_ALL_with_ID.head())
sFileName2=sFileDir + '/Retrieve_IP_DATA.csv'
IP_DATA_ALL_with_ID.to_csv(sFileName2, index = True, encoding="latin-1")
print('### Done!! #####')
```

OUTPUT:

```
----- RESTART: C:/VKHCG/01-Vermeulen/00-RawData/IP_
Loading : C:/VKHCG/01-Vermeulen/00-RawData/IP_DATA_ALL.csv
Rows: 1048575
Columns: 8
### Raw Data Set #####
ID <class 'str'>
Country <class 'str'>
Place.Name <class 'str'>
Post.Code <class 'str'>
Latitude <class 'str'>
Longitude <class 'str'>
First.IP.Number <class 'str'>
Last.IP.Number <class 'str'>
### Fixed Data Set #####
Last.IP.Number <class 'str'>
Fixed Data Set with ID
### Done!! #####
```

PRACTICAL 4**ASSESSING DATA****Assess Superstep**

Data quality refers to the condition of a set of qualitative or quantitative variables. Data quality is a multidimensional measurement of the acceptability of specific data sets. In business, data quality is measured to determine whether data can be used as a basis for reliable intelligence extraction for supporting organizational decisions. Data profiling involves observing in your data sources all the viewpoints that the information offers. The main goal is to determine if individual viewpoints are accurate and complete. The Assess superstep determines what additional processing to apply to the entries that are non-compliant.

Errors

Typically, one of four things can be done with an error to the data.

1. Accept the Error
2. Reject the Error
3. Correct the Error
4. Create a Default Value

A. Perform error management on the given data using the pandas package.

Python pandas package enables several automatic error-management features.

File Location: C:\VKHCG\01-Vermeulen\02-Assess

Missing Values in Pandas:**i. Drop the Columns Where All Elements Are Missing Values****CODE :**

```
##### Assess-Good-Bad-01.py#####
# -*- coding: utf-8 -*-
import sys
import os
import pandas as pd
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sInputFileName='Good-or-Bad.csv'
sOutputFileName='Good-or-Bad-01.csv'
Company='01-Vermeulen'
Base='C:/VKHCG'
sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir):
    os.makedirs(sFileDir)
### Import Warehouse
sFileName=Base + '/' + Company + '/00-RawData/' + sInputFileName
print('Loading :',sFileName)
RawData=pd.read_csv(sFileName,header=0)
print('#####')
print('## Raw Data Values')
print('#####')
```

```
print(RawData)
print('#####')
print('## Data Profile')
print('#####')
print('Rows :',RawData.shape[0])
print('Columns :',RawData.shape[1])
print('#####')
sFileName=sFileDir + '/' + sInputFileName
RawData.to_csv(sFileName, index = False)
TestData=RawData.dropna(axis=1, how='all')
print('#####')
print('## Test Data Values')
print('#####')
print(TestData)
print('#####')
print('## Data Profile')
print('#####')
print('Rows :',TestData.shape[0])
print('Columns :',TestData.shape[1])
print('#####')
sFileName=sFileDir + '/' + sOutputFileName
TestData.to_csv(sFileName, index = False)
print('#####')
print('### Done!! #####')
print('#####')
```

OUTPUT:

>>>

===== RESTART: C:\VKHCG\01-Vermeulen\02-Assess\Assess-Good-Bad-01.py =====

Working Base : C:\VKHCG using win32

Loading : C:\VKHCG\01-Vermeulen\00-RawData\Good-or-Bad.csv

Raw Data Values

ID FieldA FieldB FieldC FieldD FieldE FieldF FieldG

0 1.0 Good Better Best 1024.0 NaN 10241.0 1

1 2.0 Good NaN Best 512.0 NaN 5121.0 2

2 3.0 Good Better NaN 256.0 NaN 256.0 3

3 4.0 Good Better Best NaN NaN 211.0 4

4 5.0 Good Better NaN 64.0 NaN 6411.0 5

5 6.0 Good NaN Best 32.0 NaN 32.0 6

6 7.0 NaN Better Best 16.0 NaN 1611.0 7

7 8.0 NaN NaN Best 8.0 NaN 8111.0 8

8 9.0 NaN NaN NaN 4.0 NaN 41.0 9

9 10.0 A B C 2.0 NaN 21111.0 10

10 NaN NaN NaN NaN NaN NaN NaN 11

Seat No.: 3254185

```
11 10.0 Good Better Best 1024.0 NaN 102411.0 12
12 10.0 Good NaN Best 512.0 NaN 512.0 13
13 10.0 Good Better NaN 256.0 NaN 1256.0 14
14 10.0 Good Better Best NaN NaN NaN 15
15 10.0 Good Better NaN 64.0 NaN 164.0 16
16 10.0 Good NaN Best 32.0 NaN 322.0 17
17 10.0 NaN Better Best 16.0 NaN 163.0 18
18 10.0 NaN NaN Best 8.0 NaN 844.0 19
19 10.0 NaN NaN NaN 4.0 NaN 4555.0 20
20 10.0 A B C 2.0 NaN 111.0 21
```

Data Profile

Rows : 21

Columns : 8

Test Data Values

ID FieldA FieldB FieldC FieldD FieldF FieldG

```
0 1.0 Good Better Best 1024.0 10241.0 1
1 2.0 Good NaN Best 512.0 5121.0 2
2 3.0 Good Better NaN 256.0 256.0 3
3 4.0 Good Better Best NaN 211.0 4
4 5.0 Good Better NaN 64.0 6411.0 5
5 6.0 Good NaN Best 32.0 32.0 6
6 7.0 NaN Better Best 16.0 1611.0 7
7 8.0 NaN NaN Best 8.0 8111.0 8
8 9.0 NaN NaN NaN 4.0 41.0 9
9 10.0 A B C 2.0 21111.0 10
10 NaN NaN NaN NaN NaN NaN 11
11 10.0 Good Better Best 1024.0 102411.0 12
12 10.0 Good NaN Best 512.0 512.0 13
13 10.0 Good Better NaN 256.0 1256.0 14
14 10.0 Good Better Best NaN NaN 15
15 10.0 Good Better NaN 64.0 164.0 16
16 10.0 Good NaN Best 32.0 322.0 17
17 10.0 NaN Better Best 16.0 163.0 18
18 10.0 NaN NaN Best 8.0 844.0 19
19 10.0 NaN NaN NaN 4.0 4555.0 20
20 10.0 A B C 2.0 111.0 21
```

Data Profile

Rows: 21

Columns : 7

Done!!

>>>

All of column E has been deleted, owing to the fact that all values in that column were missing values/errors.

ii Drop the Columns Where Any of the Elements Is Missing Values

Code:

```
##### Assess-Good-Bad-02.py#####
# -*- coding: utf-8 -*-
import sys
import os
import pandas as pd
Base='C:/VKHCG'
sInputFileName='Good-or-Bad.csv'
sOutputFileName='Good-or-Bad-02.csv'
Company='01-Vermeulen'
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir):
os.makedirs(sFileDir)
### Import Warehouse
sFileName=Base + '/' + Company + '/00-RawData/' + sInputFileName
print('Loading :',sFileName)
RawData=pd.read_csv(sFileName,header=0)
print('#####')
print('## Raw Data Values')
print('#####')
print(RawData)
print('#####')
print('## Data Profile')
print('#####')
print('Rows :',RawData.shape[0])
print('Columns :',RawData.shape[1])
print('#####')
sFileName=sFileDir + '/' + sInputFileName
RawData.to_csv(sFileName, index = False)
TestData=RawData.dropna(axis=1, how='any')
print('#####')
print('## Test Data Values')
print('#####')
print(TestData)
print('#####')
print('## Data Profile')
print('#####')
print('Rows :',TestData.shape[0])
```

Seat No.: 3254185

```
print('Columns :',TestData.shape[1])
print('#####')
sFileName=sFileDir + '/' + sOutputFileName
TestData.to_csv(sFileName, index = False)
print('#####')
print('### Done!! #####')
print('#####')
```

OUTPUT:

>>>

===== RESTART: C:\VKHCG\01-Vermeulen\02-Assess\Assess-Good-Bad-02.py =====

Working Base : C:/VKHCG using win32

Loading : C:/VKHCG/01-Vermeulen/00-RawData/Good-or-Bad.csv

Raw Data Values

ID FieldA FieldB FieldC FieldD FieldE FieldF FieldG

0 1.0 Good Better Best 1024.0 NaN 10241.0 1

1 2.0 Good NaN Best 512.0 NaN 5121.0 2

2 3.0 Good Better NaN 256.0 NaN 256.0 3

3 4.0 Good Better Best NaN NaN 211.0 4

4 5.0 Good Better NaN 64.0 NaN 6411.0 5

5 6.0 Good NaN Best 32.0 NaN 32.0 6

6 7.0 NaN Better Best 16.0 NaN 1611.0 7

7 8.0 NaN NaN Best 8.0 NaN 8111.0 8

8 9.0 NaN NaN NaN 4.0 NaN 41.0 9

9 10.0 A B C 2.0 NaN 21111.0 10

10 NaN NaN NaN NaN NaN NaN NaN 11

11 10.0 Good Better Best 1024.0 NaN 102411.0 12

12 10.0 Good NaN Best 512.0 NaN 512.0 13

13 10.0 Good Better NaN 256.0 NaN 1256.0 14

14 10.0 Good Better Best NaN NaN NaN 15

15 10.0 Good Better NaN 64.0 NaN 164.0 16

16 10.0 Good NaN Best 32.0 NaN 322.0 17

17 10.0 NaN Better Best 16.0 NaN 163.0 18

18 10.0 NaN NaN Best 8.0 NaN 844.0 19

19 10.0 NaN NaN NaN 4.0 NaN 4555.0 20

20 10.0 A B C 2.0 NaN 111.0 21

Data Profile

Rows : 21

Columns : 8

Test Data Values

FieldG

0 1

1 2

2 3

Seat No.: 3254185

3 4
4 5
5 6
6 7
7 8
8 9
9 10
10 11
11 12
12 13
13 14
14 15
15 16
16 17
17 18
18 19
19 20
20 21

Data Profile

Rows : 21

Columns : 1

Done!!

>>>

iii. Keep Only the Rows That Contain a Maximum of Two Missing Values

Assess-Good-Bad-03.py

-*- coding: utf-8 -*-

import sys

import os

import pandas as pd

sInputFileName='Good-or-Bad.csv'

sOutputFileName='Good-or-Bad-03.csv'

Company='01-Vermeulen'

Base='C:/VKHCG'

print('#####')

print('Working Base :',Base, ' using Windows ~~~~')

print('#####')

sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'

if not os.path.exists(sFileDir):

os.makedirs(sFileDir)

Import Warehouse

sFileName=Base + '/' + Company + '/00-RawData/' + sInputFileName

print('Loading :',sFileName)

RawData=pd.read_csv(sFileName,header=0)

```

print('#####')
print('## Raw Data Values')
print('#####')
print(RawData)
print('#####')
print('## Data Profile')
print('#####')
print('Rows :',RawData.shape[0])
print('Columns :',RawData.shape[1])
print('#####')
sFileName=sFileDir + '/' + sInputFileName
RawData.to_csv(sFileName, index = False)
TestData=RawData.dropna(thresh=2)
print('#####')
print('## Test Data Values')
print('#####')
print(TestData)
print('#####')
print('## Data Profile')
print('#####')
print('Rows :',TestData.shape[0])
print('Columns :',TestData.shape[1])
print('#####')
sFileName=sFileDir + '/' + sOutputFileName
TestData.to_csv(sFileName, index = False)
print('#####')
print('### Done!! #####')
print('#####')

```

OUTPUT:

	A	B	C	D	E	F	G	H
1	ID	FieldA	FieldB	FieldC	FieldD	FieldE	FieldF	FieldG
2	1	Good	Better	Best	1024		10241	1
3	2	Good		Best	512		5121	2
4	3	Good	Better		256		256	3
5	4	Good	Better	Best			211	4
6	5	Good	Better		64		6411	5
7	6	Good		Best	32		32	6
8	7		Better	Best	16		1611	7
9	8			Best	8		8111	8
10	9				4		41	9
11	10 A		B	C	2		21111	10
12								11
13	10	Good	Better	Best	1024		102411	12
14	10	Good		Best	512		512	13
15	10	Good	Better		256		1256	14
16	10	Good	Better	Best				15
17	10	Good	Better		64		164	16
18	10	Good		Best	32		322	17
19	10		Better	Best	16		163	18
20	10			Best	8		844	19
21	10				4		4555	20
22	10 A		B	C	2		111	21

Before

	A	B	C	D	E	F	G	H
1	ID	FieldA	FieldB	FieldC	FieldD	FieldE	FieldF	FieldG
2	1	Good	Better	Best	1024		10241	1
3	2	Good		Best	512		5121	2
4	3	Good	Better		256		256	3
5	4	Good	Better	Best			211	4
6	5	Good	Better		64		6411	5
7	6	Good		Best	32		32	6
8	7		Better	Best	16		1611	7
9	8			Best	8		8111	8
10	9				4		41	9
11	10	A	B	C	2		21111	10
12	10	Good	Better	Best	1024		102411	12
13	10	Good		Best	512		512	13
14	10	Good	Better		256		1256	14
15	10	Good	Better	Best				15
16	10	Good	Better		64		164	16
17	10	Good		Best	32		322	17
18	10		Better	Best	16		163	18
19	10			Best	8		844	19
20	10				4		4555	20
21	10	A	B	C	2		111	21

After

Row with more than two missing values got deleted.

The next step along the route is to generate a full network routing solution for the company, to resolve the data issues in the retrieve data.

B. Write a Python / R program to create the network routing diagram from the given data On routers.

Code:

```
##### Assess-Network-Routing-Company.py #####
import sys
import os
import pandas as pd
pd.options.mode.chained_assignment = None
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using Windows')
print('#####')
sInputFileName1='01-Retrieve/01-EDS/01-R/Retrieve_Country_Code.csv'
sInputFileName2='01-Retrieve/01-EDS/02-Python/Retrieve_Router_Location.csv'
sInputFileName3='01-Retrieve/01-EDS/01-R/Retrieve_IP_DATA.csv'
sOutputFileName='Assess-Network-Routing-Company.csv'
Company='01-Vermeulen'
### Import Country Data
sFileName=Base + '/' + Company + '/' + sInputFileName1
print('#####')
print('Loading :',sFileName)
print('#####')
CountryData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
```

```

print('Loaded Country:',CountryData.columns.values)
print('#####')
## Assess Country Data
print('#####')
print('Changed :',CountryData.columns.values)
CountryData.rename(columns={'Country': 'Country_Name'}, inplace=True)
CountryData.rename(columns={'ISO-2-CODE': 'Country_Code'}, inplace=True)
CountryData.drop('ISO-M49', axis=1, inplace=True)
CountryData.drop('ISO-3-Code', axis=1, inplace=True)
CountryData.drop('RowID', axis=1, inplace=True)
print('To :',CountryData.columns.values)
print('#####')
### Import Company Data
sFileName=Base + '/' + Company + '/' + sInputFileName2
print('#####')
print('Loading :',sFileName)
print('#####')
CompanyData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('Loaded Company :',CompanyData.columns.values)
print('#####')
## Assess Company Data
print('#####')
print('Changed :',CompanyData.columns.values)
CompanyData.rename(columns={'Country': 'Country_Code'}, inplace=True)
print('To :',CompanyData.columns.values)
print('#####')
### Import Customer Data
sFileName=Base + '/' + Company + '/' + sInputFileName3
print('#####')
print('Loading :',sFileName)
print('#####')
CustomerRawData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('#####')
print('Loaded Customer :',CustomerRawData.columns.values)
print('#####')
CustomerData=CustomerRawData.dropna(axis=0, how='any')
print('#####')
print('Remove Blank Country Code')
print('Reduce Rows from', CustomerRawData.shape[0],' to ', CustomerData.shape[0])
print('#####')
print('#####')
print('Changed :',CustomerData.columns.values)
CustomerData.rename(columns={'Country': 'Country_Code'}, inplace=True)
print('To :',CustomerData.columns.values)

```

```

print('#####')
print('#####')
print('Merge Company and Country Data')
print('#####')
CompanyNetworkData=pd.merge(CompanyData, CountryData,how='inner'on='Country_Code')
print('#####')
print('Change ',CompanyNetworkData.columns.values)
for i in CompanyNetworkData.columns.values:
    j='Company_'+i
    CompanyNetworkData.rename(columns={i: j}, inplace=True)
print('To ', CompanyNetworkData.columns.values)
print('#####')
sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir):
    os.makedirs(sFileDir)
sFileName=sFileDir + '/' + sOutputFileName
print('#####')
print('Storing :, sFileName)
print('#####')
CompanyNetworkData.to_csv(sFileName, index = False, encoding="latin-1")
print('#####')
print('### Done!! #####')
print('#####')

```

OUTPUT:

Go to C:\VKHCG\01-Vermeulen\02-Assess\01-EDS\02-Python folder and open Assess-Network-Routing-Company.csv

	A	B	C	D	E
1	ny_Country	Company_Place_Name	Company_Latitude	Company_Longitude	Company_Country_Name
2	US	New York	40.7528	-73.9725	United States of America
3	US	New York	40.7214	-74.0052	United States of America
4	US	New York	40.7662	-73.9862	United States of America
5	US	New York	40.7449	-73.9782	United States of America
6	US	New York	40.7605	-73.9933	United States of America
7	US	New York	40.7588	-73.968	United States of America
8	US	New York	40.7637	-73.9727	United States of America
9	US	New York	40.7553	-73.9924	United States of America

Next, Access the the customers location using network router location

```

#####Assess-Network-Routing-Customer.py #####
import sys
import os
import pandas as pd
pd.options.mode.chained_assignment = None
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')

```

```

sInputFileName=Base+'/01-Vermeulen/02-Assess/01-EDS/02-Python/Assess-Network-Routing-
Customer.csv'
sOutputFileName='Assess-Network-Routing-Customer.gml'
Company='01-Vermeulen'
### Import Country Data
sFileName=sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
CustomerData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('Loaded Country:',CustomerData.columns.values)
print('#####')
print(CustomerData.head())
print('#####')
print('### Done!! #####')
print('#####')

```

OUTPUT:**Assess-Network-Routing-Customer.csv**

	A	B	C	D	E
1	ier_Countr	otomer_Place_Na	Customer_Latitude	Customer_Longitude	Customer_Country_Name
2	BW	Gaborone	-24.6464	25.9119	Botswana
3	BW	Francistown	-21.1667	27.5167	Botswana
4	BW	Maun	-19.9833	23.4167	Botswana
5	BW	Molepolole	-24.4167	25.5333	Botswana
6	NE	Niamey	13.5167	2.1167	Niger
7	MZ	Maputo	-25.9653	32.5892	Mozambique
8	MZ	Tete	-16.1564	33.5867	Mozambique
9	MZ	Quelimane	-17.8786	36.8883	Mozambique
10	MZ	Chimoio	-19.1164	33.4833	Mozambique
11	MZ	Matola	-25.9622	32.4589	Mozambique
12	MZ	Pemba	-12.9608	40.5078	Mozambique
13	MZ	Lichinga	-13.3128	35.2406	Mozambique
14	MZ	Maxixe	-23.8597	35.3472	Mozambique
15	MZ	Chibuto	-24.6867	33.5306	Mozambique
16	MZ	Ressano Garcia	-25.4428	31.9953	Mozambique
17	GH	Tema	5.6167	-0.0167	Ghana
18	GH	Kumasi	6.6833	-1.6167	Ghana
19	GH	Takoradi	4.8833	-1.75	Ghana
20	GH	Accra	5.55	-0.2167	Ghana

Assess-Network-Routing-Node.py

```

import sys
import os
import pandas as pd
pd.options.mode.chained_assignment = None
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sInputFileName='01-Retrieve/01-EDS/02-Python/Retrieve_IP_DATA.csv'

```

```
sOutputFileName='Assess-Network-Routing-Node.csv'
Company='01-Vermeulen'
### Import IP Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
IPData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('Loaded IP :', IPData.columns.values)
print('#####')
print('#####')
print('Changed :',IPData.columns.values)
IPData.drop('RowID', axis=1, inplace=True)
IPData.drop('ID', axis=1, inplace=True)
IPData.rename(columns={'Country': 'Country_Code'}, inplace=True)
IPData.rename(columns={'Place.Name': 'Place_Name'}, inplace=True)
IPData.rename(columns={'Post.Code': 'Post_Code'}, inplace=True)
IPData.rename(columns={'First.IP.Number': 'First_IP_Number'}, inplace=True)
IPData.rename(columns={'Last.IP.Number': 'Last_IP_Number'}, inplace=True)
print('To :',IPData.columns.values)
print('#####')
print('#####')
print('Change ',IPData.columns.values)
for i in IPData.columns.values:
    j='Node_'+i
    IPData.rename(columns={i: j}, inplace=True)
print('To ', IPData.columns.values)
print('#####')
sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir):
    os.makedirs(sFileDir)
sFileName=sFileDir + '/' + sOutputFileName
print('#####')
print('Storing :', sFileName)
print('#####')
IPData.to_csv(sFileName, index = False, encoding="latin-1")
print('#####')
print('### Done!! #####')
print('#####')
```

OUTPUT:**C:\VKHCG\01-Vermeulen\02-Assess\01-EDS\02-Python\Assess-Network-Routing-Node.csv**

	A	B	C	D	E	F	G
1	Node_Country_Code	Node_Place_Name	Node_Post_Code	Node_Latitude	Node_Longitude	Node_First_IP_Number	Node_Last_IP_Number
2	BW	Gaborone		-24.6464	25.9119	692781056	692781567
3	BW	Gaborone		-24.6464	25.9119	692781824	692783103
4	BW	Gaborone		-24.6464	25.9119	692909056	692909311
5	BW	Gaborone		-24.6464	25.9119	692909568	692910079
6	BW	Gaborone		-24.6464	25.9119	693051392	693052415
7	BW	Gaborone		-24.6464	25.9119	693078272	693078527
8	BW	Gaborone		-24.6464	25.9119	693608448	693616639
9	BW	Gaborone		-24.6464	25.9119	696929792	696930047
10	BW	Gaborone		-24.6464	25.9119	700438784	700439039
11	BW	Gaborone		-24.6464	25.9119	702075904	702076927
12	BW	Gaborone		-24.6464	25.9119	702498816	702499839
13	BW	Gaborone		-24.6464	25.9119	702516224	702517247
14	BW	Gaborone		-24.6464	25.9119	774162663	774162667
15	BW	Gaborone		-24.6464	25.9119	1401887232	1401887743
16	BW	Gaborone		-24.6464	25.9119	1754209024	1754209279
17	NE	Niamey		13.5167	2.1167	696918528	696919039
18	NE	Niamey		13.5167	2.1167	696922112	696924159
19	NE	Niamey		13.5167	2.1167	701203456	701203711
20	NE	Niamey		13.5167	2.1167	758886912	758887167
21	NE	Niamey		13.5167	2.1167	1347294153	1347294160
22	NE	Niamey		13.5167	2.1167	1755108096	1755108351
23	NE	Niamey		13.5167	2.1167	1755828480	1755828735
24	MZ	Maputo		-25.9653	32.5892	692883456	692883967
25	MZ	Maputo		-25.9653	32.5892	692944896	692946943

Directed Acyclic Graph (DAG)

A directed acyclic graph is a specific graph that only has one path through the graph.

C. Write a Python / R program to build directed acyclic graph.

Open your python editor and create a file named Assess-DAG-Location.py in directory

C:\VKHCG\01-Vermeulen\02-Assess

```
import networkx as nx
import matplotlib.pyplot as plt
import sys
import os
import pandas as pd
Base='C:\VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sInputFileName='01-Retrieve\01-EDS\02-Python\Retrieve_Router_Location.csv'
sOutputFileName1='Assess-DAG-Company-Country.png'
sOutputFileName2='Assess-DAG-Company-Country-Place.png'
Company='01-Vermeulen'
### Import Company Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
```

```
CompanyData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('Loaded Company :',CompanyData.columns.values)
print('#####')
print(CompanyData)
print('#####')
print('Rows : ',CompanyData.shape[0])
print('#####')
G1=nx.DiGraph()
G2=nx.DiGraph()
for i in range(CompanyData.shape[0]):
    G1.add_node(CompanyData['Country'][i])
    sPlaceName= CompanyData['Place_Name'][i] + '-' + CompanyData['Country'][i]
    G2.add_node(sPlaceName)
    print('#####')
    for n1 in G1.nodes():
        for n2 in G1.nodes():
            if n1 != n2:
                print('Link :',n1,' to ', n2)
                G1.add_edge(n1,n2)
                print('#####')
                print('#####')
    print("Nodes of graph: ")
    print(G1.nodes())
    print("Edges of graph: ")
    print(G1.edges())
    print('#####')
    sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
    if not os.path.exists(sFileDir):
        os.makedirs(sFileDir)
    sFileName=sFileDir + '/' + sOutputFileName1
    print('#####')
    print('Storing :', sFileName)
    print('#####')
    nx.draw(G1,pos=nx.spectral_layout(G1),
    nodecolor='r',edge_color='g',
    with_labels=True,node_size=8000,
    font_size=12)
    plt.savefig(sFileName) # save as png
    plt.show() # display
    print('#####')
    for n1 in G2.nodes():
        for n2 in G2.nodes():
            if n1 != n2:
                print('Link :',n1,' to ', n2)
```

```
G2.add_edge(n1,n2)
print('#####')
print('#####')
print("Nodes of graph: ")
print(G2.nodes())
print("Edges of graph: ")
print(G2.edges())
print('#####')
sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir):
os.makedirs(sFileDir)
sFileName=sFileDir + '/' + sOutputFileName2
print('#####')
print('Storing :', sFileName)
print('#####')
nx.draw(G2,pos=nx.spectral_layout(G2),
nodecolor='r',edge_color='b',
with_labels=True,node_size=8000,
font_size=12)
plt.savefig(sFileName) # save as png
plt.show() # display
```

OUTPUT:

Rows : 150

Link : US to DE

Link : US to GB

Link : DE to US

Link : DE to GB

Link : GB to US

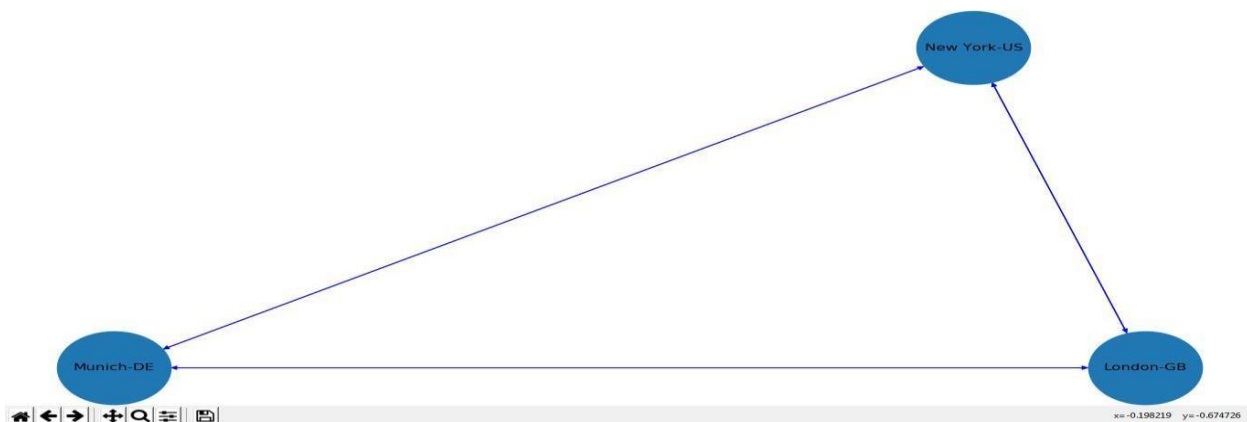
Link : GB to DE

Nodes of graph:

['US', 'DE', 'GB']

Edges of graph:

[('US', 'DE'), ('US', 'GB'), ('DE', 'US'), ('DE', 'GB'), ('GB', 'US'), ('GB', 'DE')]



Open your Python editor and create a file named Assess-DAG-GPS.py in directory
C:\VKHCG\01-Vermeulen\02-Assess.

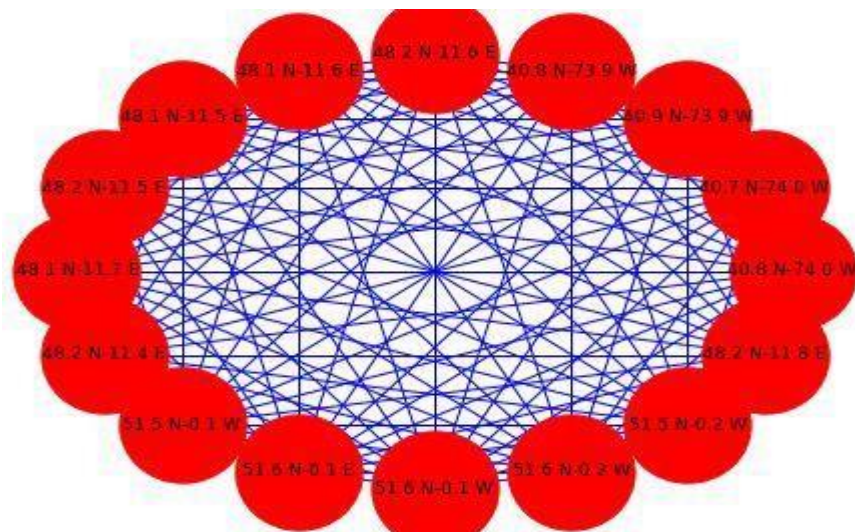
```
import networkx as nx
import matplotlib.pyplot as plt
import sys
import os
import pandas as pd
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sInputFileName='01-Retrieve/01-EDS/02-Python/Retrieve_Router_Location.csv'
sOutputFileName='Assess-DAG-Company-GPS.png'
Company='01-Vermeulen'
### Import Company Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
CompanyData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('Loaded Company :',CompanyData.columns.values)
print('#####')
print(CompanyData)
print('#####')
print('Rows : ',CompanyData.shape[0])
print('#####')
G=nx.Graph()
for i in range(CompanyData.shape[0]):
    nLatitude=round(CompanyData['Latitude'][i],2)
    nLongitude=round(CompanyData['Longitude'][i],2)
    if nLatitude < 0:
        sLatitude = str(nLatitude*-1) + ' S'
    else:
        sLatitude = str(nLatitude) + ' N'
    if nLongitude < 0:
        sLongitude = str(nLongitude*-1) + ' W'
    else:
        sLongitude = str(nLongitude) + ' E'
    sGPS= sLatitude + '-' + sLongitude
    G.add_node(sGPS)
print('#####')
for n1 in G.nodes():
    for n2 in G.nodes():
        if n1 != n2:
```

Seat No.: 3254185

```
print('Link :',n1,' to ', n2)
G.add_edge(n1,n2)
print('#####')
print('#####')
print("Nodes of graph: ")
print(G.number_of_nodes())
```

OUTPUT:

```
=== RESTART: C:\VKHCG\01-Vermeulen\02-Assess\Assess-DAG-GPS-unsmoothed.py ===
Working Base : C:\VKHCG using win32
Loading : C:\VKHCG\01-Vermeulen\01-Retrieve\01-EDS\02-Python\Retrieve_Router_Location.csv
Loaded Company : ['Country' 'Place_Name' 'Latitude' 'Longitude']
Country Place_Name Latitude Longitude
0 US New York 40.7528 -73.9725
1 US New York 40.7214 -74.0052
-
-
-
Link : 48.15 N-11.74 E to 48.15 N-11.46 E
Link : 48.15 N-11.74 E to 48.09 N-11.54 E
Link : 48.15 N-11.74 E to 48.18 N-11.75 E
Link : 48.15 N-11.74 E to 48.1 N-11.47 E
Nodes of graph:
117
Edges of graph:
6786
```



D. Write a Python / R program to build directed acyclic graph.

Open your python editor and create a file named Assess-DAG-Location.py in directory
C:\VKHCG\01-Vermeulen\02-Assess

```
import networkx as nx
import matplotlib.pyplot as plt
```

```
import sys
import os
import pandas as pd
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sInputFileName='01-Retrieve/01-EDS/02-Python/Retrieve_Router_Location.csv'
sOutputFileName1='Assess-DAG-Company-Country.png'
sOutputFileName2='Assess-DAG-Company-Country-Place.png'
Company='01-Vermeulen'
### Import Company Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
CompanyData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('Loaded Company :',CompanyData.columns.values)
print('#####')
print(CompanyData)
print('#####')
print('Rows : ',CompanyData.shape[0])
print('#####')
G1=nx.DiGraph()
G2=nx.DiGraph()
for i in range(CompanyData.shape[0]):
    G1.add_node(CompanyData['Country'][i])
sPlaceName= CompanyData['Place_Name'][i] + '-' + CompanyData['Country'][i]
G2.add_node(sPlaceName)
print('#####')
for n1 in G1.nodes():
    for n2 in G1.nodes():
        if n1 != n2:
            print('Link :',n1,' to ', n2)
G1.add_edge(n1,n2)
print('#####')
print('#####')
print("Nodes of graph: ")
print(G1.nodes())
print("Edges of graph: ")
print(G1.edges())
print('#####')
sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir):
```

```
os.makedirs(sFileDir)
sFileName=sFileDir + '/' + sOutputFileName1
print('#####')
print('Storing :', sFileName)
print('#####')
nx.draw(G1,pos=nx.spectral_layout(G1),
nodecolor='r',edge_color='g',
with_labels=True,node_size=8000,
font_size=12)
plt.savefig(sFileName) # save as png
plt.show() # display
print('#####')
for n1 in G2.nodes():
    for n2 in G2.nodes():
        if n1 != n2:
            print('Link :',n1,' to ', n2)
G2.add_edge(n1,n2)
print('#####')
print('#####')
print("Nodes of graph: ")
print(G2.nodes())
print("Edges of graph: ")
print(G2.edges())
print('#####')
sFileDir=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir):
    os.makedirs(sFileDir)
sFileName=sFileDir + '/' + sOutputFileName2
print('#####')
print('Storing :', sFileName)
print('#####')
nx.draw(G2,pos=nx.spectral_layout(G2),
nodecolor='r',edge_color='b',
with_labels=True,node_size=8000,
font_size=12)
plt.savefig(sFileName) # save as png
plt.show()
```

OUTPUT:

```

Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep  5 2022, 14:08:36) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Student/Documents/KARAN/practical 1.py =====
#####
Working Base : C:/VKHCG using win32
#####
Loading : C:/VKHCG/01-Vermeulen/01-Retrieve/01-EDS/02-Python/Retrieve_Router_Location.csv
#####
Loaded Company : ['Country' 'Place_Name' 'Latitude' 'Longitude']
#####
Country Place_Name Latitude Longitude
0      US      New York  40.7528   -73.9725
1      US      New York  40.7214   -74.0052
2      US      New York  40.7662   -73.9862
3      US      New York  40.7449   -73.9782
4      US      New York  40.7605   -73.9933
..      ...      ...      ...      ...
145    DE      Munich   48.1475   11.4635
146    DE      Munich   48.0915   11.5392
147    DE      Munich   48.1833   11.7500
148    DE      Munich   48.1000   11.4667
149    DE      Munich   48.1480   11.7434

[150 rows x 4 columns]
#####
Rows : 150
#####
Link : US to DE
Link : US to GB
Link : DE to US
Link : DE to GB
Link : GB to US
Link : GB to DE
#####
Nodes of graph:
['US', 'DE', 'GB']
Edges of graph:
[('GB', 'GB')]
#####
Storing : C:/VKHCG/01-Vermeulen/02-Assess/01-EDS/02-Python/Assess-DAG-Company-Country.png
#####

```



Practical 5**Processing Data****A. Build the time hub, links, and satellites.**

Open your Python editor and create a file named Process_Time.py. Save it into directory C:\VKHCG\01-Vermeulen\03-Process.

```
# -*- coding: utf-8 -*-
import sys
import os
from datetime import datetime
from datetime import timedelta
from pytz import timezone, all_timezones
import pandas as pd
import sqlite3 as sq
from pandas.io import sql
import uuid
pd.options.mode.chained_assignment = None
if sys.platform == 'linux':
    Base=os.path.expanduser('~') + '/VKHCG'
else:
    Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='01-Vermeulen'
InputDir='00-RawData'
InputFileName='VehicleData.csv'
sDataBaseDir=Base + '/' + Company + '/03-Process/SQLite'
if not os.path.exists(sDataBaseDir):
    os.makedirs(sDataBaseDir)
sDatabaseName=sDataBaseDir + '/Hillman.db'
conn1 = sq.connect(sDatabaseName)
sDataVaultDir=Base + '/88-DV'
if not os.path.exists(sDataVaultDir):
    os.makedirs(sDataVaultDir)
sDatabaseName=sDataVaultDir + '/datavault.db'
conn2 = sq.connect(sDatabaseName)
base = datetime(2018,1,1,0,0,0)
numUnits=10*365*24
date_list = [base - timedelta(hours=x) for x in range(0, numUnits)]
t=0
for i in date_list:
    now_utc=i.replace(tzinfo=timezone('UTC'))
    sDateTime=now_utc.strftime("%Y-%m-%d %H:%M:%S")
```

```
print(sDateTime)
sDateTimeKey=sDateTime.replace(' ','-').replace(':', '-')
t+=1
IDNumber=str(uuid.uuid4())
TimeLine=[('ZoneBaseKey', ['UTC']),
('IDNumber', [IDNumber]),
('nDateTimeValue', [now_utc]),
('DateTimeValue', [sDateTime]),
('DateTimeKey', [sDateTimeKey])]
if t==1:
    TimeFrame = pd.DataFrame.from_items(TimeLine)
else:
    TimeRow = pd.DataFrame.from_items(TimeLine)
    TimeFrame = TimeFrame.append(TimeRow)
TimeHub=TimeFrame[['IDNumber','ZoneBaseKey','DateTimeKey','DateTimeValue']]
TimeHubIndex=TimeHub.set_index(['IDNumber'],inplace=False)
TimeFrame.set_index(['IDNumber'],inplace=True)
sTable = 'Process-Time'
print('Storing :',sDatabaseName,' Table:',sTable)
TimeHubIndex.to_sql(sTable, conn1, if_exists="replace")
sTable = 'Hub-Time'
print('Storing :',sDatabaseName,' Table:',sTable)
TimeHubIndex.to_sql(sTable, conn2, if_exists="replace")
active_timezones=all_timezones
z=0
for zone in active_timezones:
    t=0
    for j in range(TimeFrame.shape[0]):
        now_date=TimeFrame['nDateTimeValue'][j]
        DateTimeKey=TimeFrame['DateTimeKey'][j]
        now_utc=now_date.replace(tzinfo=timezone('UTC'))
        sDateTime=now_utc.strftime("%Y-%m-%d %H:%M:%S")
        now_zone = now_utc.astimezone(timezone(zone))
        sZoneDateTime=now_zone.strftime("%Y-%m-%d %H:%M:%S")
        print(sZoneDateTime)
    t+=1
    z+=1
IDZoneNumber=str(uuid.uuid4())
TimeZoneLine=[('ZoneBaseKey', ['UTC']),
('IDZoneNumber', [IDZoneNumber]),
('DateTimeKey', [DateTimeKey]),
('UTCDateTimeValue', [sDateTime]),
('Zone', [zone]),
('DateTimeValue', [sZoneDateTime])]
```

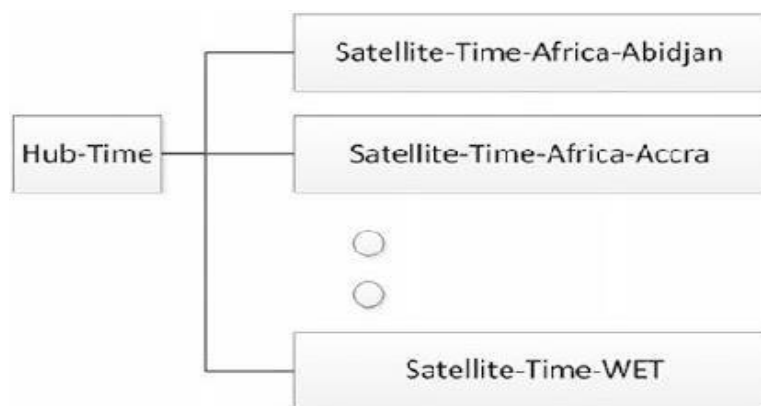
```

if t==1:
TimeZoneFrame = pd.DataFrame.from_items(TimeZoneLine)
else:
TimeZoneRow = pd.DataFrame.from_items(TimeZoneLine)
TimeZoneFrame = TimeZoneFrame.append(TimeZoneRow)
TimeZoneFrameIndex=TimeZoneFrame.set_index(['IDZoneNumber'],inplace=False)
sZone=zone.replace('/', '-').replace(' ', '')
sTable = 'Process-Time-'+sZone
print('Storing :',sDatabaseName,' Table:',sTable)
TimeZoneFrameIndex.to_sql(sTable, conn1, if_exists="replace")
sTable = 'Satellite-Time-'+sZone
print('Storing :',sDatabaseName,' Table:',sTable)
TimeZoneFrameIndex.to_sql(sTable, conn2, if_exists="replace")
print('#####')
print('Vacuum Databases')
sSQL="VACUUM;"
sql.execute(sSQL,conn1)
sql.execute(sSQL,conn2)
print('#####')
print('### Done!! #####')

```

You have built your first hub and satellites for time in the data vault.

The data vault has been built in directory ..\ VKHCG\88-DV\datavault.db. You can access it with your SQLite Tools



Golden Nominal

A golden nominal record is a single person's record, with distinctive references for use by all systems.

This gives the system a single view of the person. I use first name, other names, last name, and birth date as my golden nominal. The data we have in the assess directory requires a birth date to become a golden nominal. The proram will generate a golden nominal using our sample data set.

Open your Python editor and create a file called Process-People.py in the

C:\VKHCG\04-Clark\03-Process directory.

Code:

```
import sys
import os
import sqlite3 as sq
import pandas as pd
from pandas.io import sql
from datetime import datetime, timedelta
from pytz import timezone, all_timezones
from random import randint
import uuid
if sys.platform == 'linux':
    Base=os.path.expanduser('~') + '/VKHCG'
else:
    Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='04-Clark'
sInputFileName='02-Assess/01-EDS/02-Python/Assess_People.csv'
sDataBaseDir=Base + '/' + Company + '/03-Process/SQLite'
if not os.path.exists(sDataBaseDir):
    os.makedirs(sDataBaseDir)
sDatabaseName=sDataBaseDir + '/clark.db'
conn1 = sq.connect(sDatabaseName)
sDataVaultDir=Base + '/88-DV'
if not os.path.exists(sDataVaultDir):
    os.makedirs(sDataVaultDir)
sDatabaseName=sDataVaultDir + '/datavault.db'
conn2 = sq.connect(sDatabaseName)
### Import Female Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
print(sFileName)
RawData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
RawData.drop_duplicates(subset=None, keep='first', inplace=True)
start_date = datetime(1900,1,1,0,0,0)
start_date_utc=start_date.replace(tzinfo=timezone('UTC'))
HoursBirth=100*365*24
RawData['BirthDateUTC']=RawData.apply(lambda row:
(start_date_utc + timedelta(hours=randint(0, HoursBirth)))
,axis=1)
zonemax=len(all_timezones)-1
```

```

RawData['TimeZone']=RawData.apply(lambda row: (all_timezones[randint(0, zonemax)]),axis=1)
RawData['BirthDateISO']=RawData.apply(lambda row:
row["BirthDateUTC"].astimezone(timezone(row["TimeZone"])),axis=1)
RawData['BirthDateKey']=RawData.apply(lambda row:
row["BirthDateUTC"].strftime("%Y-%m-%d %H:%M:%S"),axis=1)
RawData['BirthDate']=RawData.apply(lambda row:
row["BirthDateISO"].strftime("%Y-%m-%d %H:%M:%S") ,axis=1)
RawData['PersonID']=RawData.apply(lambda row:str(uuid.uuid4()),axis=1)
Data=RawData.copy()
Data.drop('BirthDateUTC', axis=1,inplace=True)
Data.drop('BirthDateISO', axis=1,inplace=True)
indexed_data = Data.set_index(['PersonID'])
print('#####')
print('#####')
sTable='Process_Person'
print('Storing :',sDatabaseName,' Table:',sTable)
indexed_data.to_sql(sTable, conn1, if_exists="replace")
print('#####')
PersonHubRaw=Data[['PersonID','FirstName','SecondName','LastName','BirthDateKey']]
PersonHubRaw['PersonHubID']=RawData.apply(lambda row:
str(uuid.uuid4())
,axis=1)
PersonHub=PersonHubRaw.drop_duplicates(subset=None, \
keep='first',\
inplace=False)
indexed_PersonHub = PersonHub.set_index(['PersonHubID'])
sTable = 'Hub-Person'
print('Storing :',sDatabaseName,' Table:',sTable)
indexed_PersonHub.to_sql(sTable, conn2, if_exists="replace")
PersonSatelliteGenderRaw=Data[['PersonID','FirstName','SecondName','LastName',\
,'BirthDateKey','Gender']]
PersonSatelliteGenderRaw['PersonSatelliteID']=RawData.apply(lambda row:
str(uuid.uuid4())
,axis=1)
PersonSatelliteGender=PersonSatelliteGenderRaw.drop_duplicates(subset=None, \
keep='first', \
inplace=False)
indexed_PersonSatelliteGender = PersonSatelliteGender.set_index(['PersonSatelliteID'])
sTable = 'Satellite-Person-Gender'
print('Storing :',sDatabaseName,' Table:',sTable)
indexed_PersonSatelliteGender.to_sql(sTable, conn2, if_exists="replace")
PersonSatelliteBirthdayRaw=Data[['PersonID','FirstName','SecondName','LastName',\
,'BirthDateKey','TimeZone','BirthDate']]
PersonSatelliteBirthdayRaw['PersonSatelliteID']=RawData.apply(lambda row:

```

```
str(uuid.uuid4()))
,axis=1)
PersonSatelliteBirthday=PersonSatelliteBirthdayRaw.drop_duplicates(subset=None, \
keep='first',\
inplace=False)
indexed_PersonSatelliteBirthday = PersonSatelliteBirthday.set_index(['PersonSatelliteID'])
sTable = 'Satellite-Person-Names'
print('Storing :',sDatabaseName,' Table:',sTable)
indexed_PersonSatelliteBirthday.to_sql(sTable, conn2, if_exists="replace")
sFileDir=Base + '/' + Company + '/03-Process/01-EDS/02-Python'
if not os.path.exists(sFileDir):
os.makedirs(sFileDir)
sOutputFileName = sTable + '.csv'
sFileName=sFileDir + '/' + sOutputFileName
print('#####')
print('Storing :', sFileName)
print('#####')
RawData.to_csv(sFileName, index = False)
print('#####')
print('#####')
print('Vacuum Databases')
sSQL="VACUUM;"
sql.execute(sSQL,conn1)
sql.execute(sSQL,conn2)
print('#####')
print('### Done!! #####')
```

OUTPUT :

It will apply golden nominal rules by assuming nobody born before January 1, 1900, dropping to two ISO complex date time structures, as the code does not translate into SQLite's data types and saves your new golden nominal to a CSV file.

Load the person into the data vault

```
===== RESTART: C:\VKHCG\04-Clark\03-Process\Process-People.py =====
Working Base : C:\VKHCG using win32
Loading : C:\VKHCG\04-Clark\02-Assess\01-EDS\02-Python\Assess_People.csv
C:\VKHCG\04-Clark\02-Assess\01-EDS\02-Python\Assess_People.csv
Storing : C:\VKHCG\88-DV\datavault.db Table: Process_Person
Storing : C:\VKHCG\88-DV\datavault.db Table: Satellite-Person-Gender
Storing : C:\VKHCG\88-DV\datavault.db Table: Satellite-Person-Names
Storing : C:\VKHCG\04-Clark\03-Process\01-EDS\02-Python\Satellite-Person-Names.csv
Vacuum Databases
### Done!! #####
```

Vehicles

The international classification of vehicles is a complex process. There are standards, but these are not universally applied or similar between groups or countries. Let's load the vehicle data for Hillman Ltd

into the data vault, as we will need it later. Create a new file named Process-Vehicle-Logistics.py in the Python editor in directory ..\VKHCG\03-Hillman\03-Process.

```
# -*- coding: utf-8 -*-
```

```
import sys
```

```
import os
```

```
import pandas as pd
```

```
import sqlite3 as sq
```

```
from pandas.io import sql
```

```
import uuid
```

```
pd.options.mode.chained_assignment = None
```

```
if sys.platform == 'linux':
```

```
Base=os.path.expanduser('~') + '/VKHCG'
```

```
else:
```

```
Base='C:/VKHCG'
```

```
print('#####')
```

```
print('Working Base :',Base, ' using ', sys.platform)
```

```
print('#####')
```

```
Company='03-Hillman'
```

```
InputDir='00-RawData'
```

```
InputFileName='VehicleData.csv'
```

```
sDataBaseDir=Base + '/' + Company + '/03-Process/SQLite'
```

```
if not os.path.exists(sDataBaseDir):
```

```
os.makedirs(sDataBaseDir)
```

```
sDatabaseName=sDataBaseDir + '/Hillman.db'
```

```
conn1 = sq.connect(sDatabaseName)
```

```
sDataVaultDir=Base + '/88-DV'
```

```
if not os.path.exists(sDataVaultDir):
```

```
os.makedirs(sDataVaultDir)
```

```
sDatabaseName=sDataVaultDir + '/datavault.db'
```

```
conn2 = sq.connect(sDatabaseName)
```

```
sFileName=Base + '/' + Company + '/' + InputDir + '/' + InputFileName
```

```
print('#####')
```

```
print('Loading :',sFileName)
```

```
VehicleRaw=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
```

```
sTable='Process_Vehicles'
```

```
print('Storing :',sDatabaseName,' Table:',sTable)
```

```
VehicleRaw.to_sql(sTable, conn1, if_exists="replace")
```

```
VehicleRawKey=VehicleRaw[['Make','Model']].copy()
```

```
VehicleKey=VehicleRawKey.drop_duplicates()
```

```
VehicleKey['ObjectKey']=VehicleKey.apply(lambda row:
```

```
str('(' + str(row['Make']).strip().replace(' ', '-').replace('/', '-').lower() + ')-('
```

```
+(str(row['Model']).strip().replace(' ', '-').replace('/', '-').lower())+'))',axis=1)
```

```
VehicleKey['ObjectType']=VehicleKey.apply(lambda row: 'vehicle' ,axis=1)
```

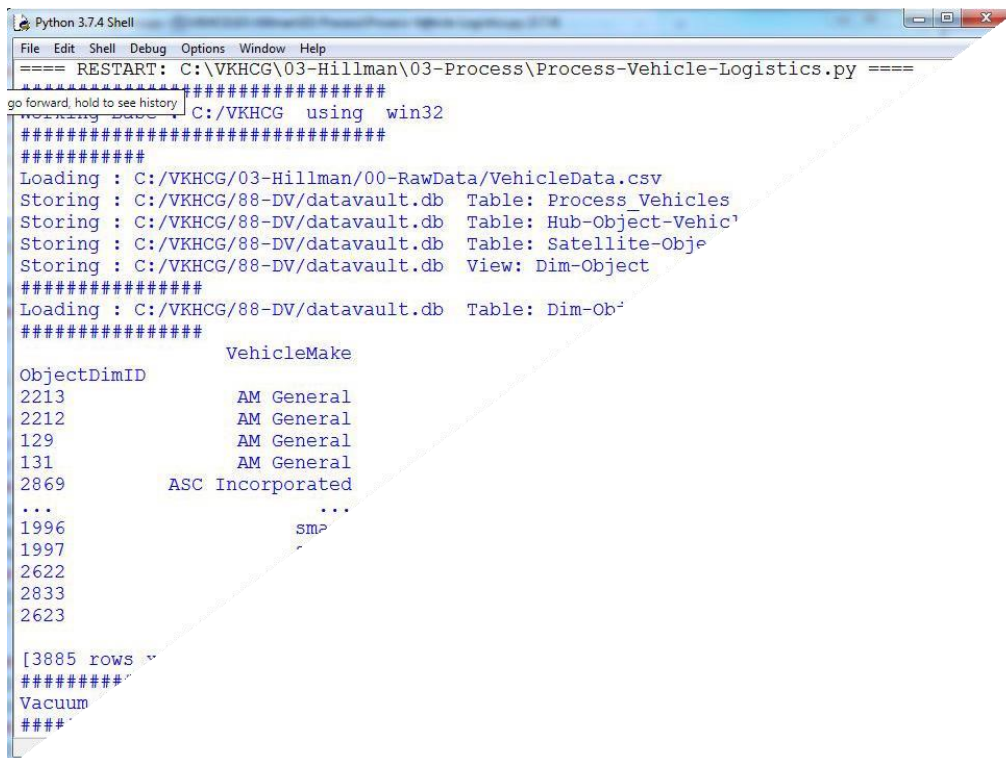
```
VehicleKey['ObjectUUID']=VehicleKey.apply(lambda row:str(uuid.uuid4()),axis=1)
```

```
#### Vehicle Hub
#
VehicleHub=VehicleKey[['ObjectType','ObjectKey','ObjectUUID']].copy()
VehicleHub.index.name='ObjectHubID'
sTable = 'Hub-Object-Vehicle'
print('Storing :',sDatabaseName,' Table:',sTable)
VehicleHub.to_sql(sTable, conn2, if_exists="replace")
#### Vehicle Satellite
#
VehicleSatellite=VehicleKey[['ObjectType','ObjectKey','ObjectUUID','Make','Model']].copy()
VehicleSatellite.index.name='ObjectSatelliteID'
sTable = 'Satellite-Object-Make-Model'
print('Storing :',sDatabaseName,' Table:',sTable)
VehicleSatellite.to_sql(sTable, conn2, if_exists="replace")
#### Vehicle Dimension
sView='Dim-Object'
print('Storing :',sDatabaseName,' View:',sView)
sSQL="CREATE VIEW IF NOT EXISTS [" + sView + "] AS"
sSQL=sSQL+ " SELECT DISTINCT"
sSQL=sSQL+ " H.ObjectType,"
sSQL=sSQL+ " H.ObjectKey AS VehicleKey,"
sSQL=sSQL+ " TRIM(S.Make) AS VehicleMake,"
sSQL=sSQL+ " TRIM(S.Model) AS VehicleModel"
sSQL=sSQL+ " FROM"
sSQL=sSQL+ " [Hub-Object-Vehicle] AS H"
sSQL=sSQL+ " JOIN"
sSQL=sSQL+ " [Satellite-Object-Make-Model] AS S"
sSQL=sSQL+ " ON"
sSQL=sSQL+ " H.ObjectType=S.ObjectType"
sSQL=sSQL+ " AND"
sSQL=sSQL+ " H.ObjectUUID=S.ObjectUUID;"
sql.execute(sSQL,conn2)
print('#####')
print('Loading :',sDatabaseName,' Table:',sView)
sSQL=" SELECT DISTINCT"
sSQL=sSQL+ " VehicleMake,"
sSQL=sSQL+ " VehicleModel"
sSQL=sSQL+ " FROM"
sSQL=sSQL+ " [" + sView + "]"
sSQL=sSQL+ " ORDER BY"
sSQL=sSQL+ " VehicleMake"
sSQL=sSQL+ " AND"
DimObjectData=pd.read_sql_query(sSQL, conn2)
DimObjectData.index.name='ObjectDimID'
```

```

DimObjectData.sort_values(['VehicleMake','VehicleModel'],inplace=True, ascending=True)
print('#####')
print(DimObjectData)
print('#####')
print('Vacuum Databases')
sSQL="VACUUM;"
sql.execute(sSQL,conn1)
sql.execute(sSQL,conn2)
print('#####')
conn1.close()
conn2.close()
#print('### Done!! #####')
OUTPUT:

```



```

Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
==== RESTART: C:\VKHCG\03-Hillman\03-Process\Process-Vehicle-Logistics.py ====
#####
C:/VKHCG using win32
#####
Loading : C:/VKHCG/03-Hillman/00-RawData/VehicleData.csv
Storing : C:/VKHCG/88-DV/datavault.db Table: Process_Vehicles
Storing : C:/VKHCG/88-DV/datavault.db Table: Hub-Object-Vehic
Storing : C:/VKHCG/88-DV/datavault.db Table: Satellite-Obj
Storing : C:/VKHCG/88-DV/datavault.db View: Dim-Object
#####
Loading : C:/VKHCG/88-DV/datavault.db Table: Dim-Ob
#####
VehicleMake
ObjectDimID
2213 AM General
2212 AM General
129 AM General
131 AM General
2869 ASC Incorporated
...
1996 sm
1997
2622
2833
2623

[3885 rows ~
#####
Vacuum
###

```

Human-Environment Interaction

The interaction of humans with their environment is a major relationship that guides people's behavior and the characteristics of the location. Activities such as mining and other industries, roads, and landscaping at a location create both positive and negative effects on the environment, but also on humans. A location earmarked as a green belt, to assist in reducing the carbon footprint, or a new interstate change its current and future characteristics. The location is a main data source for the data science, and, normally, we find unknown or unexpected effects on the data insights. In the Python editor, open a new file named Process_Location.py in directory ..\VKHCG\01-Vermeulen\03-Process.

```

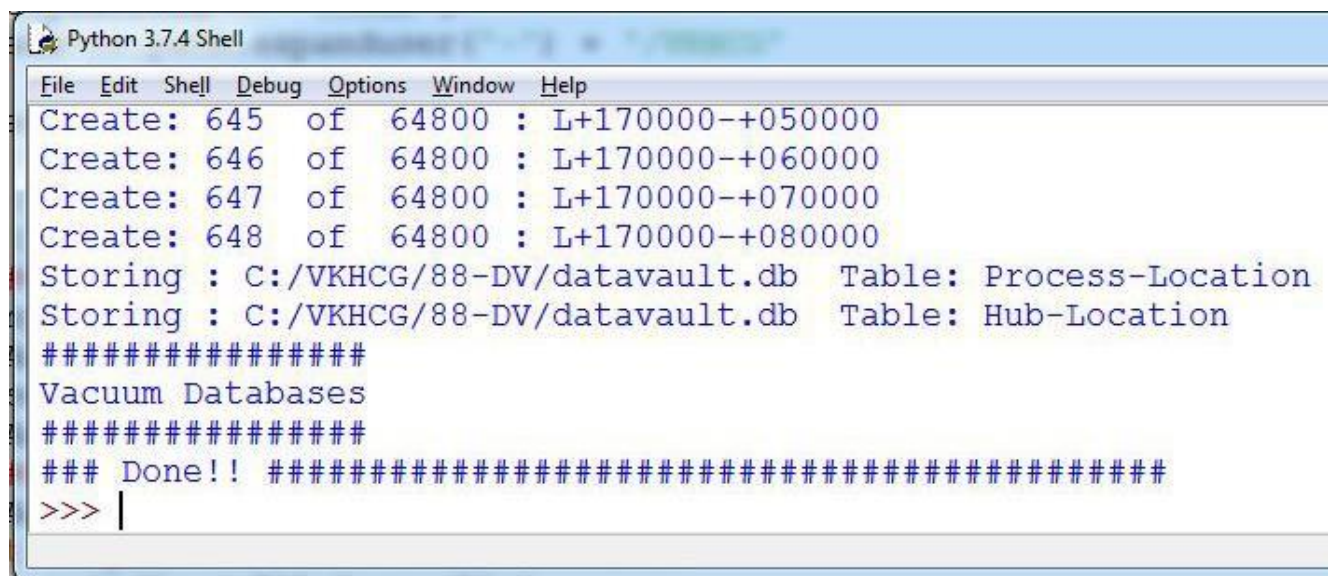
# -*- coding: utf-8 -*-
import sys
import os
import pandas as pd
import sqlite3 as sq

```

```
from pandas.io import sql
import uuid
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='01-Vermeulen'
InputAssessGraphName='Assess_All_Animals.gml'
EDSAssessDir='02-Assess/01-EDS'
InputAssessDir=EDSAssessDir + '/02-Python'
sFileAssessDir=Base + '/' + Company + '/' + InputAssessDir
if not os.path.exists(sFileAssessDir):
os.makedirs(sFileAssessDir)
sDataBaseDir=Base + '/' + Company + '/03-Process/SQLite'
if not os.path.exists(sDataBaseDir):
os.makedirs(sDataBaseDir)
sDatabaseName=sDataBaseDir + '/Vermeulen.db'
conn1 = sq.connect(sDatabaseName)
sDataVaultDir=Base + '/88-DV'
if not os.path.exists(sDataVaultDir):
os.makedirs(sDataVaultDir)
sDatabaseName=sDataVaultDir + '/datavault.db'
conn2 = sq.connect(sDatabaseName)
t=0
tMax=360*180
for Longitude in range(-180,180,10):
for Latitude in range(-90,90,10):
t+=1
IDNumber=str(uuid.uuid4())
LocationName='L'+format(round(Longitude,3)*1000, '+07d') +\
'-' +format(round(Latitude,3)*1000, '+07d')
print('Create:',t, ' of ',tMax,':',LocationName)
LocationLine=[('ObjectBaseKey', ['GPS']),
('IDNumber', [IDNumber]),
('LocationNumber', [str(t)]),
('LocationName', [LocationName]),
('Longitude', [Longitude]),
('Latitude', [Latitude])]
if t==1:
LocationFrame = pd.DataFrame.from_items(LocationLine)
else:
LocationRow = pd.DataFrame.from_items(LocationLine)
LocationFrame = LocationFrame.append(LocationRow)
LocationHubIndex=LocationFrame.set_index(['IDNumber'],inplace=False)
```

```
sTable = 'Process-Location'
print('Storing :',sDatabaseName,' Table:',sTable)
LocationHubIndex.to_sql(sTable, conn1, if_exists="replace")
sTable = 'Hub-Location'
print('Storing :',sDatabaseName,' Table:',sTable)
LocationHubIndex.to_sql(sTable, conn2, if_exists="replace")
print('#####')
print('Vacuum Databases')
sSQL="VACUUM;"
sql.execute(sSQL,conn1)
sql.execute(sSQL,conn2)
print('#####')
print('### Done!! #####')
```

OUTPUT:



```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Create: 645 of 64800 : L+170000-+050000
Create: 646 of 64800 : L+170000-+060000
Create: 647 of 64800 : L+170000-+070000
Create: 648 of 64800 : L+170000-+080000
Storing : C:/VKHCG/88-DV/datavault.db Table: Process-Location
Storing : C:/VKHCG/88-DV/datavault.db Table: Hub-Location
#####
Vacuum Databases
#####
### Done!! #####
>>> |
```

Forecasting

Forecasting is the ability to project a possible future, by looking at historical data. The datavault enables these types of investigations, owing to the complete history it collects as it processes the source's systems data. A data scientist supply answers to such questions as the following:

- What should we buy?
- What should we sell?
- Where will our next business come from?

People want to know what you calculate to determine what is about to happen.

Open a new file in your Python editor and save it as Process-Shares-Data.py in directory

C: \VKHCG\04-Clark\03-Process. I will guide you through this

process. You will require a library called quandl

type pip install quandl in cmd

import sys

import os

```
import sqlite3 as sq
import quandl
import pandas as pd
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='04-Clark'
sInputFileName='00-RawData/VKHCG_Shares.csv'
sOutputFileName='Shares.csv'
sDataBaseDir=Base + '/' + Company + '/03-Process/SQLite'
if not os.path.exists(sDataBaseDir):
os.makedirs(sDataBaseDir)
sFileDir1=Base + '/' + Company + '/01-Retrieve/01-EDS/02-Python'
if not os.path.exists(sFileDir1):
os.makedirs(sFileDir1)
sFileDir2=Base + '/' + Company + '/02-Assess/01-EDS/02-Python'
if not os.path.exists(sFileDir2):
os.makedirs(sFileDir2)
sFileDir3=Base + '/' + Company + '/03-Process/01-EDS/02-Python'
if not os.path.exists(sFileDir3):
os.makedirs(sFileDir3)
sDatabaseName=sDataBaseDir + '/clark.db'
conn = sq.connect(sDatabaseName)
### Import Share Names Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
RawData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
RawData.drop_duplicates(subset=None, keep='first', inplace=True)
print('Rows :',RawData.shape[0])
print('Columns:',RawData.shape[1])
print('#####')
sFileName=sFileDir1 + '/Retrieve_' + sOutputFileName
print('#####')
print('Storing :', sFileName)
print('#####')
RawData.to_csv(sFileName, index = False)
print('#####')
sFileName=sFileDir2 + '/Assess_' + sOutputFileName
print('#####')
print('Storing :', sFileName)
print('#####')
```

```

RawData.to_csv(sFileName, index = False)
print('#####')
sFileName=sFileDir3 + '/Process_' + sOutputFileName
print('#####')
print('Storing :', sFileName)
print('#####')
RawData.to_csv(sFileName, index = False)
print('#####')
### Import Shares Data Details
nShares=RawData.shape[0]
#nShares=6
for sShare in range(nShares):
sShareName=str(RawData['Shares'][sShare])
ShareData = quandl.get(sShareName)
UnitsOwn=RawData['Units'][sShare]
ShareData['UnitsOwn']=ShareData.apply(lambda row:(UnitsOwn),axis=1)
ShareData['ShareCode']=ShareData.apply(lambda row:(sShareName),axis=1)
print('#####')
print('Share :',sShareName)
print('Rows :',ShareData.shape[0])
print('Columns:',ShareData.shape[1])
print('#####')
print('#####')
sTable=str(RawData['sTable'][sShare])
print('Storing :',sDatabaseName,' Table:',sTable)
ShareData.to_sql(sTable, conn, if_exists="replace")
print('#####')
sOutputFileName = sTable.replace("/", "-") + '.csv'
sFileName=sFileDir1 + '/Retrieve_' + sOutputFileName
print('#####')
print('Storing :', sFileName)
print('#####')
ShareData.to_csv(sFileName, index = False)
print('#####')
sOutputFileName = sTable.replace("/", "-") + '.csv'
sFileName=sFileDir2 + '/Assess_' + sOutputFileName
print('#####')
print('Storing :', sFileName)
print('#####')
ShareData.to_csv(sFileName, index = False)
print('#####')
sOutputFileName = sTable.replace("/", "-") + '.csv'
sFileName=sFileDir3 + '/Process_' + sOutputFileName
print('#####')

```

```
print('Storing :', sFileName)
print('#####')
ShareData.to_csv(sFileName, index = False)
print('#####')
print('### Done!! #####')
```

OUTPUT:

```
===== RESTART: C:\VKHCG\04-Clark\03-Process\Process-Shares-Data.py =====
Working Base : C:/VKHCG using win32
Loading : C:/VKHCG/04-Clark/00-RawData/VKHCG_Shares.csv
Rows : 10
Columns: 3
Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_Shares.csv
Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_Shares.csv
Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_Shares.csv
Share : WIKI/GOOGL
Rows : 3424
Columns: 14
Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: WIKI_Google
Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_WIKI_Google.csv
Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_WIKI_Google.csv
Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_WIKI_Google.csv
Share : WIKI/MSFT
Rows : 8076
Columns: 14
Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: WIKI_Microsoft
Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_WIKI_Microsoft.csv
Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_WIKI_Microsoft.csv
Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_WIKI_Microsoft.csv
Share : WIKI/UPS
Rows : 4622
Columns: 14
Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: WIKI_UPS
Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_WIKI_UPS.csv
Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_WIKI_UPS.csv
Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_WIKI_UPS.csv
Share : WIKI/AMZN
Rows : 5248
Columns: 14
Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: WIKI_Amazon
Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_WIKI_Amazon.csv
Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_WIKI_Amazon.csv
Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_WIKI_Amazon.csv
Share : LOCALBTC/USD
```

Rows : 1863

Columns: 6

Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: LOCALBTC_USD

Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_LOCALBTC_USD.csv

Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_LOCALBTC_USD.csv

Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_LOCALBTC_USD.csv

Share : PERTH/AUD_USD_M

Rows : 340

Columns: 8

Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: PERTH_AUD_USD_M

Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_PERTH_AUD_USD_M.csv

Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_PERTH_AUD_USD_M.csv

Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_PERTH_AUD_USD_M.csv

Share : PERTH/AUD_USD_D

Rows : 7989

Columns: 8

Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: PERTH_AUD_USD_D

Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_PERTH_AUD_USD_D.csv

Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_PERTH_AUD_USD_D.csv

Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_PERTH_AUD_USD_D.csv

Share : FRED/GDP

Rows : 290

Columns: 3

Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: FRED/GDP

Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_FRED-GDP.csv

Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_FRED-GDP.csv

Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_FRED-GDP.csv

Share : FED/RXI_US_N_A_UK

Rows : 49

Columns: 3

Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: FED_RXI_US_N_A_UK

Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_FED_RXI_US_N_A_UK.csv

Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_FED_RXI_US_N_A_UK.csv

Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_FED_RXI_US_N_A_UK.csv

Share : FED/RXI_N_A_CA

Rows : 49

Columns: 3

Storing : C:/VKHCG/04-Clark/03-Process/SQLite/clark.db Table: FED_RXI_N_A_CA

Storing : C:/VKHCG/04-Clark/01-Retrieve/01-EDS/02-Python/Retrieve_FED_RXI_N_A_CA.csv

Storing : C:/VKHCG/04-Clark/02-Assess/01-EDS/02-Python/Assess_FED_RXI_N_A_CA.csv

Storing : C:/VKHCG/04-Clark/03-Process/01-EDS/02-Python/Process_FED_RXI_N_A_CA.csv

Done!!

Practical 6**TRANSFORMING DATA****Transform Superstep**

The Transform superstep allows you, as a data scientist, to take data from the data vault and formulate answers to questions raised by your investigations. The transformation step is the data science process that converts results into insights. It takes standard data science techniques and methods to attain insight and knowledge about the data that then can be transformed into actionable decisions, which, through storytelling, you can explain to non-data scientists what you have discovered in the data lake.

To illustrate the consolidation process, the example shows a person being borne. Open a new file in the Python editor and save it as Transform-Gunnarsson_is_Born.py in the directory

C: \VKHCG\01-Vermeulen\04-Transform.

```
# -*- coding: utf-8 -*-
```

```
import sys
```

```
import os
```

```
from datetime import datetime
```

```
from pytz import timezone
```

```
import pandas as pd
```

```
import sqlite3 as sq
```

```
import uuid
```

```
pd.options.mode.chained_assignment = None
```

```
Base='C:/VKHCG'
```

```
print('#####')
```

```
print('Working Base :',Base, ' using ', sys.platform)
```

```
print('#####')
```

```
Company='01-Vermeulen'
```

```
InputDir='00-RawData'
```

```
InputFileName='VehicleData.csv'
```

```
sDataBaseDir=Base + '/' + Company + '/04-Transform/SQLite'
```

```
if not os.path.exists(sDataBaseDir):
```

```
os.makedirs(sDataBaseDir)
```

```
sDatabaseName=sDataBaseDir + '/Vermeulen.db'
```

```
conn1 = sq.connect(sDatabaseName)
```

```
sDataVaultDir=Base + '/88-DV'
```

```
if not os.path.exists(sDataVaultDir):
```

```
os.makedirs(sDataVaultDir)
```

```
sDatabaseName=sDataVaultDir + '/datavault.db'
```

```
conn2 = sq.connect(sDatabaseName)
```

```
sDataWarehouseDir=Base + '/99-DW'
```

```
if not os.path.exists(sDataWarehouseDir):
```

```
os.makedirs(sDataWarehouseDir)
```

```
sDatabaseName=sDataWarehouseDir + '/datawarehouse.db'
```

```
conn3 = sq.connect(sDatabaseName)
```

```
print('\n#####')
```

```
print('Time Category')
```

```

print('UTC Time')
BirthDateUTC = datetime(1960,12,20,10,15,0)
BirthDateZoneUTC=BirthDateUTC.replace(tzinfo=timezone('UTC'))
BirthDateZoneStr=BirthDateZoneUTC.strftime("%Y-%m-%d %H:%M:%S")
BirthDateZoneUTCStr=BirthDateZoneUTC.strftime("%Y-%m-%d %H:%M:%S (%Z) (%z)")
print(BirthDateZoneUTCStr)
print('#####')
print('Birth Date in Reykjavik :)')
BirthZone = 'Atlantic/Reykjavik'
BirthDate = BirthDateZoneUTC.astimezone(timezone(BirthZone))
BirthDateStr=BirthDate.strftime("%Y-%m-%d %H:%M:%S (%Z) (%z)")
BirthDateLocal=BirthDate.strftime("%Y-%m-%d %H:%M:%S")
print(BirthDateStr)
print('#####')
IDZoneNumber=str(uuid.uuid4())
sDateTimeKey=BirthDateZoneStr.replace(' ','-').replace(':', '-')
TimeLine=[('ZoneBaseKey', ['UTC']),
('IDNumber', [IDZoneNumber]),
('DateTimeKey', [sDateTimeKey]),
('UTCDateTimeValue', [BirthDateZoneUTC]),
('Zone', [BirthZone]),
('DateTimeValue', [BirthDateStr])]
TimeFrame = pd.DataFrame.from_items(TimeLine)
TimeHub=TimeFrame[['IDNumber','ZoneBaseKey','DateTimeKey','DateTimeValue']]
TimeHubIndex=TimeHub.set_index(['IDNumber'],inplace=False)
sTable = 'Hub-Time-Gunnarsson'
print("\n#####")
print('Storing :',sDatabaseName,"\n Table:",sTable)
print("\n#####")
TimeHubIndex.to_sql(sTable, conn2, if_exists="replace")
sTable = 'Dim-Time-Gunnarsson'
TimeHubIndex.to_sql(sTable, conn3, if_exists="replace")
TimeSatellite=TimeFrame[['IDNumber','DateTimeKey','Zone','DateTimeValue']]
TimeSatelliteIndex=TimeSatellite.set_index(['IDNumber'],inplace=False)
BirthZoneFix=BirthZone.replace(' ','-').replace('/', '-')
sTable = 'Satellite-Time-' + BirthZoneFix + '-Gunnarsson'
print("\n#####")
print('Storing :',sDatabaseName,"\n Table:",sTable)
print("\n#####")
TimeSatelliteIndex.to_sql(sTable, conn2, if_exists="replace")
sTable = 'Dim-Time-' + BirthZoneFix + '-Gunnarsson'
TimeSatelliteIndex.to_sql(sTable, conn3, if_exists="replace")
print("\n#####")
print('Person Category')

```

Seat No.: 3254185

```
FirstName = 'Guðmundur'
LastName = 'Gunnarsson'
print('Name:',FirstName,LastName)
print('Birth Date:',BirthDateLocal)
print('Birth Zone:',BirthZone)
print('UTC Birth Date:',BirthDateZoneStr)
print('#####')
IDPersonNumber=str(uuid.uuid4())
PersonLine=[('IDNumber', [IDPersonNumber]),
('FirstName', [FirstName]),
('LastName', [LastName]),
('Zone', ['UTC']),
('DateTimeValue', [BirthDateZoneStr])]
PersonFrame = pd.DataFrame.from_items(PersonLine)
TimeHub=PersonFrame
TimeHubIndex=TimeHub.set_index(['IDNumber'],inplace=False)
sTable = 'Hub-Person-Gunnarsson'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
TimeHubIndex.to_sql(sTable, conn2, if_exists="replace")
sTable = 'Dim-Person-Gunnarsson'
TimeHubIndex.to_sql(sTable, conn3, if_exists="replace")
```

OUTPUT :

Guðmundur Gunnarsson was born on December 20, 1960, at 9:15 in Landspítali,Hringbraut 101, 101 Reykjavík, Iceland.

```
>>>
RESTART: C:\VKHCG\01-Vermeulen\04-Transform\Transform-Gunnarsson_is_Born.py
Working Base : C:\VKHCG using win32
Time Category
UTC Time
1960-12-20 10:15:00 (UTC) (+0000)
#####
Birth Date in Reykjavik :
1960-12-20 09:15:00 (-01) (-0100)
#####
Storing : C:\VKHCG\99-DW\datawarehouse.db
Table: Hub-Time-Gunnarsson
#####
Storing : C:\VKHCG\99-DW\datawarehouse.db
Table: Satellite-Time-Atlantic-Reykjavik-Gunnarsson
#####
Person Category
Name: Guðmundur Gunnarsson
Birth Date: 1960-12-20 09:15:00
Birth Zone: Atlantic/Reykjavik
UTC Birth Date: 1960-12-20 10:15:00
#####
Storing : C:\VKHCG\99-DW\datawarehouse.db
Table: Hub-Person-Gunnarsson
#####
```

You must build three items: **dimension Person**, **dimension Time**, and **factPersonBornAtTime**.
Open your Python editor and create a file named Transform-Gunnarsson-Sun-Model.py in directory
C:\VKHCG\01-Vermeulen\04-Transform.

```
# -*- coding: utf-8 -*-
```

```
import sys
```

```
import os
```

```
from datetime import datetime
```

```
from pytz import timezone
```

```
import pandas as pd
```

```
import sqlite3 as sq
```

```
import uuid
```

```
pd.options.mode.chained_assignment = None
```

```
if sys.platform == 'linux':
```

```
Base=os.path.expanduser('~') + '/VKHCG'
```

```
else:
```

```
Base='C:/VKHCG'
```

```
print('#####')
```

```
print('Working Base :',Base, ' using ', sys.platform)
```

```
print('#####')
```

```
Company='01-Vermeulen'
```

```
sDataBaseDir=Base + '/' + Company + '/04-Transform/SQLite'
```

```
if not os.path.exists(sDataBaseDir):
```

```
os.makedirs(sDataBaseDir)
```

```
sDatabaseName=sDataBaseDir + '/Vermeulen.db'
```

```
conn1 = sq.connect(sDatabaseName)
```

```
sDataWarehousetDir=Base + '/99-DW'
```

```
if not os.path.exists(sDataWarehousetDir):
```

```
os.makedirs(sDataWarehousetDir)
```

```
sDatabaseName=sDataWarehousetDir + '/datawarehouse.db'
```

```
conn2 = sq.connect(sDatabaseName)
```

```
print('\n#####')
```

```
print('Time Dimension')
```

```
BirthZone = 'Atlantic/Reykjavik'
```

```
BirthDateUTC = datetime(1960,12,20,10,15,0)
```

```
BirthDateZoneUTC=BirthDateUTC.replace(tzinfo=timezone('UTC'))
```

```
BirthDateZoneStr=BirthDateZoneUTC.strftime("%Y-%m-%d %H:%M:%S")
```

```
BirthDateZoneUTCStr=BirthDateZoneUTC.strftime("%Y-%m-%d %H:%M:%S (%Z) (%z)")
```

```
BirthDate = BirthDateZoneUTC.astimezone(timezone(BirthZone))
```

```
BirthDateStr=BirthDate.strftime("%Y-%m-%d %H:%M:%S (%Z) (%z)")
```

```
BirthDateLocal=BirthDate.strftime("%Y-%m-%d %H:%M:%S")
```

```
IDTimeNumber=str(uuid.uuid4())
```

```
TimeLine=[('TimeID', [IDTimeNumber]),
```

```
('UTCDate', [BirthDateZoneStr]),
```

```
('LocalTime', [BirthDateLocal]),
```

```

('TimeZone', [BirthZone]))
TimeFrame = pd.DataFrame.from_items(TimeLine)
DimTime=TimeFrame
DimTimeIndex=DimTime.set_index(['TimeID'],inplace=False)
sTable = 'Dim-Time'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimTimeIndex.to_sql(sTable, conn1, if_exists="replace")
DimTimeIndex.to_sql(sTable, conn2, if_exists="replace")
print("\n#####")
print('Dimension Person')
print("\n#####")
FirstName = 'Guðmundur'
LastName = 'Gunnarsson'
IDPersonNumber=str(uuid.uuid4())
PersonLine=[('PersonID', [IDPersonNumber]),
('FirstName', [FirstName]),
('LastName', [LastName]),
('Zone', ['UTC']),
('DateTimeValue', [BirthDateZoneStr])]
PersonFrame = pd.DataFrame.from_items(PersonLine)
DimPerson=PersonFrame
DimPersonIndex=DimPerson.set_index(['PersonID'],inplace=False)
sTable = 'Dim-Person'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimPersonIndex.to_sql(sTable, conn1, if_exists="replace")
DimPersonIndex.to_sql(sTable, conn2, if_exists="replace")
print("\n#####")
print('Fact - Person - time')
print("\n#####")
IDFactNumber=str(uuid.uuid4())
PersonTimeLine=[('IDNumber', [IDFactNumber]),
('IDPersonNumber', [IDPersonNumber]),
('IDTimeNumber', [IDTimeNumber])]
PersonTimeFrame = pd.DataFrame.from_items(PersonTimeLine)
FctPersonTime=PersonTimeFrame
FctPersonTimeIndex=FctPersonTime.set_index(['IDNumber'],inplace=False)
sTable = 'Fact-Person-Time'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")

```

Seat No.: 3254185

```
FctPersonTimeIndex.to_sql(sTable, conn1, if_exists="replace")
FctPersonTimeIndex.to_sql(sTable, conn2, if_exists="replace")
```

OUTPUT:

Building a Data Warehouse

Open the Transform-Sun-Models.py file from directory C:\VKHCG\01-Vermeulen\04-Transform.

```
# -*- coding: utf-8 -*-
```

```
import sys
import os
from datetime import datetime
from pytz import timezone
import pandas as pd
import sqlite3 as sq
import uuid
pd.options.mode.chained_assignment = None
if sys.platform == 'linux':
    Base=os.path.expanduser('~') + '/VKHCG'
else:
    Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='01-Vermeulen'
sDataBaseDir=Base + '/' + Company + '/04-Transform/SQLite'
if not os.path.exists(sDataBaseDir):
```

```

os.makedirs(sDataBaseDir)
sDatabaseName=sDataBaseDir + '/Vermeulen.db'
conn1 = sq.connect(sDatabaseName)
sDataVaultDir=Base + '/88-DV'
if not os.path.exists(sDataVaultDir):
os.makedirs(sDataVaultDir)
sDatabaseName=sDataVaultDir + '/datavault.db'
conn2 = sq.connect(sDatabaseName)
sDataWarehouseDir=Base + '/99-DW'
if not os.path.exists(sDataWarehouseDir):
os.makedirs(sDataWarehouseDir)
sDatabaseName=sDataWarehouseDir + '/datawarehouse.db'
conn3 = sq.connect(sDatabaseName)
sSQL=" SELECT DateTimeValue FROM [Hub-Time];"
DateDataRow=pd.read_sql_query(sSQL, conn2)
DateData=DateDataRow.head(1000)
print(DateData)
print("\n#####")
print("Time Dimension")
print("\n#####")
t=0
mt=DateData.shape[0]
for i in range(mt):
BirthZone = ('Atlantic/Reykjavik','Europe/London','UCT')
for j in range(len(BirthZone)):
t+=1
print(t,mt*3)
BirthDateUTC = datetime.strptime(DateData['DateTimeValue'][i],"%Y-%m-%d %H:%M:%S")
BirthDateZoneUTC=BirthDateUTC.replace(tzinfo=timezone('UTC'))
BirthDateZoneStr=BirthDateZoneUTC.strftime("%Y-%m-%d %H:%M:%S")
BirthDateZoneUTCStr=BirthDateZoneUTC.strftime("%Y-%m-%d %H:%M:%S (%Z) (%z)")
BirthDate = BirthDateZoneUTC.astimezone(timezone(BirthZone[j]))
BirthDateStr=BirthDate.strftime("%Y-%m-%d %H:%M:%S (%Z) (%z)")
BirthDateLocal=BirthDate.strftime("%Y-%m-%d %H:%M:%S")
IDTimeNumber=str(uuid.uuid4())
TimeLine=[('TimeID', [str(IDTimeNumber)]),
('UTCDate', [str(BirthDateZoneStr)]),
('LocalTime', [str(BirthDateLocal)]),
('TimeZone', [str(BirthZone)])]
if t==1:
TimeFrame = pd.DataFrame.from_items(TimeLine)
else:
TimeRow = pd.DataFrame.from_items(TimeLine)
TimeFrame=TimeFrame.append(TimeRow)

```

```
DimTime=TimeFrame
DimTimeIndex=DimTime.set_index(['TimeID'],inplace=False)
sTable = 'Dim-Time'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimTimeIndex.to_sql(sTable, conn1, if_exists="replace")
DimTimeIndex.to_sql(sTable, conn3, if_exists="replace")
sSQL=" SELECT " + \
" FirstName," + \
" SecondName," + \
" LastName," + \
" BirthDateKey " + \
" FROM [Hub-Person];"
PersonDataRow=pd.read_sql_query(sSQL, conn2)
PersonData=PersonDataRow.head(1000)
print("\n#####")
print('Dimension Person')
print("\n#####")
t=0
mt=DateData.shape[0]
for i in range(mt):
    t+=1
    print(t,mt)
    FirstName = str(PersonData["FirstName"])
    SecondName = str(PersonData["SecondName"])
    if len(SecondName) > 0:
        SecondName=""
    LastName = str(PersonData["LastName"])
    BirthDateKey = str(PersonData["BirthDateKey"])
    IDPersonNumber=str(uuid.uuid4())
    PersonLine=[('PersonID', [str(IDPersonNumber)]),
    ('FirstName', [FirstName]),
    ('SecondName', [SecondName]),
    ('LastName', [LastName]),
    ('Zone', [str('UTC')]),
    ('BirthDate', [BirthDateKey])]
    if t==1:
        PersonFrame = pd.DataFrame.from_items(PersonLine)
    else:
        PersonRow = pd.DataFrame.from_items(PersonLine)
        PersonFrame = PersonFrame.append(PersonRow)
    DimPerson=PersonFrame
    print(DimPerson)
```

```
DimPersonIndex=DimPerson.set_index(['PersonID'],inplace=False)
sTable = 'Dim-Person'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimPersonIndex.to_sql(sTable, conn1, if_exists="replace")
DimPersonIndex.to_sql(sTable, conn3, if_exists="replace")
```

OUTPUT:

You have successfully performed data vault to data warehouse transformation.

Simple Linear Regression

Linear regression is used if there is a relationship or significant association between the variables. This can be checked by scatterplots. If no linear association appears between the variables, fitting a linear regression model to the data will not provide a useful model.

A linear regression line has equations in the following form: $Y = a + bX$,

Where, X = explanatory variable and Y = dependent variable

b = slope of the line & a = intercept (the value of y when x = 0)

-*- coding: utf-8 -*-

```
import sys
import os
import pandas as pd
import sqlite3 as sq
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='01-Vermeulen'
sDataBaseDir=Base + '/' + Company + '/04-Transform/SQLite'
if not os.path.exists(sDataBaseDir):
    os.makedirs(sDataBaseDir)
sDatabaseName=sDataBaseDir + '/Vermeulen.db'
conn1 = sq.connect(sDatabaseName)
sDataVaultDir=Base + '/88-DV'
if not os.path.exists(sDataVaultDir):
    os.makedirs(sDataVaultDir)
sDatabaseName=sDataVaultDir + '/datavault.db'
conn2 = sq.connect(sDatabaseName)
sDataWarehouseDir=Base + '/99-DW'
if not os.path.exists(sDataWarehouseDir):
    os.makedirs(sDataWarehouseDir)
```

```
sDatabaseName=sDataWarehouseDir + '/datawarehouse.db'
conn3 = sq.connect(sDatabaseName)
t=0
tMax=((300-100)/10)*((300-30)/5)
for heightSelect in range(100,300,10):
for weightSelect in range(30,300,5):
height = round(heightSelect/100,3)
weight = int(weightSelect)
bmi = weight/(height*height)
if bmi <= 18.5:
BMI_Result=1
elif bmi > 18.5 and bmi < 25:
BMI_Result=2
elif bmi > 25 and bmi < 30:
BMI_Result=3
elif bmi > 30:
BMI_Result=4
else:
BMI_Result=0
PersonLine=[('PersonID', [str(t)]),
('Height', [height]),
('Weight', [weight]),
('bmi', [bmi]),
('Indicator', [BMI_Result])]
t+=1
print('Row:',t,'of',tMax)
if t==1:
PersonFrame = pd.DataFrame.from_items(PersonLine)
else:
PersonRow = pd.DataFrame.from_items(PersonLine)
PersonFrame = PersonFrame.append(PersonRow)
DimPerson=PersonFrame
DimPersonIndex=DimPerson.set_index(['PersonID'],inplace=False)
sTable = 'Transform-BMI'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimPersonIndex.to_sql(sTable, conn1, if_exists="replace")
sTable = 'Person-Satellite-BMI'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimPersonIndex.to_sql(sTable, conn2, if_exists="replace")
sTable = 'Dim-BMI'
```

```

print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimPersonIndex.to_sql(sTable, conn3, if_exists="replace")
fig = plt.figure()
PlotPerson=DimPerson[DimPerson['Indicator']==1]
x=PlotPerson['Height']
y=PlotPerson['Weight']
plt.plot(x, y, ".")
PlotPerson=DimPerson[DimPerson['Indicator']==2]
x=PlotPerson['Height']
y=PlotPerson['Weight']
plt.plot(x, y, "o")
PlotPerson=DimPerson[DimPerson['Indicator']==3]
x=PlotPerson['Height']
y=PlotPerson['Weight']
plt.plot(x, y, "+")
PlotPerson=DimPerson[DimPerson['Indicator']==4]
x=PlotPerson['Height']
y=PlotPerson['Weight']
plt.plot(x, y, "^")
plt.axis('tight')
plt.title("BMI Curve")
plt.xlabel("Height(meters)")
plt.ylabel("Weight(kg)")
plt.plot()
# Load the diabetes dataset
diabetes = datasets.load_diabetes()
# Use only one feature
diabetes_X = diabetes.data[:, np.newaxis, 2]
diabetes_X_train = diabetes_X[:-30]
diabetes_X_test = diabetes_X[-50:]
diabetes_y_train = diabetes.target[:-30]
diabetes_y_test = diabetes.target[-50:]
regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
diabetes_y_pred = regr.predict(diabetes_X_test)
print('Coefficients: \n', regr.coef_)
print("Mean squared error: %.2f"
      % mean_squared_error(diabetes_y_test, diabetes_y_pred))
print('Variance score: %.2f' % r2_score(diabetes_y_test, diabetes_y_pred))
plt.scatter(diabetes_X_test, diabetes_y_test, color='black')
plt.plot(diabetes_X_test, diabetes_y_pred, color='blue', linewidth=3)
plt.xticks(())

```

```
plt.yticks()
plt.axis('tight')
plt.title("Diabetes")
plt.xlabel("BMI")
plt.ylabel("Age")
plt.show()
```

OUTPUT:

```
Row: 1077 of 1080.0
Row: 1078 of 1080.0
Row: 1079 of 1080.0
Row: 1080 of 1080.0

#####
Storing : C:/VKHCG/99-DW/datawarehouse.db
Table: Transform-BMI

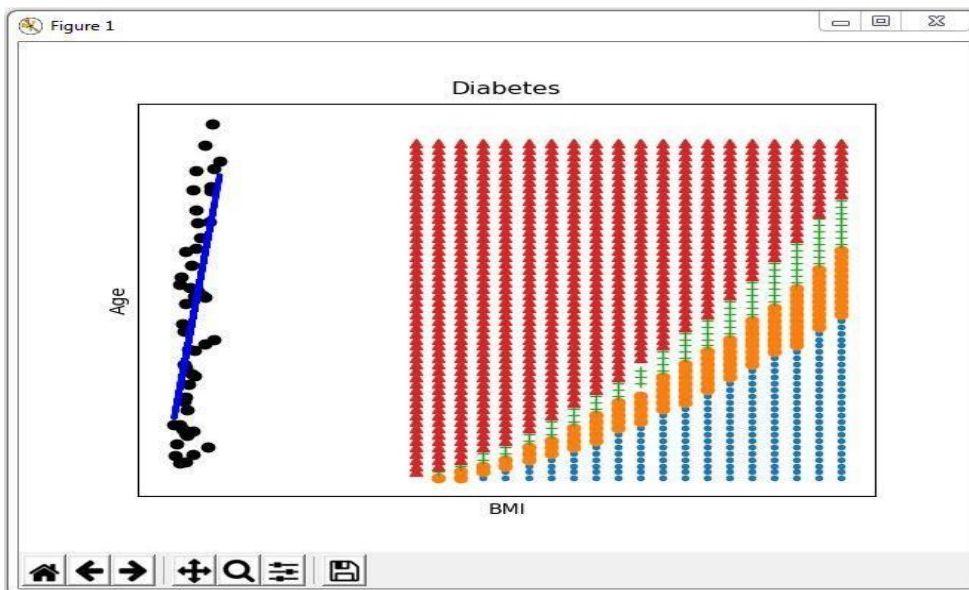
#####

#####
Storing : C:/VKHCG/99-DW/datawarehouse.db
Table: Person-Satellite-BMI

#####

#####
Storing : C:/VKHCG/99-DW/datawarehouse.db
Table: Dim-BMI

#####
>>>
```



Practical 7**ORGANIZING DATA****Organize Superstep**

The Organize superstep takes the complete data warehouse you built at the end of the Transform superstep and subsections it into business-specific data marts. A data mart is the access layer of the data warehouse environment built to expose data to the users. The data mart is a subset of the data warehouse and is generally oriented to a specific business group.

Horizontal Style

Performing horizontal-style slicing or subsetting of the data warehouse is achieved by applying a filter technique that forces the data warehouse to show only the data for a specific preselected set of filtered outcomes against the data population. The horizontal-style slicing selects the subset of rows from the population while preserving the columns. That is, the data science tool can see the complete record for the records in the subset of records.

C:\VKHCG\01-Vermeulen\05-Organise\ Organize-Horizontal.py

```
# -*- coding: utf-8 -*-
```

```
import sys
```

```
import os
```

```
import pandas as pd
```

```
import sqlite3 as sq
```

```
Base='C:/VKHCG'
```

```
print('#####')
```

```
print('Working Base :',Base, ' using ', sys.platform)
```

```
print('#####')
```

```
Company='01-Vermeulen'
```

```
sDataWarehouseDir=Base + '/99-DW'
```

```
if not os.path.exists(sDataWarehouseDir):
```

```
os.makedirs(sDataWarehouseDir)
```

```
sDatabaseName=sDataWarehouseDir + '/datawarehouse.db'
```

```
conn1 = sq.connect(sDatabaseName)
```

```
sDatabaseName=sDataWarehouseDir + '/datamart.db'
```

```
conn2 = sq.connect(sDatabaseName)
```

```
print('#####')
```

```
sTable = 'Dim-BMI'
```

```
print('Loading :',sDatabaseName,' Table:',sTable)
```

```
sSQL="SELECT * FROM [Dim-BMI];"
```

```
PersonFrame0=pd.read_sql_query(sSQL, conn1)
```

```
print('#####')
```

```
sTable = 'Dim-BMI'
```

```
print('Loading :',sDatabaseName,' Table:',sTable)
```

```
sSQL="SELECT PersonID,\ Height,\Weight,\bmi,\ Indicator\FROM [Dim-BMI]\WHERE \Height > 1.5 \
and Indicator = 1\ORDER BY \Height,\Weight;"
```

```
PersonFrame1=pd.read_sql_query(sSQL, conn1)
```

```
DimPerson=PersonFrame1
```

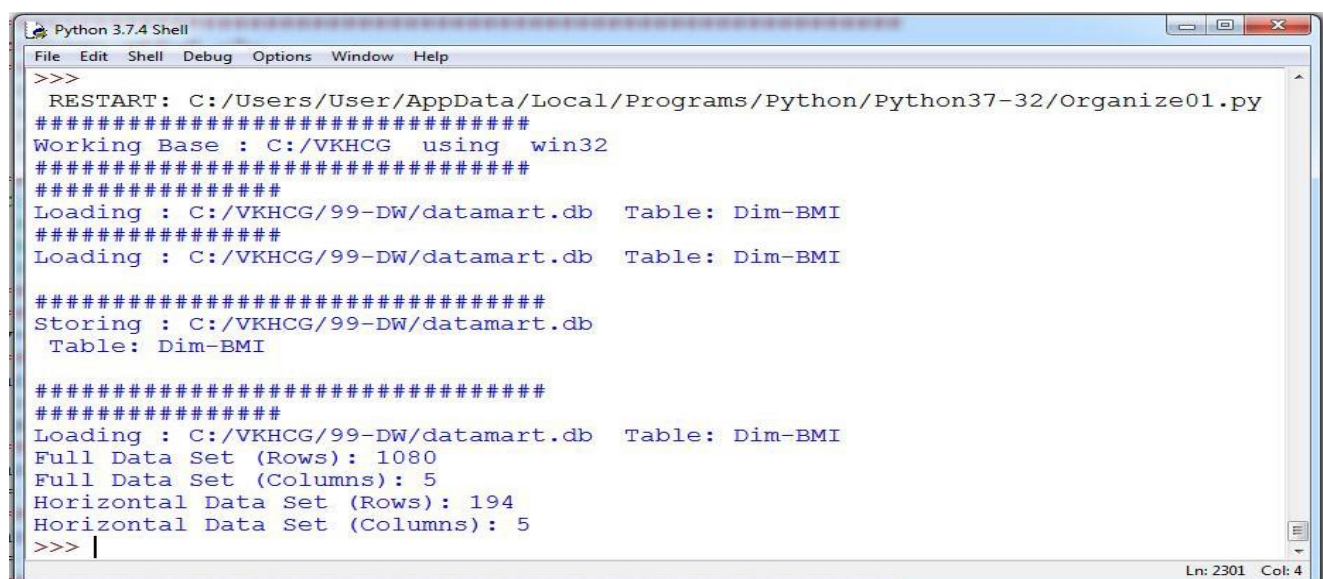
```
DimPersonIndex=DimPerson.set_index(['PersonID'],inplace=False)
```

```

sTable = 'Dim-BMI'
print('\n#####')
print('Storing :',sDatabaseName,'\n Table:',sTable)
print('\n#####')
#DimPersonIndex.to_sql(sTable, conn2, if_exists="replace")
print('#####')
sTable = 'Dim-BMI'
print('Loading :',sDatabaseName,' Table:',sTable)
sSQL="SELECT * FROM [Dim-BMI];"
PersonFrame2=pd.read_sql_query(sSQL, conn2)
print('Full Data Set (Rows):', PersonFrame0.shape[0])
print('Full Data Set (Columns):', PersonFrame0.shape[1])
print('Horizontal Data Set (Rows):', PersonFrame2.shape[0])
print('Horizontal Data Set (Columns):', PersonFrame2.shape[1])

```

OUTPUT:



```

Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
>>>
RESTART: C:/Users/User/AppData/Local/Programs/Python/Python37-32/Organize01.py
#####
Working Base : C:/VKHCG using win32
#####
Loading : C:/VKHCG/99-DW/datamart.db Table: Dim-BMI
#####
Loading : C:/VKHCG/99-DW/datamart.db Table: Dim-BMI

#####
Storing : C:/VKHCG/99-DW/datamart.db
Table: Dim-BMI

#####
#####
Loading : C:/VKHCG/99-DW/datamart.db Table: Dim-BMI
Full Data Set (Rows): 1080
Full Data Set (Columns): 5
Horizontal Data Set (Rows): 194
Horizontal Data Set (Columns): 5
>>> |
Ln: 2301 Col: 4

```

The horizontal-style slicing selects the 194 subset of rows from the 1080 rows while preserving the columns

Vertical Style

Performing vertical-style slicing or subsetting of the data warehouse is achieved by applying a filter technique that forces the data warehouse to show only the data for specific preselected filtered outcomes against the data population. The vertical-style slicing selects the subset of columns from the population, while preserving the rows. That is, the data science tool can see only the preselected columns from a record for all the records in the population.

C:\VKHCG\01-Vermeulen\05-Organise\ Organize-Vertical.py

-*- coding: utf-8 -*-

```

import sys
import os
import pandas as pd
import sqlite3 as sq

```

```

Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='01-Vermeulen'
sDataWarehouseDir=Base + '/99-DW'
if not os.path.exists(sDataWarehouseDir):
os.makedirs(sDataWarehouseDir)
sDatabaseName=sDataWarehouseDir + '/datawarehouse.db'
conn1 = sq.connect(sDatabaseName)
sDatabaseName=sDataWarehouseDir + '/datamart.db'
conn2 = sq.connect(sDatabaseName)
print('#####')
sTable = 'Dim-BMI'
print('Loading :',sDatabaseName,' Table:',sTable)
sSQL="SELECT * FROM [Dim-BMI];"
PersonFrame0=pd.read_sql_query(sSQL, conn1)
print('#####')
sTable = 'Dim-BMI'
print('Loading :',sDatabaseName,' Table:',sTable)
print('#####')
sSQL="SELECT \
Height,\
Weight,\

Indicator\
FROM [Dim-BMI];"
PersonFrame1=pd.read_sql_query(sSQL, conn1)
DimPerson=PersonFrame1
DimPersonIndex=DimPerson.set_index(['Indicator'],inplace=False)
sTable = 'Dim-BMI-Vertical'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimPersonIndex.to_sql(sTable, conn2, if_exists="replace")
print('#####')
sTable = 'Dim-BMI-Vertical'
print('Loading :',sDatabaseName,' Table:',sTable)
sSQL="SELECT * FROM [Dim-BMI-Vertical];"
PersonFrame2=pd.read_sql_query(sSQL, conn2)
print('#####')
print('Full Data Set (Rows):', PersonFrame0.shape[0])
print('Full Data Set (Columns):', PersonFrame0.shape[1])
print('#####')

```

```
print('Horizontal Data Set (Rows):', PersonFrame2.shape[0])
print('Horizontal Data Set (Columns):', PersonFrame2.shape[1])
print('#####')
```

OUTPUT:

```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 19:29:22) [MSC v.1916 32
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more informatio
>>>
===== RESTART: C:\VKHCG\01-Vermeulen\05-Organise\Organize-Vertic-
#####
Working Base : C:\VKHCG using win32
#####
Loading : C:\VKHCG\99-DW\datamart.db Table: Dim-BMI
#####
Loading : C:\VKHCG\99-DW\datamart.db Table: Dim-BMI-Vertical
#####
Storing : C:\VKHCG\99-DW\datamart.db
Table: Dim-BMI-Vertical
#####
#####
Loading : C:\VKHCG\99-DW\
#####
Full Data Set (Rows)
Full Data Set (Columns)
#####
Horizontal Data Set (Rows)
Horizontal Data Set (Columns)
#####
>>> |
```

The vertical-style slicing selects 3 of 5 from the population, while preserving the rows [1080].

Island Style

Performing island-style slicing or subsetting of the data warehouse is achieved by applying a combination of horizontal- and vertical-style slicing. This generates a subset of specific rows and specific columns reduced at the same time.

C:\VKHCG\01-Vermeulen\05-Organise\ Organize-Island.py

```
# -*- coding: utf-8 -*-
```

```
import sys
```

```
import os
```

```
import pandas as pd
```

```
import sqlite3 as sq
```

```
Base='C:/VKHCG'
```

```
print('#####')
```

```
print('Working Base :',Base, ' using ', sys.platform)
```

```
print('#####')
```

```
Company='01-Vermeulen'
```

```
sDataWarehouseDir=Base + '/99-DW'
```

```
if not os.path.exists(sDataWarehouseDir):
```

```
os.makedirs(sDataWarehouseDir)
```

```
sDatabaseName=sDataWarehouseDir + '/datawarehouse.db'
```

```
conn1 = sq.connect(sDatabaseName)
sDatabaseName=sDataWarehouseDir + '/datamart.db'
conn2 = sq.connect(sDatabaseName)
print('#####')
sTable = 'Dim-BMI'
print('Loading :',sDatabaseName,' Table:',sTable)
sSQL="SELECT * FROM [Dim-BMI];"
PersonFrame0=pd.read_sql_query(sSQL, conn1)
print('#####')
sTable = 'Dim-BMI'
print('Loading :',sDatabaseName,' Table:',sTable)
sSQL="SELECT \Height,\Weight,\Indicator\FROM [Dim-BMI]\WHERE Indicator > 2\ORDER BY
Height,\Weight;"
PersonFrame1=pd.read_sql_query(sSQL, conn1)
DimPerson=PersonFrame1
DimPersonIndex=DimPerson.set_index(['Indicator'],inplace=False)
sTable = 'Dim-BMI-Vertical'
print("\n#####")
print('Storing :',sDatabaseName,'\n Table:',sTable)
print("\n#####")
DimPersonIndex.to_sql(sTable, conn2, if_exists="replace")
print('#####')
sTable = 'Dim-BMI-Vertical'
print('Loading :',sDatabaseName,' Table:',sTable)
print('#####')
sSQL="SELECT * FROM [Dim-BMI-Vertical];"
PersonFrame2=pd.read_sql_query(sSQL, conn2)
print('#####')
print('Full Data Set (Rows):', PersonFrame0.shape[0])
print('Full Data Set (Columns):', PersonFrame0.shape[1])
print('#####')
print('Horizontal Data Set (Rows):', PersonFrame2.shape[0])
print('Horizontal Data Set (Columns):', PersonFrame2.shape[1])
print('#####')
```

OUTPUT:

Secure Vault Style

The secure vault is a version of one of the horizontal, vertical, or island slicing techniques, but the outcome is also attached to the person who performs the query. This is common in multi-security environments, where different users are allowed to see different data sets.

This process works well, if you use a role-based access control (RBAC) approach to restricting system access to authorized users. The security is applied against the “role,” and a person can then, by the security system, simply be added or removed from the role, to enable or disable access.

C:\VKHCG\01-Vermeulen\05-Organise\ Organize-Secure-Vault.py

-*- coding: utf-8 -*-

```
import sys
import os
import pandas as pd
import sqlite3 as sq
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
Company='01-Vermeulen'
sDataWarehouseDir=Base + '/99-DW'
if not os.path.exists(sDataWarehouseDir):
os.makedirs(sDataWarehouseDir)
sDatabaseName=sDataWarehouseDir + '/datawarehouse.db'
conn1 = sq.connect(sDatabaseName)
sDatabaseName=sDataWarehouseDir + '/datamart.db'
conn2 = sq.connect(sDatabaseName)
print('#####')
sTable = 'Dim-BMI'
print('Loading :',sDatabaseName,' Table:',sTable)
sSQL="SELECT * FROM [Dim-BMI];"
PersonFrame0=pd.read_sql_query(sSQL, conn1)
```

```
print('#####')
sTable = 'Dim-BMI'
print('Loading :',sDatabaseName,' Table:',sTable)
sSQL="SELECT \Height,\Weight,\Indicator,\CASE Indicator\WHEN 1 THEN 'Pip'\
WHEN 2 THEN 'Norman'\WHEN 3 THEN 'Grant'\ELSE 'Sam'\END AS Name\FROM [Dim-BMI]\
WHERE Indicator > 2\ORDER BY \Height,\Weight;"
PersonFrame1=pd.read_sql_query(sSQL, conn1)
DimPerson=PersonFrame1
DimPersonIndex=DimPerson.set_index(['Indicator'],inplace=False)
sTable = 'Dim-BMI-Secure'
print('\n#####')
print('Storing :',sDatabaseName,'\n Table:',sTable)
print('\n#####')
DimPersonIndex.to_sql(sTable, conn2, if_exists="replace")
print('#####')
sTable = 'Dim-BMI-Secure'
print('Loading :',sDatabaseName,' Table:',sTable)
print('#####')
sSQL="SELECT * FROM [Dim-BMI-Secure] WHERE Name = 'Sam';"
PersonFrame2=pd.read_sql_query(sSQL, conn2)
print('#####')
print('Full Data Set (Rows):', PersonFrame0.shape[0])
print('Full Data Set (Columns):', PersonFrame0.shape[1])
print('#####')
print('Horizontal Data Set (Rows):', PersonFrame2.shape[0])
print('Horizontal Data Set (Columns):', PersonFrame2.shape[1])
print('Only Sam Data')
print(PersonFrame2.head())
print('#####')
```

OUTPUT:

```

Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
>>>
===== RESTART: C:\VKHCG\01-Vermeulen\05-Organise\Organize-Secure-Vault.py =====
#####
Working Base : C:\VKHCG using win32
#####
#####
Loading : C:\VKHCG\99-DW\datamart.db Table: Dim-BMI
#####
Loading : C:\VKHCG\99-DW\datamart.db Table: Dim-BMI

#####
Storing : C:\VKHCG\99-DW\datamart.db
Table: Dim-BMI-Secure

#####
#####
Loading : C:\VKHCG\99-DW\datamart.db Table: Dim-BMI-Secure
#####
#####
Full Data Set (Rows): 1080
Full Data Set (Columns): 5
#####
Horizontal Data Set (Rows): 692
Horizontal Data Set (Columns): 4
Only Sam Data
  Indicator Height Weight Name
0         4     1.0     35 Sam
1         4     1.0     40 Sam
2         4     1.0     45 Sam
3         4     1.0     50 Sam
4         4     1.0     55 Sam
#####

```

Association Rule Mining

Association rule learning is a rule-based machine-learning method for discovering interesting relations between variables in large databases, similar to the data you will find in a data lake. The technique enables you to investigate the interaction between data within the same population. Lift is simply estimated by the ratio of the joint probability of two items x and y , divided by the product of their individual probabilities:

C:\VKHCG\01-Vermeulen\05-Organise\ Organize-Association-Rule.py

```
# -*- coding: utf-8 -*-
```

```
import sys
```

```
import os
```

```
import pandas as pd
```

```
from mlxtend.frequent_patterns import apriori
```

```
from mlxtend.frequent_patterns import association_rules
```

```
Base='C:/VKHCG'
```

```
print('#####')
```

```
print('Working Base :',Base, ' using ', sys.platform)
```

```
print('#####')
```

```
Company='01-Vermeulen'
```

```
InputFileName='Online-Retail-Billboard.xlsx'
```

```
EDSAssessDir='02-Assess/01-EDS'
```

```
InputAssessDir=EDSAssessDir + '/02-Python'
```

```
sFileAssessDir=Base + '/' + Company + '/' + InputAssessDir
```

```
if not os.path.exists(sFileAssessDir):
```

```
os.makedirs(sFileAssessDir)
```

```
sFileName=Base+'/' + Company + '/00-RawData/' + InputFileName
```

```
df = pd.read_excel(sFileName)
```

```
print(df.shape)
```

```
df['Description'] = df['Description'].str.strip()
```

```
df.dropna(axis=0, subset=['InvoiceNo'], inplace=True)
df['InvoiceNo'] = df['InvoiceNo'].astype('str')
df = df[~df['InvoiceNo'].str.contains('C')]
basket = (df[df['Country'] == "France"]
.groupby(['InvoiceNo', 'Description'])['Quantity']
.sum().unstack().reset_index().fillna(0)
.set_index('InvoiceNo'))
def encode_units(x):
    if x <= 0:
        return 0
    if x >= 1:
        return 1
basket_sets = basket.applymap(encode_units)
basket_sets.drop('POSTAGE', inplace=True, axis=1)
frequent_itemsets = apriori(basket_sets, min_support=0.07, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)
print(rules.head())
rules[ (rules['lift'] >= 6) &
(rules['confidence'] >= 0.8) ]
sProduct1='ALARM CLOCK BAKELIKE GREEN'
print(sProduct1)
print(basket[sProduct1].sum())
sProduct2='ALARM CLOCK BAKELIKE RED'
print(sProduct2)
print(basket[sProduct2].sum())
basket2 = (df[df['Country'] == "Germany"]
.groupby(['InvoiceNo', 'Description'])['Quantity']
.sum().unstack().reset_index().fillna(0)
.set_index('InvoiceNo'))
basket_sets2 = basket2.applymap(encode_units)
basket_sets2.drop('POSTAGE', inplace=True, axis=1)
frequent_itemsets2 = apriori(basket_sets2, min_support=0.05, use_colnames=True)
rules2 = association_rules(frequent_itemsets2, metric="lift", min_threshold=1)
print(rules2[ (rules2['lift'] >= 4) &
(rules2['confidence'] >= 0.5)])
print('### Done!! #####')
```

OUTPUT:

```

Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 19:29:22) [MSC v.1916 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
== RESTART: C:\VKHCG\01-Vermeulen\05-Organise\Organize-Association-Rule.py ==
#####
Working Base : C:/VKHCG using win32
#####
(541909, 8)
      antecedents ... conviction
0  (ALARM CLOCK BAKELIKE PINK) ... 3.283859
1  (ALARM CLOCK BAKELIKE GREEN) ... 3.791383
2  (ALARM CLOCK BAKELIKE GREEN) ... 4.916181
3  (ALARM CLOCK BAKELIKE RED) ... 5.568878
4  (ALARM CLOCK BAKELIKE PINK) ... 3.293135

[5 rows x 9 columns]
ALARM CLOCK BAKELIKE GREEN
340.0
ALARM CLOCK BAKELIKE RED
316.0

      antecedents ... conviction
0  (PLASTERS IN TIN CIRCUS PARADE) ... 2.076984
7  (PLASTERS IN TIN SPACEBOY) ... 2.011670
11 (RED RETROSPOT CHARLOTTE BAG) ... 5.587746

[3 rows x 9 columns]
### Done!! #####
>>> |
Ln: 28 Col: 4

```

Create a Network Routing Diagram

I will guide you through a possible solution for the requirement, by constructing an island -style Organize superstep that uses a graph data model to reduce the records and the columns on the data set.

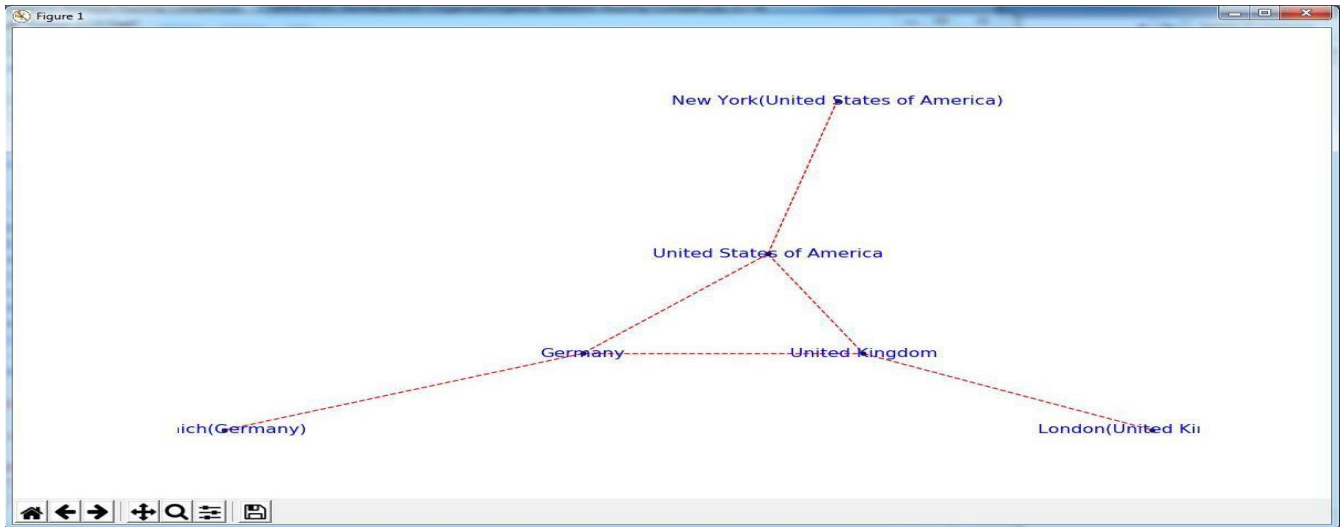
C:\VKHCG\01-Vermeulen\05-Organise\ Organise-Network-Routing-Company.py

```

import sys
import os
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
pd.options.mode.chained_assignment = None
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sInputFileName='02-Assess/01-EDS/02-Python/Assess-Network-Routing-Company.csv'
sOutputFileName1='05-Organise/01-EDS/02-Python/Organise-Network-Routing-Company.gml'
sOutputFileName2='05-Organise/01-EDS/02-Python/Organise-Network-Routing-Company.png'
Company='01-Vermeulen'
### Import Country Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
CompanyData=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('#####')
print(CompanyData.head())
print(CompanyData.shape)

```

```
G=nx.Graph()
for i in range(CompanyData.shape[0]):
for j in range(CompanyData.shape[0]):
Node0=CompanyData['Company_Country_Name'][i]
Node1=CompanyData['Company_Country_Name'][j]
if Node0 != Node1:
G.add_edge(Node0,Node1)
for i in range(CompanyData.shape[0]):
Node0=CompanyData['Company_Country_Name'][i]
Node1=CompanyData['Company_Place_Name'][i] + '('+ CompanyData['Company_Country_Name'][i] +
')'
if Node0 != Node1:
G.add_edge(Node0,Node1)
print('Nodes:', G.number_of_nodes())
print('Edges:', G.number_of_edges())
sFileName=Base + '/' + Company + '/' + sOutputFileName1
print('#####')
print('Storing :',sFileName)
print('#####')
nx.write_gml(G, sFileName)
sFileName=Base + '/' + Company + '/' + sOutputFileName2
print('#####')
print('Storing Graph Image:',sFileName)
print('#####')
plt.figure(figsize=(15, 15))
pos=nx.spectral_layout(G,dim=2)
nx.draw_networkx_nodes(G,pos, node_color='k', node_size=10, alpha=0.8)
nx.draw_networkx_edges(G, pos,edge_color='r', arrows=False, style='dashed')
nx.draw_networkx_labels(G,pos,font_size=12,font_family='sans-serif',font_color='b')
plt.axis('off')
plt.savefig(sFileName,dpi=600)
plt.show()
print('#####')
print('### Done!! #####')
print('#####')
```



Picking Content for Billboards

To enable the marketing salespeople to sell billboard content, they will require a diagram to show what billboards connect to which office content publisher. Each of Krennwallner's billboards has a proximity sensor that enables the content managers to record when a registered visitor points his/her smartphone at the billboard content or touches the near-field pad with a mobile phone.

Program will assist you in building an organized graph of the billboards' locations data to help you to gain insights into the billboard locations and content picking process.

C:\VKHCG\02-Krennwallner\05-Organise\ Organise-billboards.py

Code:

```

import sys
import os
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
pd.options.mode.chained_assignment = None
Base='C:/VKHCG'
print('#####')
print('Working Base :',Base, ' using ', sys.platform)
print('#####')
sInputFileName='02-Assess/01-EDS/02-Python/Assess-DE-Billboard-Visitor.csv'
sOutputFileName1='05-Organise/01-EDS/02-Python/Organise-Billboards.gml'
sOutputFileName2='05-Organise/01-EDS/02-Python/Organise-Billboards.png'
Company='02-Krennwallner'
### Import Company Data
sFileName=Base + '/' + Company + '/' + sInputFileName
print('#####')
print('Loading :',sFileName)
print('#####')
BillboardDataRow=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
print('#####')
print(BillboardDataRow.head())

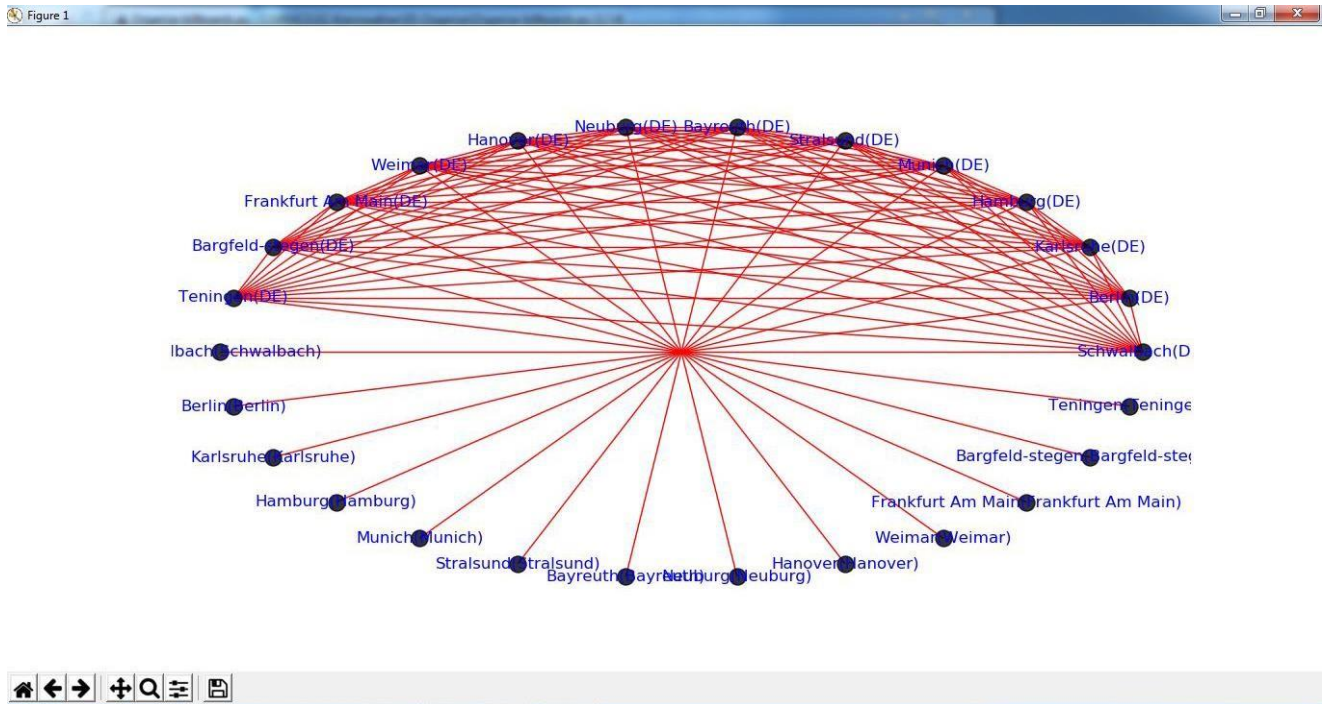
```

```

print(BillboardDataRow.shape)
BillboardData=BillboardDataRow
sSample=list(np.random.choice(BillboardData.shape[0],20))
G=nx.Graph()
for i in sSample:
for j in sSample:
Node0=BillboardData['BillboardPlaceName'][i] + '('+ BillboardData['BillboardCountry'][i] + ')'
Node1=BillboardData['BillboardPlaceName'][j] + '('+ BillboardData['BillboardCountry'][i] + ')'
if Node0 != Node1:
G.add_edge(Node0,Node1)
for i in sSample:
Node0=BillboardData['BillboardPlaceName'][i] + '('+ BillboardData['VisitorPlaceName'][i] + ')'
Node1=BillboardData['BillboardPlaceName'][i] + '('+ BillboardData['VisitorCountry'][i] + ')'
if Node0 != Node1:
G.add_edge(Node0,Node1)
print('Nodes:', G.number_of_nodes())
print('Edges:', G.number_of_edges())
sFileName=Base + '/02-Krennwallner/' + sOutputFileName1
print('#####')
print('Storing :',sFileName)
print('#####')
nx.write_gml(G, sFileName)
sFileName=Base + '/02-Krennwallner/' + sOutputFileName2
print('#####')
print('Storing Graph Image:',sFileName)
print('#####')
plt.figure(figsize=(15, 15))
pos=nx.circular_layout(G,dim=2)
nx.draw_networkx_nodes(G,pos, node_color='k', node_size=150, alpha=0.8)
nx.draw_networkx_edges(G, pos,edge_color='r', arrows=False, style='solid')
nx.draw_networkx_labels(G,pos,font_size=12,font_family='sans-serif',font_color='b')
plt.axis('off')
plt.savefig(sFileName,dpi=600)
plt.show()
print('#####')
print('### Done!! #####')
print('#####')

```

OUTPUT :



Create a Delivery Route

Hillman requires a new delivery route plan from HQ-KA13's delivery region. The managing director has to know the following:

- What his most expensive route is, if the cost is £1.50 per mile and two trips are planned per day
- What the average travel distance in miles is for the region per 30-day month

With your newfound knowledge in building the technology stack for turning data lakes into business assets, can you convert the graph stored in the Assess step called

"Assess_Best_Logistics" into the shortest path between the two points?

C:\VKHCG\03-Hillman\05-Organise\Organise-Routes.py

-*- coding: utf-8 -*-

import sys

import os

import pandas as pd

Base='C:/VKHCG'

print('#####')

print('Working Base :',Base, ' using ', sys.platform)

print('#####')

sInputFileName='02-Assess/01-EDS/02-Python/Assess_Shipping_Routes.txt'

sOutputFileName='05-Organise/01-EDS/02-Python/Organise-Routes.csv'

Company='03-Hillman'

Import Routes Data

sFileName=Base + '/' + Company + '/' + sInputFileName

print('#####')

print('Loading :',sFileName)

print('#####')

RouteDataRaw=pd.read_csv(sFileName,header=0,low_memory=False, sep='|', encoding="latin-1")

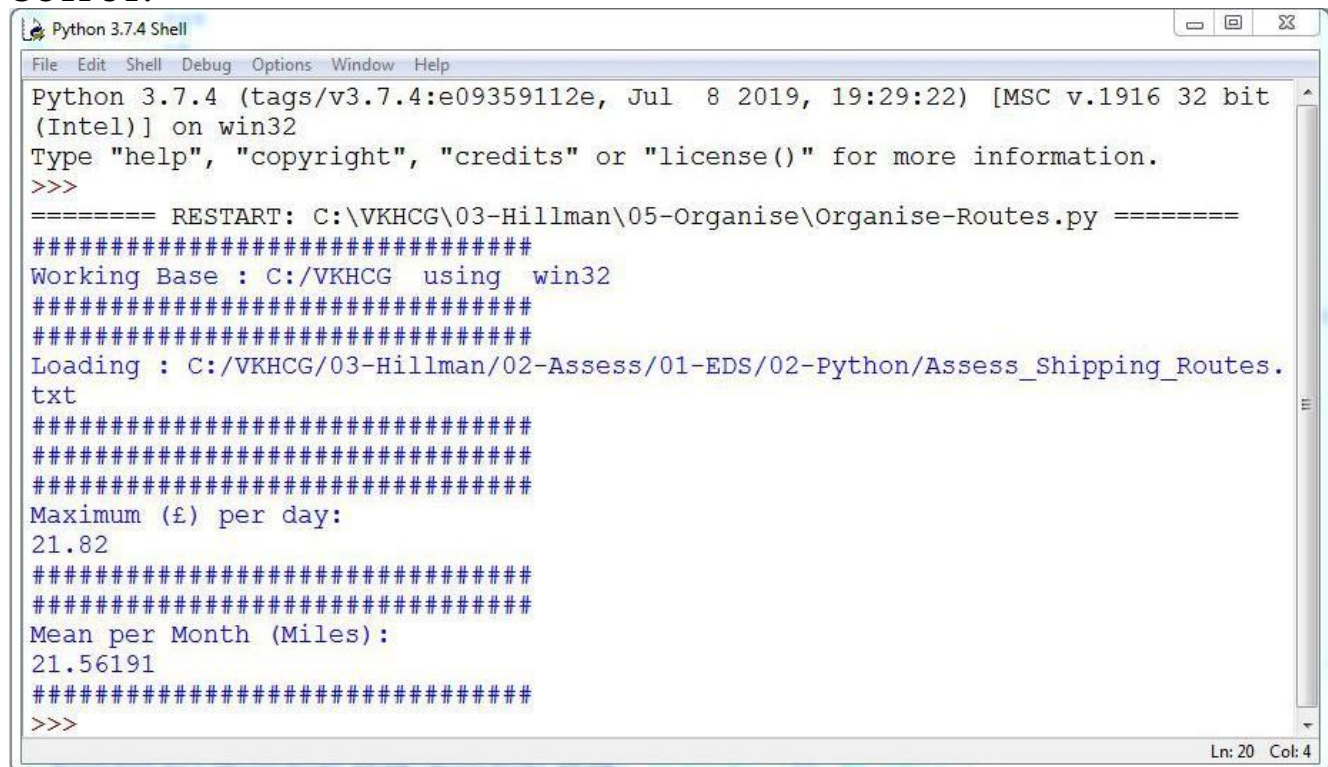
print('#####')

```

RouteStart=RouteDataRow[RouteDataRow['StartAt']=='WH-KA13']
RouteDistance=RouteStart[RouteStart['Cost']=='DistanceMiles']
RouteDistance=RouteDistance.sort_values(by=['Measure'], ascending=False)
RouteMax=RouteStart["Measure"].max()
RouteMaxCost=round((((RouteMax/1000)*1.5*2)),2)
print('#####')
print('Maximum (£) per day:')
print(RouteMaxCost)
print('#####')
RouteMean=RouteStart["Measure"].mean()
RouteMeanMonth=round((((RouteMean/1000)*2*30)),6)
print('#####')
print('Mean per Month (Miles):')
print(RouteMeanMonth)
print('#####')

```

OUTPUT:



```

Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 19:29:22) [MSC v.1916 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\VKHCG\03-Hillman\05-Organise\Organise-Routes.py =====
#####
Working Base : C:/VKHCG using win32
#####
Loading : C:/VKHCG/03-Hillman/02-Assess/01-EDS/02-Python/Assess_Shipping_Routes.
txt
#####
#####
Maximum (£) per day:
21.82
#####
Mean per Month (Miles):
21.56191
#####
>>>
Ln: 20 Col: 4

```

Clark Ltd

Our financial services company has been tasked to investigate the options to convert 1 million pounds sterling into extra income. Mr. Clark Junior suggests using the simple variance in the daily rate between the British pound sterling and the US dollar, to generate extra income from trading. Does your chief financial officer wants to know if this is feasible?

Simple Forex Trading Planner

Your challenge is to take 1 million US dollars or just over six hundred thousand pounds sterling and, by simply converting it between pounds sterling and US dollars, achieve a profit. Are you up to this

challenge?

The Program will help you how to model this problem and achieve a positive outcome. The forex data has been collected on a daily basis by Clark's accounting department, from previous overseas transactions.

C:\VKHCG\04-Clark\05-Organise\Organise-Forex.py

```
# -*- coding: utf-8 -*-
```

```
import sys
```

```
import os
```

```
import pandas as pd
```

```
import sqlite3 as sq
```

```
import re
```

```
Base='C:/VKHCG'
```

```
print('#####')
```

```
print('Working Base :',Base, ' using ', sys.platform)
```

```
print('#####')
```

```
sInputFileName='03-Process/01-EDS/02-Python/Process_ExchangeRates.csv'
```

```
sOutputFileName='05-Organise/01-EDS/02-Python/Organise-Forex.csv'
```

```
Company='04-Clark'
```

```
sDatabaseName=Base + '/' + Company + '/05-Organise/SQLite/clark.db'
```

```
conn = sq.connect(sDatabaseName)
```

```
#conn = sq.connect(':memory:')
```

```
### Import Forex Data
```

```
sFileName=Base + '/' + Company + '/' + sInputFileName
```

```
print('#####')
```

```
print('Loading :',sFileName)
```

```
print('#####')
```

```
ForexDataRaw=pd.read_csv(sFileName,header=0,low_memory=False, encoding="latin-1")
```

```
print('#####')
```

```
ForexDataRaw.index.names = ['RowID']
```

```
sTable='Forex_All'
```

```
print('Storing :',sDatabaseName,' Table:',sTable)
```

```
ForexDataRaw.to_sql(sTable, conn, if_exists="replace")
```

```
sSQL="SELECT 1 as Bag\, CAST(min(Date) AS VARCHAR(10)) as Date \,CAST(1000000.0000000 as NUMERIC(12,4)) as Money \,'USD' as Currency \FROM Forex_All \;"
```

```
sSQL=re.sub("\s+", " ", sSQL)
```

```
nMoney=pd.read_sql_query(sSQL, conn)
```

```
nMoney.index.names = ['RowID']
```

```
sTable='MoneyData'
```

```
print('Storing :',sDatabaseName,' Table:',sTable)
```

```
nMoney.to_sql(sTable, conn, if_exists="replace")
```

```
sTable='TransactionData'
```

```
print('Storing :',sDatabaseName,' Table:',sTable)
```

```
nMoney.to_sql(sTable, conn, if_exists="replace")
```

```
ForexDay=pd.read_sql_query("SELECT Date FROM Forex_All GROUP BY Date;", conn)
```

```
t=0
```

```

for i in range(ForexDay.shape[0]):
    sDay1=ForexDay['Date'][i]
    sDay=str(sDay1)
    sSQL='\SELECT M.Bag as Bag, \F.Date as Date, \round(M.Money * F.Rate,6) AS Money, \
    F.CodeIn AS PCurrency, \F.CodeOut AS Currency \FROM MoneyData AS M \JOIN \(\SELECT \
    CodeIn, CodeOut, Date, Rate \FROM \Forex_All \WHERE\CodeIn = "USD" AND CodeOut = "GBP" \
    UNION \SELECT \CodeOut AS CodeIn, CodeIn AS CodeOut, Date, (1/Rate) AS Rate \FROM \
    Forex_All \WHERE\CodeIn = "USD" AND CodeOut = "GBP" \) AS F ON \M.Currency=F.CodeIn \
    AND \F.Date = "' + sDay + "';'sSQL=re.sub("\s\s+", " ", sSQL)
    ForexDayRate=pd.read_sql_query(sSQL, conn)
    for j in range(ForexDayRate.shape[0]):
        sBag=str(ForexDayRate['Bag'][j])
        nMoney=str(round(ForexDayRate['Money'][j],2))
        sCodeIn=ForexDayRate['PCurrency'][j]
        sCodeOut=ForexDayRate['Currency'][j]
        sSQL='UPDATE MoneyData SET Date= "' + sDay + "', '
        sSQL= sSQL + ' Money = ' + nMoney + ', Currency="' + sCodeOut + "'"
        sSQL= sSQL + ' WHERE Bag=' + sBag + ' AND Currency="' + sCodeIn + "';"
        sSQL=re.sub("\s\s+", " ", sSQL)
        cur = conn.cursor()
        cur.execute(sSQL)
        conn.commit()
        t+=1
    print("Trade :", t, sDay, sCodeOut, nMoney)
    sSQL=' \INSERT INTO TransactionData ( \RowID, \Bag, \Date, \Money, \Currency \) \
    SELECT ' + str(t) + ' AS RowID, \Bag, \Date, \Money, \Currency \
    FROM MoneyData \;'
    sSQL=re.sub("\s\s+", " ", sSQL)
    cur = conn.cursor()
    cur.execute(sSQL)
    conn.commit()
    sSQL="SELECT RowID, Bag, Date, Money, Currency FROM TransactionData ORDER BY
    RowID;"
    sSQL=re.sub("\s\s+", " ", sSQL)
    TransactionData=pd.read_sql_query(sSQL, conn)
    OutputFile=Base + '/' + Company + '/' + sOutputFileName
    TransactionData.to_csv(OutputFile, index = False)

```

OUTPUT:

Save the Assess-Forex.py file, then compile and execute with your Python compiler.
This will produce a set of demonstrated values onscreen.

CLOUD COMPUTING



NURTURING POTENTIAL

SAKET GYANPEETH'S

SAKET COLLEGE OF ARTS, SCIENCE AND COMMERCE

(Permanently Affiliated to University of Mumbai)

NAAC Accredited B Grade

Saket Vidyanagari Marg, Chinchpada Road, Katemanivali, Kalyan (East) – 421306, Dist.

Department of Information Technology

Thane (MAH)

CERTIFICATE

This is to certify that

ANSHUMAN GAURISHANKAR SINGH of MSc Information Technology Part-I

Class has satisfactory carried out the required practical in the subject.

CLOUD COMPUTING

For the Academic year 2022 – 2023

Practical In-Charge

Head of Department

External Examiner

INDEX

Sr.No.	Aim	Sign
1	VMware Tutorial: Install ESXi Hypervisor	
2	Write a program for implementing Client Server communication model using TCP	
3	Write a program for implementing Client Server communication model using UDP	
4	A multicast Socket example	
5	Write a program to show the Date using RMI	
6	Write a program to convert digit using RMI	
7	Implement Virtualization using VMware ESXi Server and managing with vCenter	
8	Develop application for Google App Engine	

PRACTICAL 1

1. VMware Tutorial: Install ESXi Hypervisor

A hypervisor is software that allows a server to run multiple operating systems. It allows you to create virtual machines (VM), which enables users to customize hardware resources and perform actions such as taking a VM snapshot and migrating virtual machines to other hosts without downtime.

VMware ESXi is one of the most commonly used type-1 hypervisors in a data center environment. It runs directly using the host's hardware, rather than acting as a standard Windows or Mac application.

Do you know that you can install ESXi hypervisors at home for experimental purposes? This article introduces the steps to install VMware ESXi hypervisor on a virtual machine created in VMware Workstation.

Step 1: Download VMware ESXi Hypervisor and VMware Workstation

Head over to VMware's official website to download an evaluation version (ISO version) of VMware vSphere. Note that you will need to register a free account to download the ISO.

<https://www.vmware.com/try-vmware.html>

Data Center and Cloud Infrastructure		
Product	Public Evaluation	Hands-on Lab - Intro
 VMware vSphere	Download Free Trial >>	Try for Free >>
 VMware vSAN	Download Free Trial >>	Try for Free >>
 VMware NSX-T		Try for Free >>

Download VMware vSphere ESXi

For ease of installation, you can install VMware Workstation (a type-2 hypervisor, also known as VMware Fusion on macOS platform) on your operating system to create a virtual machine, and install ESXi hypervisor (a type-1 hypervisor) on the VM.

<https://www.vmware.com/products/workstation-pro.html>

Tips #1: If your computer does not have enough processing power, or if you prefer to skip the installation step, I would recommend you to try out VMware Hands-on Lab. It provides an interactive interface for you to use ESXi software on a web browser – without the need to install any software. You can register a free account by visiting the below website.

<https://hol.vmware.com/>

Tips #2: If you would like to use the evaluation software for an extended period, consider joining VMware User Group (VMUG) and upgrading to Advantage membership. You can get 365 -day evaluation licenses for most VMware products including ESXi, vCenter, NSX, etc. for an annual membership fee.

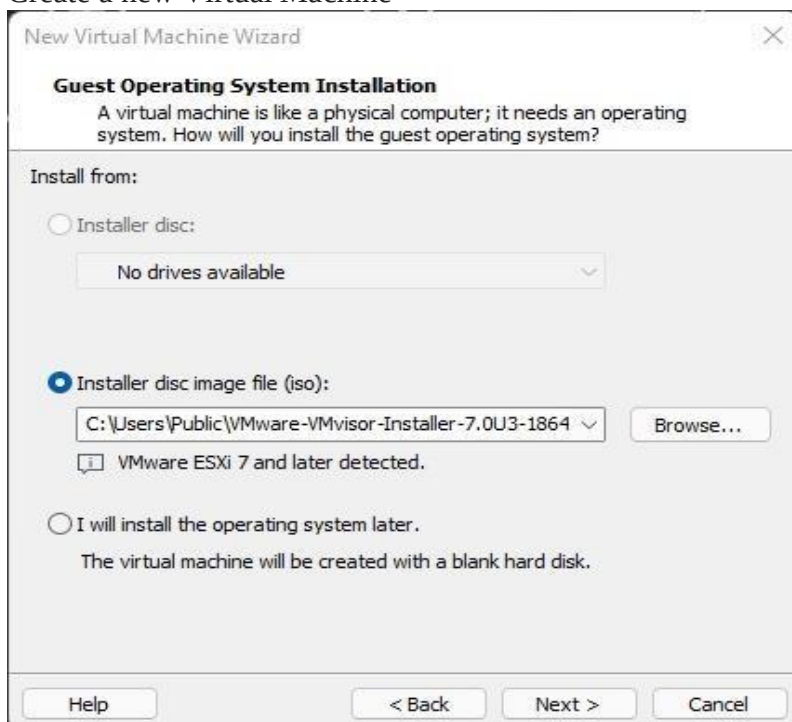
<https://www.vmug.com/>

Step 2: Create a virtual machine for ESXi installation on VMware Workstation

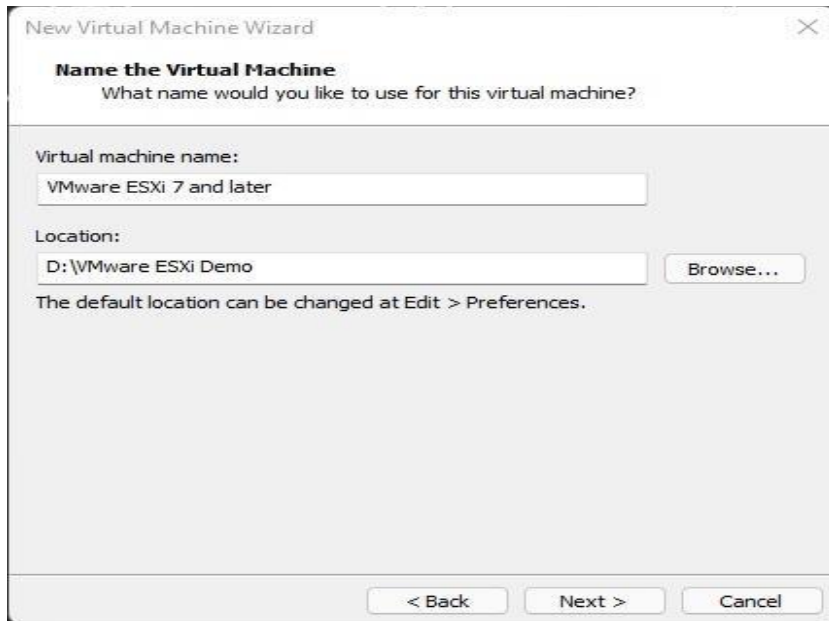
Start by creating a VM using the “New Virtual Machine Wizard” on VMware Workstation.



Create a new Virtual Machine

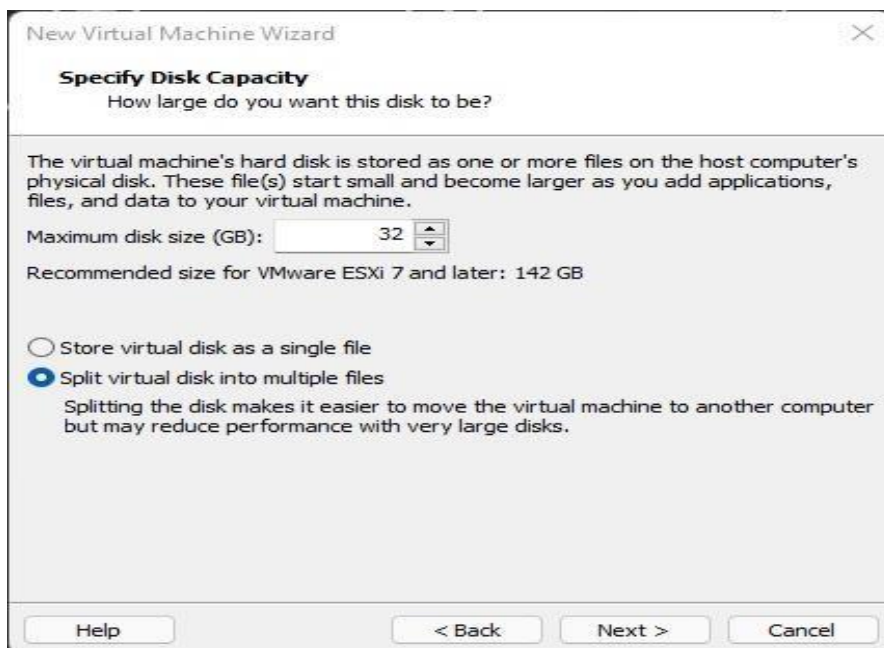


Select the ESXi ISO



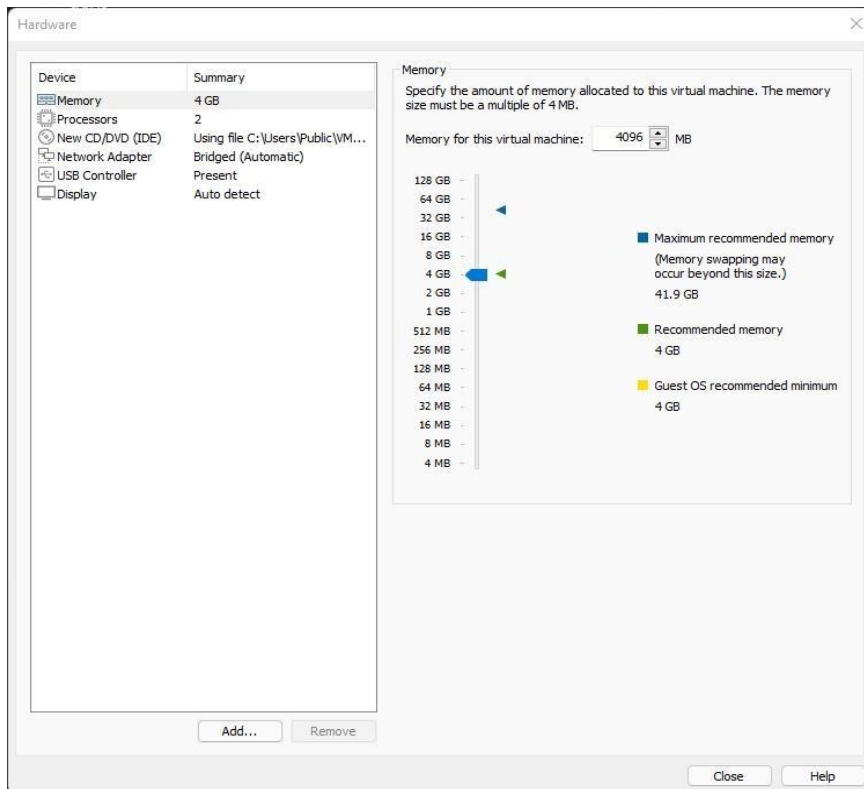
Define VM Name and Location

Refer to the [official documentation](#) for the minimum hardware requirements for ESXi 7.0 (eg. 4GB ram, 2 CPU cores, and 32GB disk space)



Specify Disk Capacity

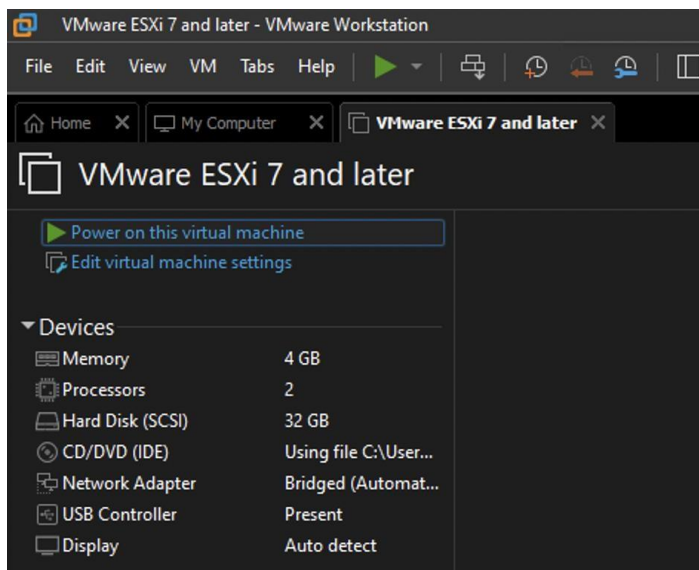
Note that you can select “Bridged” mode for the Network Adapter setting if you wish to access the [ESXi host client](#) from devices other than your host PC after installation.



Customize VM Hardware

Step 3: Power on Virtual Machine and proceed with installation

After creating the virtual machine, power on the VM to continue the ESXi installation.



Power on Virtual Machine

Seat No.: 3254185



ESXi Installer #1



ESXi Installer #2



ESXi Installer Start Wizard

After starting the installation, you will be asked to select a disk location to install ESXi. As the VM is configured with a single local 32GB hard disk (which is the minimum amount of disk space for ESXi 7.0 installation), it is the only storage device displayed on the ESXi installation wizard. Continue by selecting the disk.



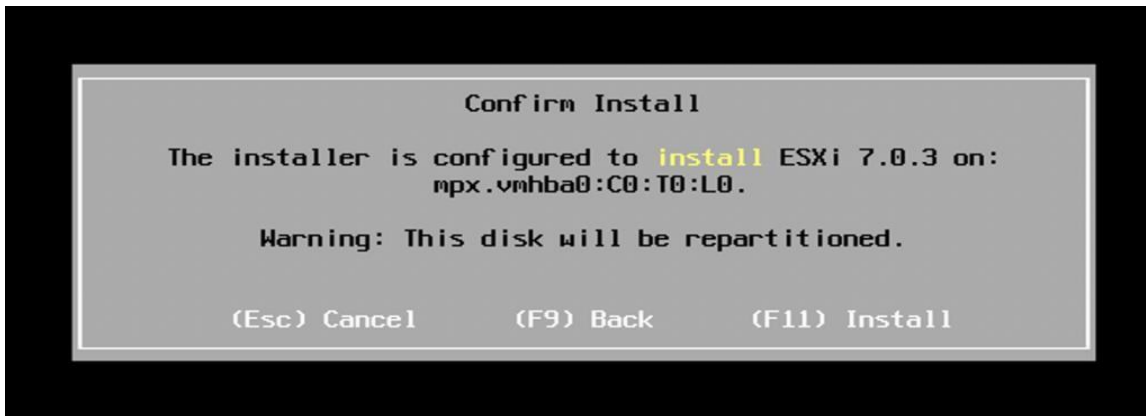
ESXi Installer Select Disk

Upon selecting the storage device for the installation, you will be asked to enter an ESXi root password, which you can later use to log in to the ESXi host client, as well as the [Direct Console User Interface \(DCUI\)](#).



ESXi Installer Enter Root Password

Press F11 to confirm installing ESXi on the selected disk location:

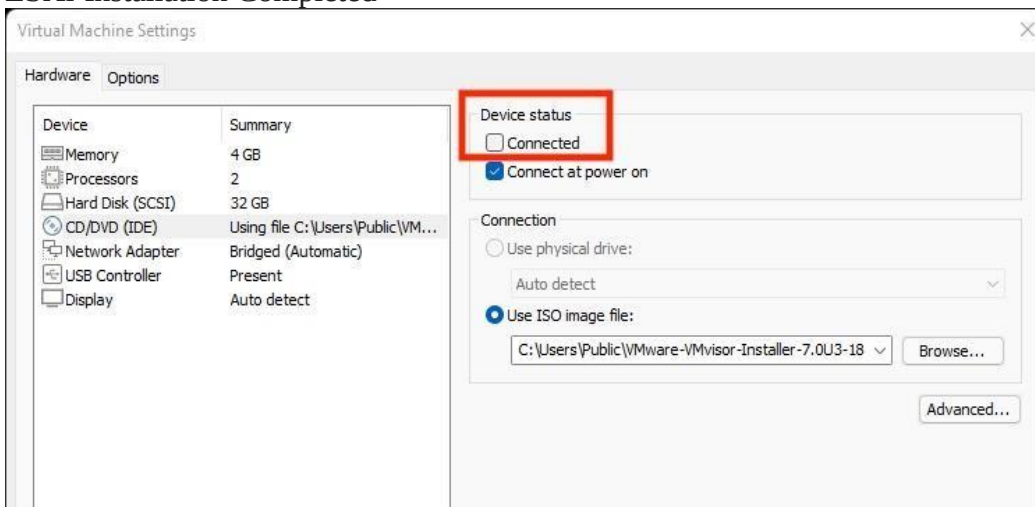


ESXi Installer Confirm Install

After installation has been completed, select “configure hardware” on VMware Workstation, remove the ESXi installation media (e.g. CD Rom, USB drive), and press “Enter” to reboot into .



ESXi Installation Completed

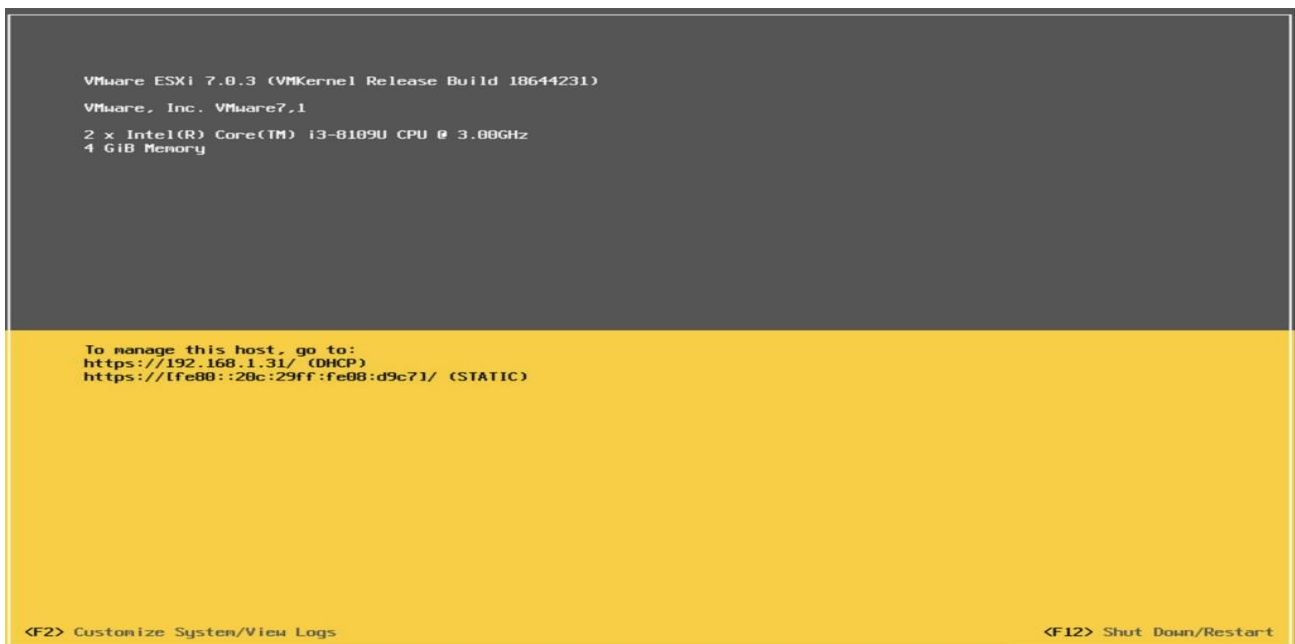


Disconnect CD/DVD from the Virtual Machine

Post-installation: Access Direct Console User Interface (DCUI) and ESXi Host Client

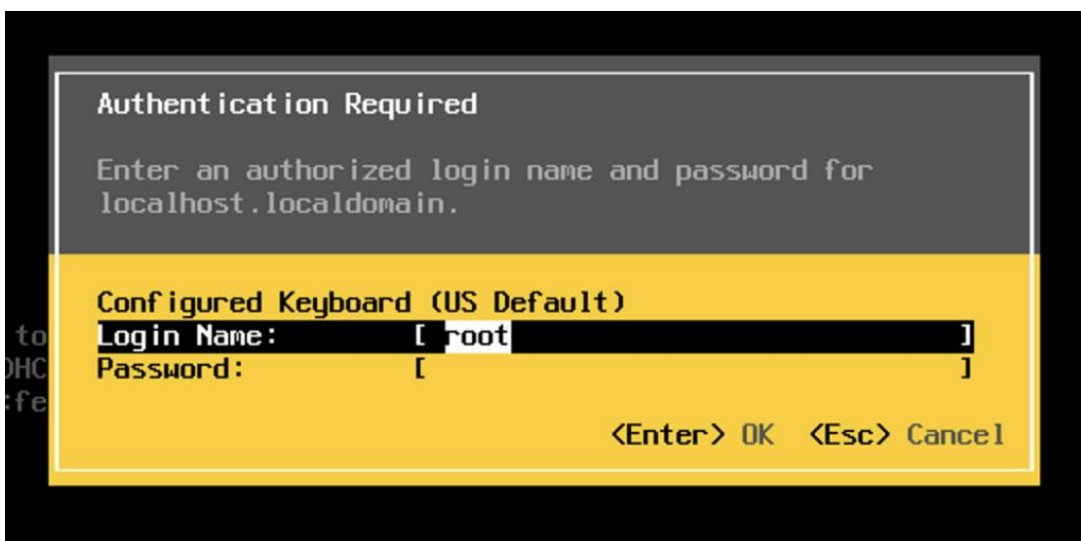
#1) Access Direct Console User Interface (DCUI) on the Connected Display Device

After installing ESXi and rebooting, you can view the Direct Console User Interface (DCUI). Basic information including ESXi version, CPU, memory, and management IP address will be shown. Note that DCUI offers limited configuration as it is not designed for daily operation. For most use cases, system admins only access DCUI via Integrated Management Module (IMM) to perform tasks such as changing ESXi management IP and checking system status when the network connection to the ESXi is lost. For performing most operational and management tasks, you will need to use the ESXi host client on a web browser.



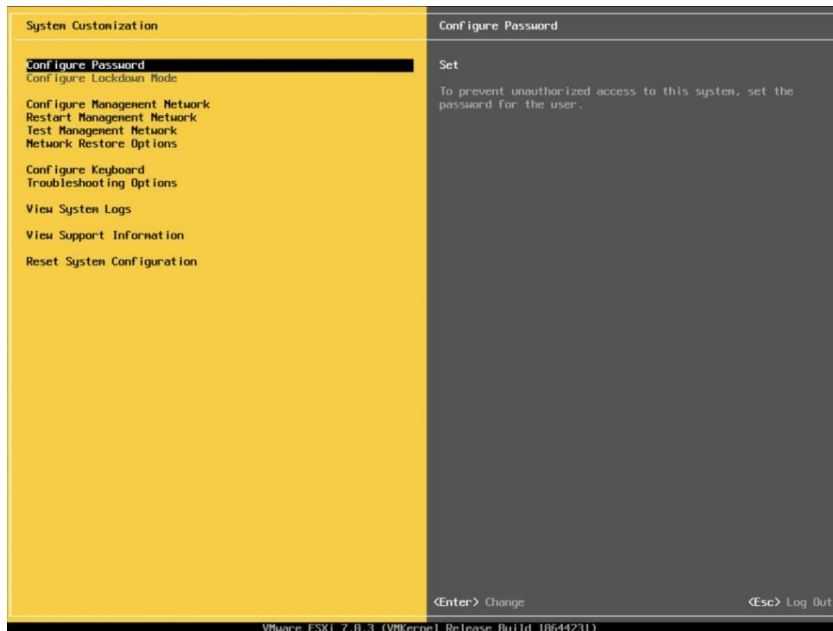
VMware ESXi Direct Console User Interface (DCUI)

For customization such as configuring the management IP address, press F2 and enter the root user credential you entered during the ESXi installation.



VMware ESXi DCUI Login

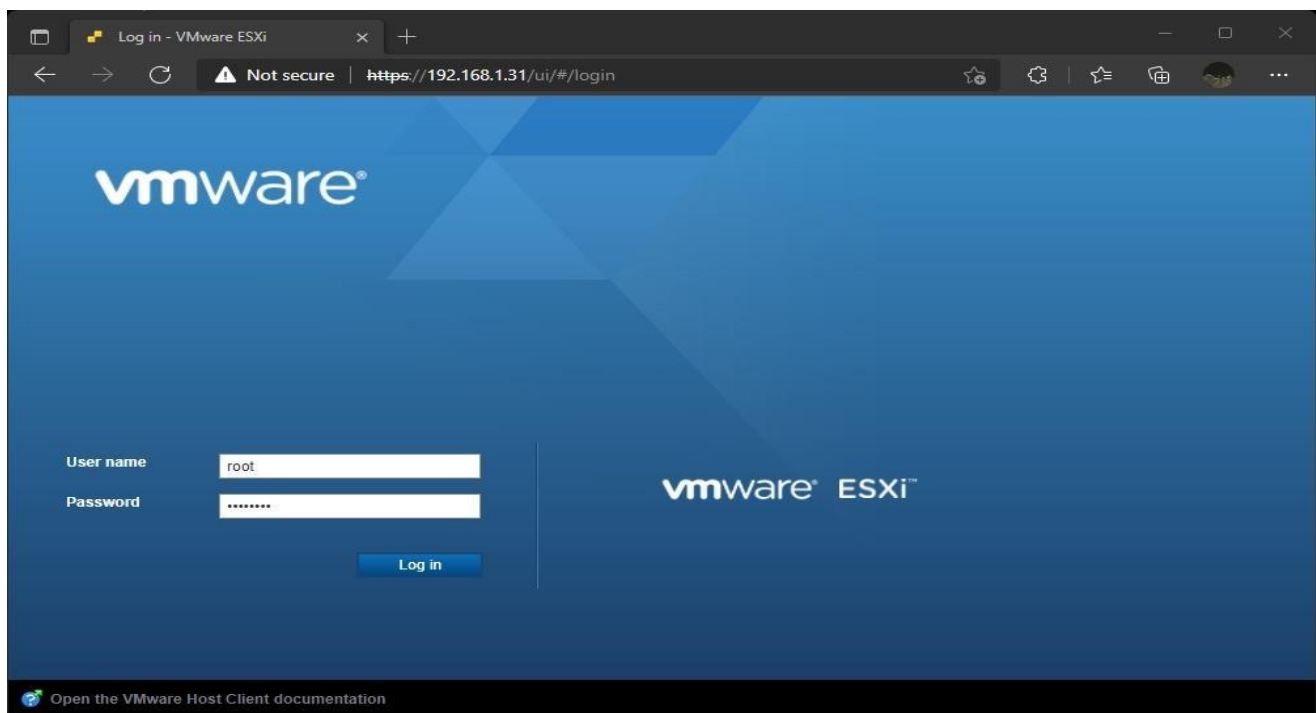
After logging in as a root user, you can perform various configurations on the DCUI including changing the ESXi root password and the management network (e.g. assign a static management IP).



VMware ESXi DCUI Settings

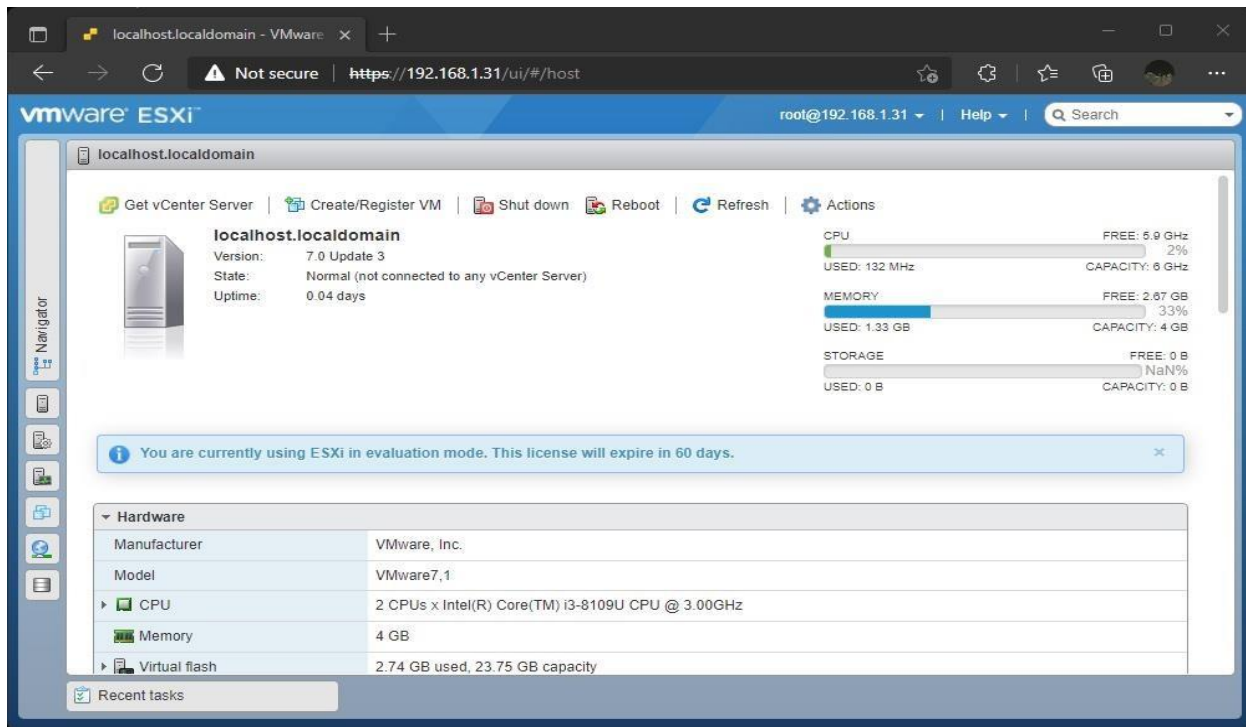
[#2\) Access ESXi Host Client via a Remote Web Browser](#)

You can also access the ESXi management console by entering the management IP of the ESXi on a web browser. This is the preferred way to perform management tasks including datastore and VM configuration etc.



ESXi Host Client Login Page

After login, you can view the system information and perform management tasks.



ESXi Host Client UI

Conclusion

ESXi hypervisor provides a virtualized computing platform where multiple operating systems can be run on a single host. It offers many more capabilities including the ability to provision virtual machines with customized hardware and the option to create a snapshot for the temporary preservation of VM state and data. Installing the ESXi hypervisor is the first step to creating a software-defined data center (SDDC), where computing resources can be pooled to provide a centralized platform for hosting enterprise applications efficiently. Stay tuned for more tutorials on cloud computing and system administration topics.

PRACTICAL 2

2. Write a program for implementing Client Server communication model using TCP.

Aim: Write a program for implementing Client Server communication model using TCP.

Practical 2A: A client server based program using TCP to find if the number entered is prime.

Code:

1. tcpServerPrime.java

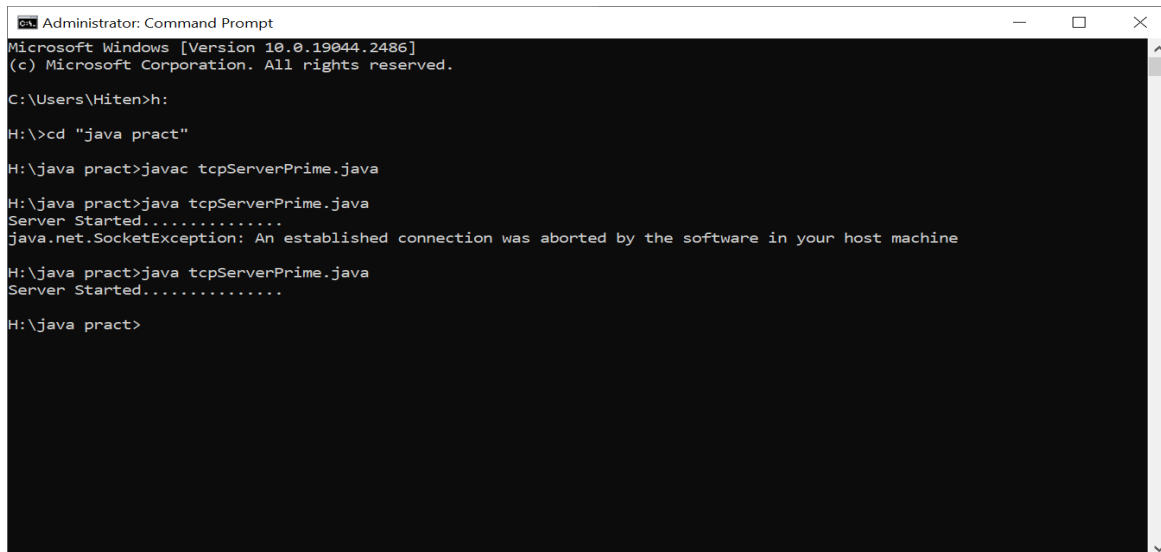
```
import java.net.*;
import java.io.*;
class tcpServerPrime
{
    public static void main(String args[])
    {
        try
        {
            ServerSocket ss = new ServerSocket(8001);
            System.out.println("Server Started .....");
            Socket s = ss.accept();
            DataInputStream in = new
            DataInputStream(s.getInputStream()); int x= in.readInt();
            DataOutputStream otc = new
            DataOutputStream(s.getOutputStream()); int y = x/2;
            if(x==1 || x==2 || x==3)
            {
                otc.writeUTF(x + "is Prime");
                System.exit(0);
            }
            for(int i=2; i<=y; i++)
            {
                if(x%i != 0)
                {
                    otc.writeUTF(x + " is Prime");
                }
                else
                {
                    otc.writeUTF(x + " is not Prime");
                }
            }
            catch(Exception e)
            {
                System.out.println(e.toString());
            }
        }
    }
}
```

```
}
```

2. tcpClientPrime.java

```
import java.net.*;
import java.io.*;
class tcpClientPrime
{
    public static void main(String args[])
    {
        try
        {
            Socket cs = new Socket("LocalHost",8001);
            BufferedReader infu = new BufferedReader(new
            InputStreamReader(System.in));
            System.out.println("Enter a number : ");
            int a = Integer.parseInt(infu.readLine());
            DataOutputStream out = new
            DataOutputStream(cs.getOutputStream());
            out.writeInt(a);
            DataInputStream in = new
            DataInputStream(cs.getInputStream());
            System.out.println(in.readUTF()); cs.close();
        }
        catch(Exception e)
        {
            System.out.println(e.toString());
        }
    }
}
```

OUTPUT:



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19044.2486]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Hiten>h:

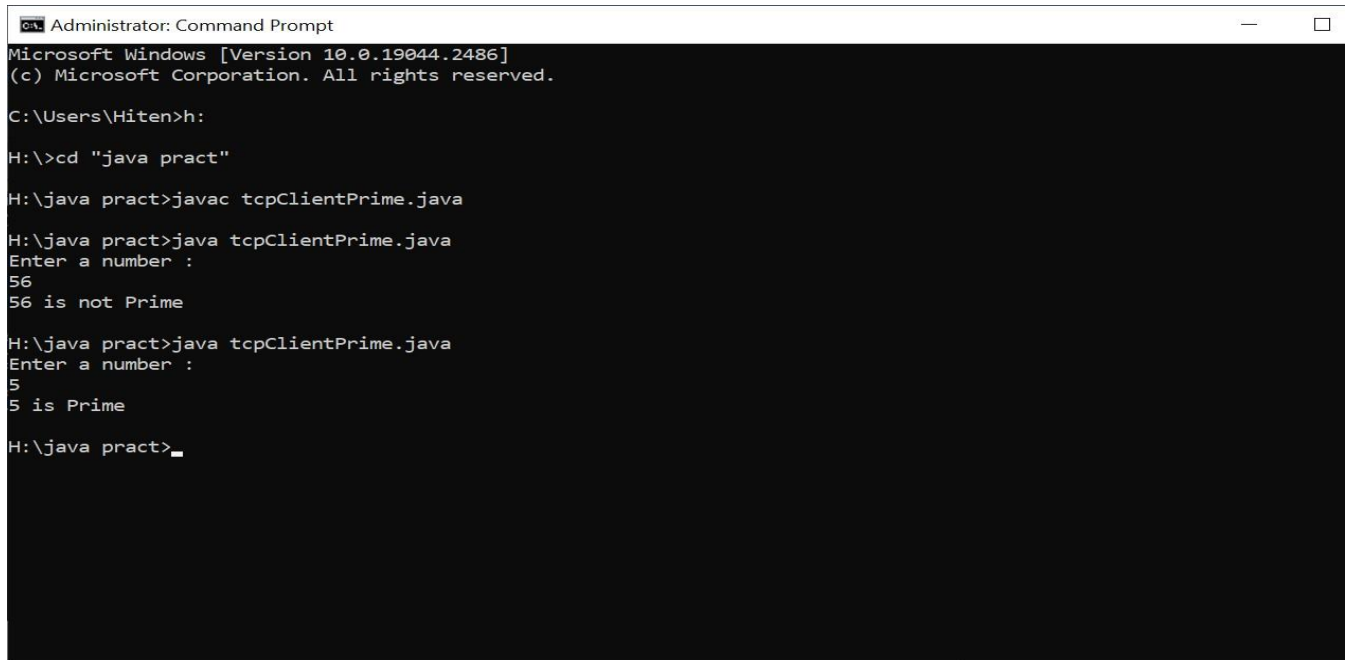
H:\>cd "java pract"

H:\java pract>javac tcpServerPrime.java

H:\java pract>java tcpServerPrime.java
Server Started.....
java.net.SocketException: An established connection was aborted by the software in your host machine

H:\java pract>java tcpServerPrime.java
Server Started.....

H:\java pract>
```



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19044.2486]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Hiten>h:
H:\>cd "java pract"
H:\java pract>javac tcpClientPrime.java
H:\java pract>java tcpClientPrime.java
Enter a number :
56
56 is not Prime

H:\java pract>java tcpClientPrime.java
Enter a number :
5
5 is Prime

H:\java pract>_
```

Practical 2B: A client server TCP based chatting application.

Code:

1. ChatServer.java:

```
import java.net.*;
import java.io.*;
class ChatServer
{
    public static void main(String args[])
    {
        try
        {
            ServerSocket ss = new ServerSocket(8000);
            System.out.println("Waiting for client to
            connect.."); Socket s = ss.accept();
            BufferedReader br = new
            BufferedReader(new InputStreamReader(System.in));
            DataOutputStream out = new
            DataOutputStream(s.getOutputStream()); DataInputStream in = new
            DataInputStream(s.getInputStream()); String receive, send;
            while((receive = in.readLine()) != null)
            {
                if(receive.equals("STOP"))
                break;
                System.out.println("Client Says : "+receive);
                System.out.print("Server Says : ");
                send = br.readLine();
                out.writeBytes(send+"\n");
            }
        }
    }
}
```

```
    }  
    br.close();  
    in.close();  
    out.close();  
    s.close();  
    }  
    catch(Exception e)  
    {  
        e.printStackTrace();  
    }  
}
```

2. ChatClient.java

```
import java.net.*;  
import java.io.*;  
class ChatClient  
{  
    public static void main(String args[])  
    {  
        try  
        {  
            Socket s = new Socket("Localhost",8000);  
            BufferedReader br = new BufferedReader(new  
                InputStreamReader(System.in));  
            DataOutputStream out = new  
                DataOutputStream(s.getOutputStream()); DataInputStream in = new  
                DataInputStream(s.getInputStream()); String msg;  
            System.out.println("To stop chatting with server type  
                STOP"); System.out.print("Client Says: "); while((msg =  
                br.readLine()) != null)  
            {  
                out.writeBytes(msg+"\n");  
                if(msg.equals("STOP"))  
                    break;  
                System.out.println("Server Says : "+in.readLine());  
                System.out.print("Client Says : ");  
            }  
            br.close();  
            in.close();  
            out.close();  
            s.close();  
        }  
        catch(Exception e)
```

```

{
    e.printStackTrace();
}
}
}

```

OUTPUT:

```

Administrator: Command Prompt - java ChatServer.java
ChatServer.java:10: error: unclosed string literal
System.out.println("Waiting for client to
^
ChatServer.java:11: error: illegal '.'
connect.."); Socket s = ss.accept();
^
ChatServer.java:11: error: not a statement
connect.."); Socket s = ss.accept();
^
ChatServer.java:11: error: unclosed string literal
connect.."); Socket s = ss.accept();
^
4 errors

H:\java pract>javac ChatServer.java
Note: ChatServer.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

H:\java pract>javac ChatServer.java
Note: ChatServer.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

H:\java pract>java ChatServer.java
ChatServer.java:17: warning: [deprecation] readLine() in DataInputStream has been deprecated
while((receive = in.readLine()) != null)
^
1 warning
Waiting for client to connect..
Client Says : hi
Server Says : hello how are you

```

```

Administrator: Command Prompt - java ChatClient.java
STOP"); System.out.print("Client Says: "); while((msg =
^
ChatClient.java:17: error: ';' expected
br.readLine()) != null)
^
6 errors

H:\java pract>javac ChatClient.java
ChatClient.java:10: error: cannot find symbol
BufferedReader br = new BufferedReader(newInputStreamReader(System.in));
^
symbol:   method newInputStreamReader(InputStream)
location: class ChatClient
Note: ChatClient.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
1 error

H:\java pract>javac ChatClient.java
Note: ChatClient.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

H:\java pract>java ChatClient.java
ChatClient.java:19: warning: [deprecation] readLine() in DataInputStream has been deprecated
System.out.println("Server Says : "+in.readLine());
^
1 warning
To stop chatting with server type STOP
Client Says: hi
Server Says : hello how are you
Client Says : 

```

PRACTICAL 3

3. Write a program for implementing Client Server communication model using UDP.

Aim: Write a program for implementing Client Server communication model using UDP.

Practical 3A: A client server based program using UDP to find if the number entered is even or odd.

Code:

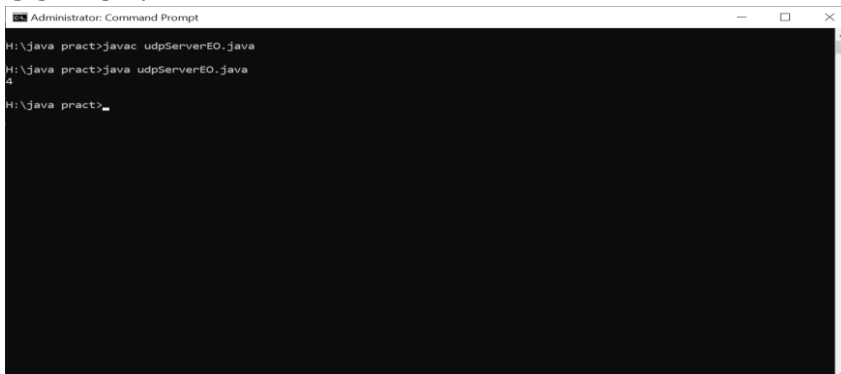
1. udpServerEO.java

```
import java.io.*;
import java.net.*;
public class udpServerEO
{
    public static void main(String args[])
    {
        try
        {
            DatagramSocket ds = new DatagramSocket(2000);
            byte b[] = new byte[1024];
            DatagramPacket dp = new DatagramPacket(b,b.length);
            ds.receive(dp);
            String str = new
            String(dp.getData(),0,dp.getLength());
            System.out.println(str);
            int a= Integer.parseInt(str);
            String s= new String();
            if (a%2 == 0)
            s = "Number is even";
            else
            s = "Number is odd";
            byte b1[] = new byte[1024];
            b1 = s.getBytes();
            DatagramPacket dp1 = new
            DatagramPacket(b1,b1.length,InetAddress.getLocalHost(),1000);
            ds.send(dp1);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

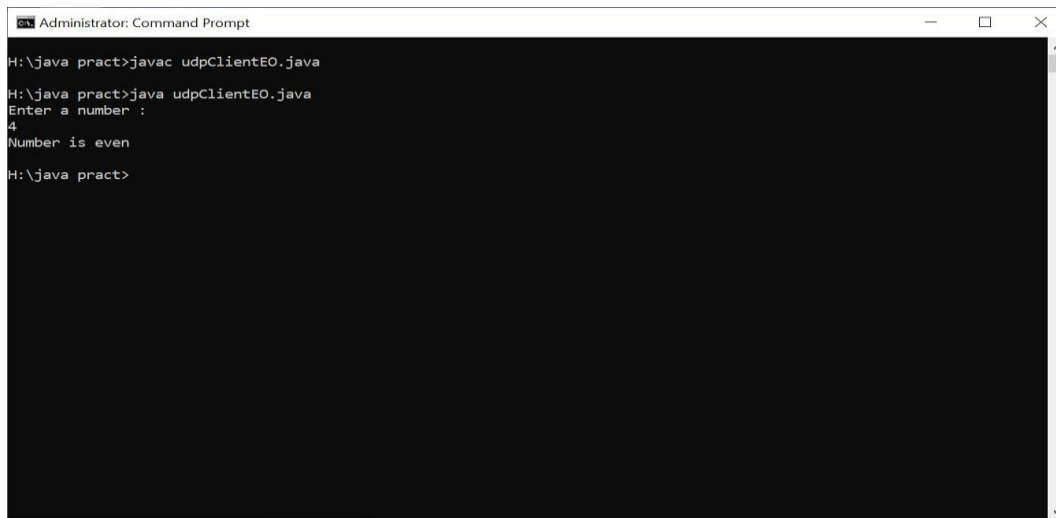
2. udpClientEO.java

```
import java.io.*;
import java.net.*;
public class udpClientEO
{
    public static void main(String args[])
    {
        try
        {
            DatagramSocket ds = new DatagramSocket(1000);
            BufferedReader br = new BufferedReader(new
            InputStreamReader(System.in));
            System.out.println("Enter a number : ");
            String num = br.readLine();
            byte b[] = new byte[1024];
            b=num.getBytes();
            DatagramPacket dp = new
            DatagramPacket(b,b.length,InetAddress.getLocalHost(),2000);
            ds.send(dp);
            byte b1[] = new byte[1024];
            DatagramPacket dp1 = new
            DatagramPacket(b1,b1.length); ds.receive(dp1);
            String str = new
            String(dp1.getData(),0,dp1.getLength());
            System.out.println(str);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

OUTPUT:



```
Administrator: Command Prompt
H:\Java pract>javac udpServerEO.java
H:\Java pract>java udpServerEO.java
H:\Java pract>
```



```
Administrator: Command Prompt
H:\java pract>javac udpClientEO.java
H:\java pract>java udpClientEO.java
Enter a number :
4
Number is even
H:\java pract>
```

Practical 3B: A client server based program using UDP to find the factorial of the entered number. **Code:**

1. udpServerFact.java

```
import java.io.*;
import java.net.*;
public class udpServerFact
{
    public static void main(String args[])
    {
        try
        {
            DatagramSocket ds = new DatagramSocket(2000);
            byte b[] = new byte[1024];
            DatagramPacket dp = new DatagramPacket(b,b.length);
            ds.receive(dp);
            String str = new
            String(dp.getData(),0,dp.getLength());
            System.out.println(str);
            int a= Integer.parseInt(str);
            int f = 1, i;
            String s= new String();
            for(i=1;i<=a;i++)
            {
                f=f*i;
            }
            s=Integer.toString(f);
            String str1 = "The Factorial of " + str + " is : " +
            f; byte b1[] = new byte[1024]; b1 =
            str1.getBytes();
```

```
DatagramPacket dp1 = new
DatagramPacket(b1,b1.length,InetAddress.getLocalHost(),1000);
ds.send(dp1);
}
catch(Exception e)
{
e.printStackTrace();
}
}
}
```

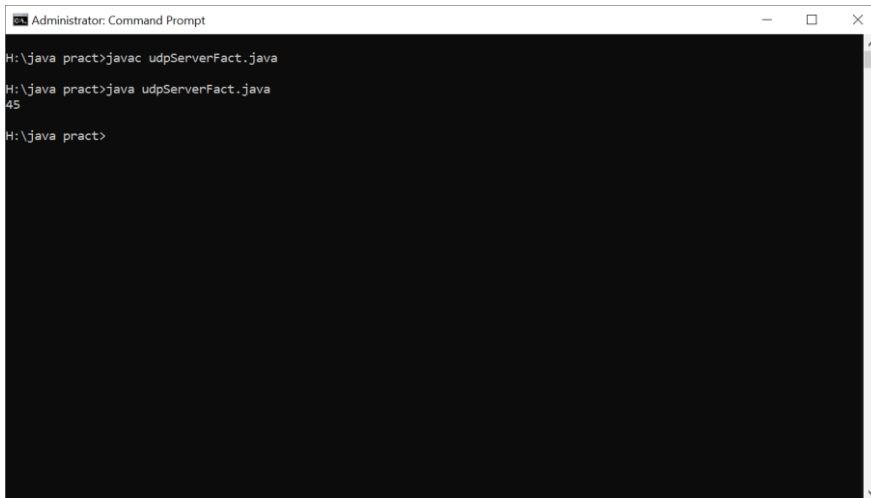
2. **udpClientFact.java**

```
import java.io.*;
import java.net.*;
public class udpClientFact
{
public static void main(String args[])
{
try
{
DatagramSocket ds = new DatagramSocket(1000);
BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
System.out.println("Enter a number : ");
String num = br.readLine();
byte b[] = new byte[1024];
b=num.getBytes();
DatagramPacket dp = new
DatagramPacket(b,b.length,InetAddress.getLocalHost(),2000);
ds.send(dp);
byte b1[] = new byte[1024];
DatagramPacket dp1 = new DatagramPacket(b1,b1.length);
ds.receive(dp1);
String str = new
String(dp1.getData(),0,dp1.getLength());
System.out.println(str);
}
catch(Exception e)
{
e.printStackTrace();
}
}
```

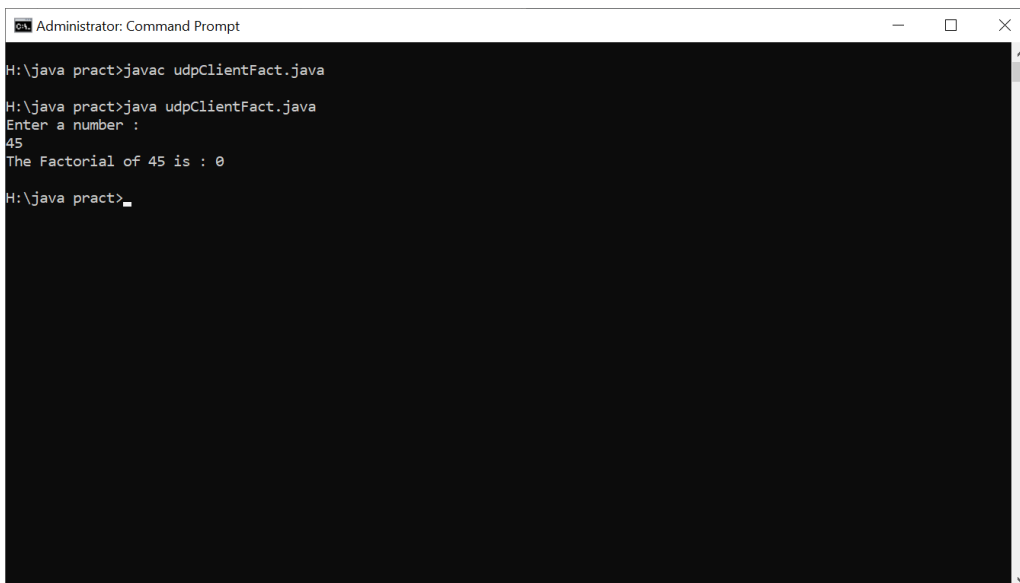
Seat No.: 3254185

}

OUTPUT:



```
Administrator: Command Prompt
H:\java pract>javac udpServerFact.java
H:\java pract>java udpServerFact.java
45
H:\java pract>
```



```
Administrator: Command Prompt
H:\java pract>javac udpClientFact.java
H:\java pract>java udpClientFact.java
Enter a number :
45
The Factorial of 45 is : 0
H:\java pract>
```

Practical 3C:A program to implement simple calculator operations like addition, subtraction, multiplication and division.

Code:

1. **RPCServer.java**

```
import java.util.*;
import java.net.*;
class RPCServer
{
    DatagramSocket ds;
    DatagramPacket dp;
    String str,methodName,result;
    int val1,val2;
    RPCServer()
```

```
{
try
{
ds=new DatagramSocket(1200);
byte b[]=new byte[4096];
while(true)
{
dp=new DatagramPacket(b,b.length);
ds.receive(dp);
str=new String(dp.getData(),0,dp.getLength());
if(str.equalsIgnoreCase("q"))
{
System.exit(1);
}
else
{
StringTokenizer st = new StringTokenizer(str,"
"); int i=0;
while(st.hasMoreTokens())
{
String token=st.nextToken();
methodName=token;
val1 = Integer.parseInt(st.nextToken());
val2 = Integer.parseInt(st.nextToken());
}
}
System.out.println(str);
InetAddress ia = InetAddress.getLocalHost();
if(methodName.equalsIgnoreCase("add"))
{
result= "" + add(val1,val2);
}
else if(methodName.equalsIgnoreCase("sub"))
{
result= "" + sub(val1,val2);
}
else if(methodName.equalsIgnoreCase("mul"))
{
result= "" + mul(val1,val2);
}
else if(methodName.equalsIgnoreCase("div"))
```

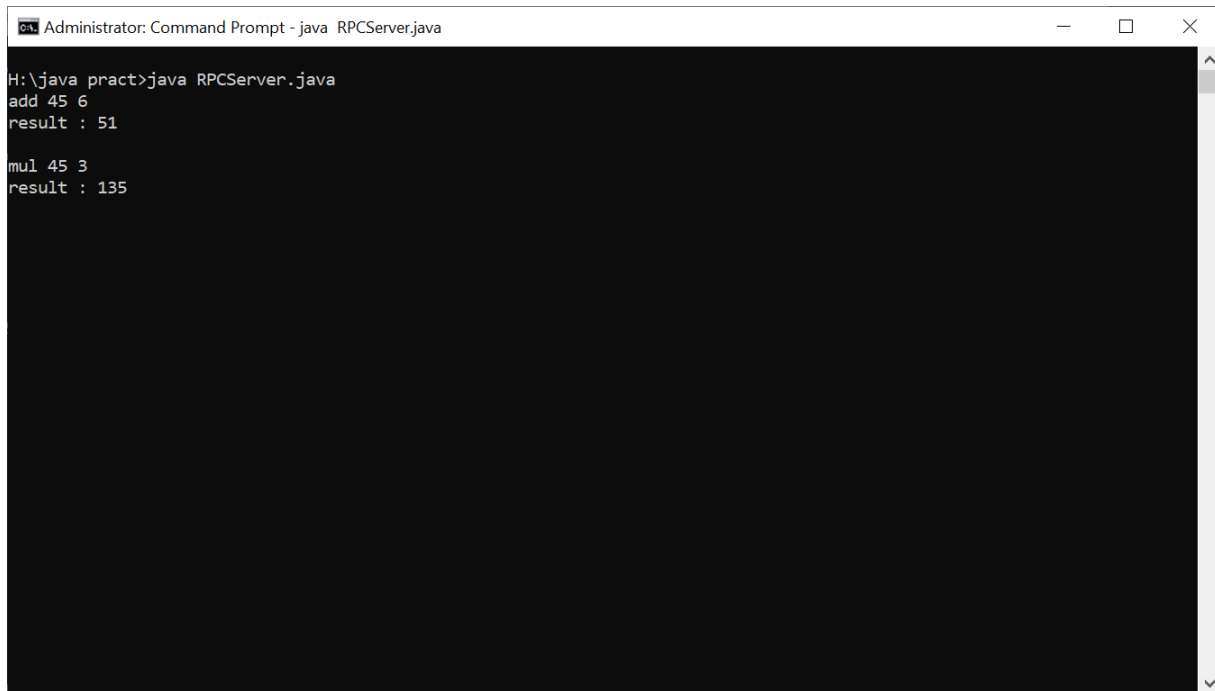
```
{
result= "" + div(val1,val2);
}
byte b1[]=result.getBytes();
DatagramSocket ds1 = new DatagramSocket();
DatagramPacket dp1 = new
DatagramPacket(b1,b1.length,InetAddress.getLocalHost(), 1300);
System.out.println("result :
"+result+"\n"); ds1.send(dp1);
}
}
catch (Exception e)
{
e.printStackTrace();
}
}
public int add(int val1, int val2)
{
return val1+val2;
}
public int sub(int val3, int val4)
{
return val3-val4;
}
public int mul(int val3, int val4)
{
return val3*val4;
}
public int div(int val3, int val4)
{
return val3/val4;
}
public static void main(String[] args)
{
new RPCServer();
}
}
```

2. **RPCClient.java**

```
import java.io.*;
import java.net.*;
class RPCClient
```

```
{
RPCClient()
{
try
{
InetAddress ia = InetAddress.getLocalHost();
DatagramSocket ds = new DatagramSocket();
DatagramSocket ds1 = new DatagramSocket(1300);
System.out.println("\nRPC Client\n");
System.out.println("Enter method name and parameter like add 3 4\n");
while (true)
{
BufferedReader br = new
BufferedReader(new InputStreamReader(System.in));
String str = br.readLine();
byte b[] = str.getBytes();
DatagramPacket dp = new
DatagramPacket(b,b.length,ia,1200);
ds.send(dp);
dp = new DatagramPacket(b,b.length);
ds1.receive(dp);
String s = new String(dp.getData(),0,dp.getLength());
System.out.println("\nResult = " + s + "\n");
}
}
catch (Exception e)
{
e.printStackTrace();
}
}
public static void main(String[] args)
{
new RPCClient();
}
}
```

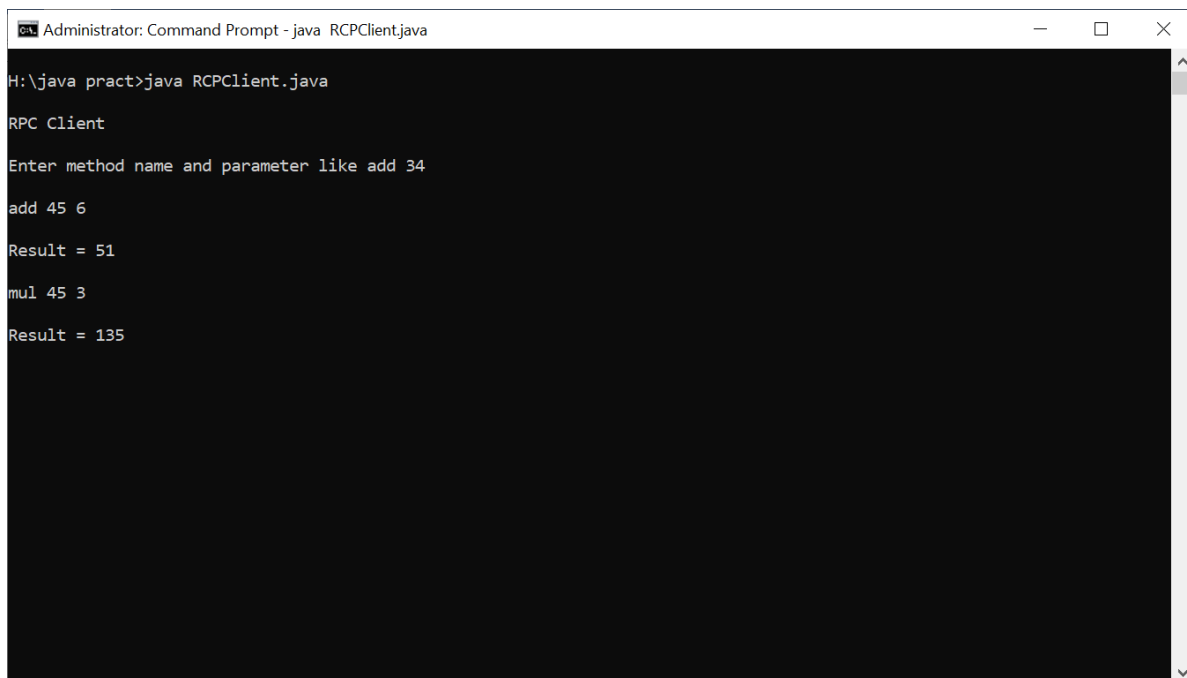
OUTPUT:



```
Administrator: Command Prompt - java RPCServer.java

H:\java pract>java RPCServer.java
add 45 6
result : 51

mul 45 3
result : 135
```



```
Administrator: Command Prompt - java RPCClient.java

H:\java pract>java RPCClient.java

RPC Client

Enter method name and parameter like add 34

add 45 6

Result = 51

mul 45 3

Result = 135
```

PRACTICAL 4

4. A multicast Socket example

Aim: A multicast Socket example.

Code:

1. BroadcastServer.java

```
import java.net.*;
import java.io.*;
import java.util.*;
public class BroadcastServer
{
    public static final int PORT = 1234;
    public static void main(String args[])throws
    Exception {
        MulticastSocket socket;
        DatagramPacket packet;
        InetAddress address;
        // set the multicast address to your local subnet
        address = InetAddress.getByName("239.1.2.3");
        socket = new MulticastSocket();
        // join a Multicast group and send the group
        messages socket.joinGroup(address);
        byte[] data = null;
        for(;;)
        {
            Thread.sleep(10000);
            System.out.println("Sending "); String
            str = ("This is Neha Calling ... ");
            data = str.getBytes();
            packet = new DatagramPacket(data, str.length(),address,PORT);
            // Sends the packet
            socket.send(packet);
        } // end for
    } // end main
} // end class BroadcastServer
```

2. BroadcastClient.java

```
import java.net.*;
import java.io.*;
public class BroadcastClient
{
    public static final int PORT = 1234;
    public static void main(String args[])throws
    Exception {
```

Seat No.: 3254185

```
MulticastSocket socket;
DatagramPacket packet;
InetAddress address;
// set the mulitcast address to your local subnet
address = InetAddress.getByName("239.1.2.3");
socket = new MulticastSocket(PORT);
//join a Multicast group and wait for a
message socket.joinGroup(address); byte[]
data = new byte[100];
packet = new DatagramPacket(data,data.length);
for(;;)
{
// receive the packets
socket.receive(packet);
String str = new String(packet.getData()); System.out.println("Message received from "+
packet.getAddress() + "Message is : "+str);
} // for
} // main
} // end BroadcastClient
```

OUTPUT:

 C:\WINDOWS\system32\cmd.exe

```
E:\Ds_Yugi>javac BroadcastServer.java
```

```
E:\Ds_Yugi>java BroadcastServer
Sending
Sending
Sending
Sending
- ..
```

```
E:\Ds_Yugi>javac BroadcastClient.java
```

```
E:\Ds_Yugi>java BroadcastClient
```

```
Message received from /10.29.26.232 Message is: This is Neha Calling....
Message received from /10.29.26.232 Message is: This is Neha Calling....
Message received from /10.29.26.232 Message is: This is Neha Calling....
Message received from /10.29.26.232 Message is: This is Neha Calling....
```

PRACTICAL 5

5. Write a program to show the Date using RMI.

Aim: Write a program to show the Date using RMI.

Code:

1. InterDate.java

```
import java.rmi.*;
public interface InterDate extends Remote
{
    public String display() throws Exception;
}
```

2. ServerDate.java

```
import java.rmi.*;
import java.rmi.server.*;
import java.util.*;
public class ServerDate extends UnicastRemoteObject implements
InterDate {
    public ServerDate() throws Exception
    {
    }
    public String display() throws Exception
    {
        String str = "";
        Date d = new Date();
        str = d.toString();
        return str;
    }
    public static void main(String args[]) throws
Exception {
        ServerDate s1 = new ServerDate();
        Naming.bind("DS",s1);
        System.out.println("Object registered .....");
    }
}
```

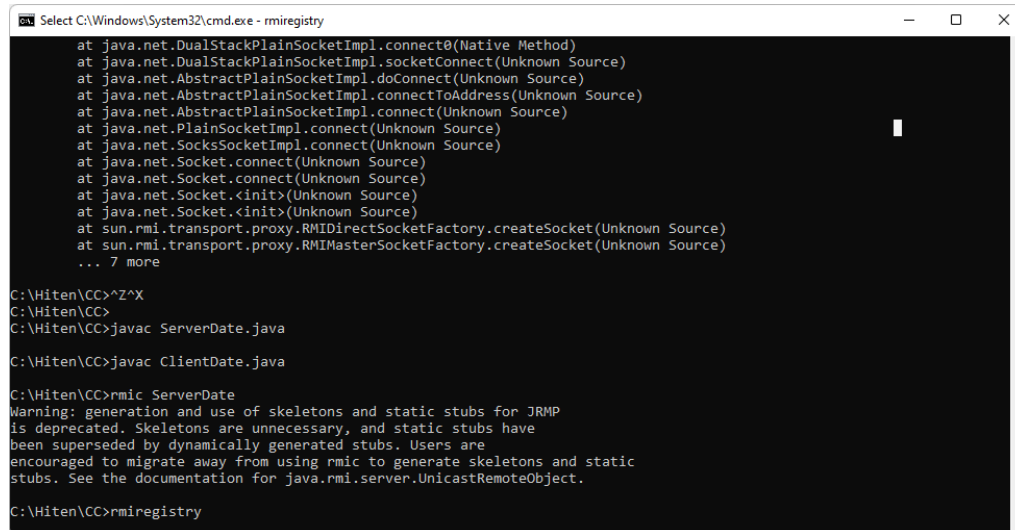
3. ClientDate.java

```
import java.rmi.*;
import java.io.*;
public class ClientDate
{
    public static void main(String args[]) throws
Exception {
        String s1;
        InterDate h1 = (InterDate)Naming.lookup("DS");
```

Seat No.: 3254185

```
s1 = h1.display();
System.out.println(s1);
}
}
```

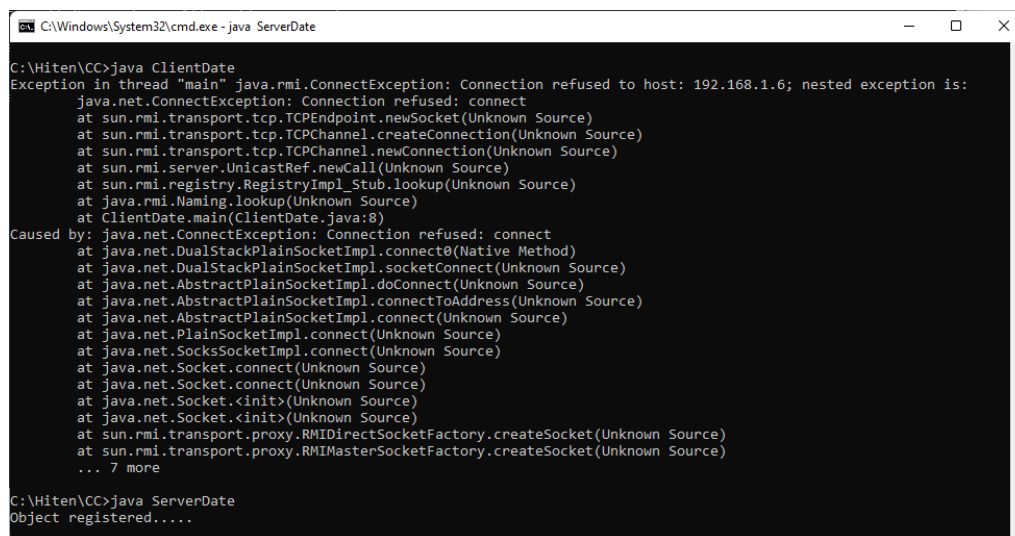
OUTPUT:



```
Select C:\Windows\System32\cmd.exe - rmiregistry

at java.net.DualStackPlainSocketImpl.connect0(Native Method)
at java.net.DualStackPlainSocketImpl.socketConnect(Unknown Source)
at java.net.AbstractPlainSocketImpl.doConnect(Unknown Source)
at java.net.AbstractPlainSocketImpl.connectToAddress(Unknown Source)
at java.net.AbstractPlainSocketImpl.connect(Unknown Source)
at java.net.PlainSocketImpl.connect(Unknown Source)
at java.net.SocksSocketImpl.connect(Unknown Source)
at java.net.Socket.connect(Unknown Source)
at java.net.Socket.connect(Unknown Source)
at java.net.Socket.<init>(Unknown Source)
at java.net.Socket.<init>(Unknown Source)
at sun.rmi.transport.proxy.RMIDirectSocketFactory.createSocket(Unknown Source)
at sun.rmi.transport.proxy.RMIMasterSocketFactory.createSocket(Unknown Source)
... 7 more

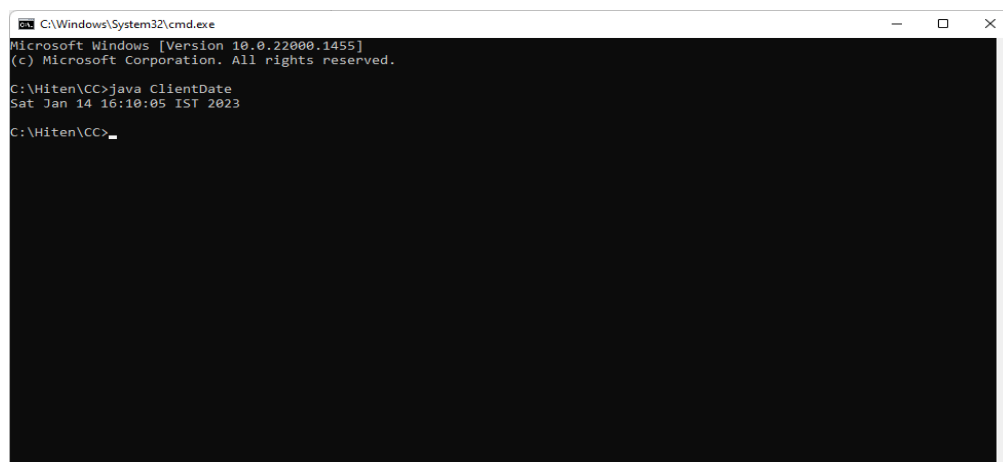
C:\Hiten\CC>^Z^X
C:\Hiten\CC>
C:\Hiten\CC>javac ServerDate.java
C:\Hiten\CC>javac ClientDate.java
C:\Hiten\CC>rmiic ServerDate
Warning: generation and use of skeletons and static stubs for JRMP
is deprecated. Skeletons are unnecessary, and static stubs have
been superseded by dynamically generated stubs. Users are
encouraged to migrate away from using rmiic to generate skeletons and static
stubs. See the documentation for java.rmi.server.UnicastRemoteObject.
C:\Hiten\CC>rmiregistry
```



```
C:\Windows\System32\cmd.exe - java ServerDate

C:\Hiten\CC>java ClientDate
Exception in thread "main" java.rmi.ConnectException: Connection refused to host: 192.168.1.6; nested exception is:
    java.net.ConnectException: connect
    at sun.rmi.transport.tcp.TCPEndpoint.newSocket(Unknown Source)
    at sun.rmi.transport.tcp.TCPChannel.createConnection(Unknown Source)
    at sun.rmi.transport.tcp.TCPChannel.newConnection(Unknown Source)
    at sun.rmi.server.UnicastRef.newCall(Unknown Source)
    at sun.rmi.registry.RegistryImpl_Stub.lookup(Unknown Source)
    at java.rmi.Naming.lookup(Unknown Source)
    at ClientDate.main(ClientDate.java:8)
Caused by: java.net.ConnectException: Connection refused: connect
    at java.net.DualStackPlainSocketImpl.connect0(Native Method)
    at java.net.DualStackPlainSocketImpl.socketConnect(Unknown Source)
    at java.net.AbstractPlainSocketImpl.doConnect(Unknown Source)
    at java.net.AbstractPlainSocketImpl.connectToAddress(Unknown Source)
    at java.net.AbstractPlainSocketImpl.connect(Unknown Source)
    at java.net.PlainSocketImpl.connect(Unknown Source)
    at java.net.SocksSocketImpl.connect(Unknown Source)
    at java.net.Socket.connect(Unknown Source)
    at java.net.Socket.connect(Unknown Source)
    at java.net.Socket.<init>(Unknown Source)
    at java.net.Socket.<init>(Unknown Source)
    at sun.rmi.transport.proxy.RMIDirectSocketFactory.createSocket(Unknown Source)
    at sun.rmi.transport.proxy.RMIMasterSocketFactory.createSocket(Unknown Source)
    ... 7 more

C:\Hiten\CC>java ServerDate
Object registered.....
```



```
C:\Windows\System32\cmd.exe

Microsoft Windows [Version 10.0.22000.1455]
(c) Microsoft Corporation. All rights reserved.

C:\Hiten\CC>java ClientDate
Sat Jan 14 16:10:05 IST 2023

C:\Hiten\CC>_
```

PRACTICAL 6

6. Write a program to convert digit using RMI.

Aim: Write a program to convert digit using RMI.

Code:

1. InterDate.java

```
import java.rmi.*;
public interface InterConvert extends Remote
{
    public String convertDigit(String no) throws Exception;
}
```

2. ServerDate.java

```
import java.rmi.*;
import java.rmi.server.*;
public class ServerConvert extends UnicastRemoteObject implements
InterConvert {
    public ServerConvert() throws Exception
    {
    }
    public String convertDigit(String no) throws Exception
    {
        String str = "";
        for(int i = 0; i < no.length(); i++)
        {
            int p = no.charAt(i);
            if( p == 48)
            {
                str += "zero ";
            }
            if( p == 49)
            {
                str += "one ";
            }
            if( p == 50)
            {
                str += "two ";
            }
            if( p == 51)
            {
                str += "three ";
            }
            if( p == 52)
            {
                str += "four ";
            }
        }
    }
}
```

```
}
if( p == 53)
{
    str += "five ";
}
if( p == 54)
{
    str += "six ";
}
if( p == 55)
{
    str += "seven ";
}
if( p == 56)
{
    str += "eight ";
}
if( p == 57)
{
    str += "nine ";
}
}
return str;
}

public static void main(String args[]) throws
Exception {
    ServerConvert s1 = new ServerConvert();
    Naming.bind("Wrd",s1);
    System.out.println("Object registered ....");
}
}
```

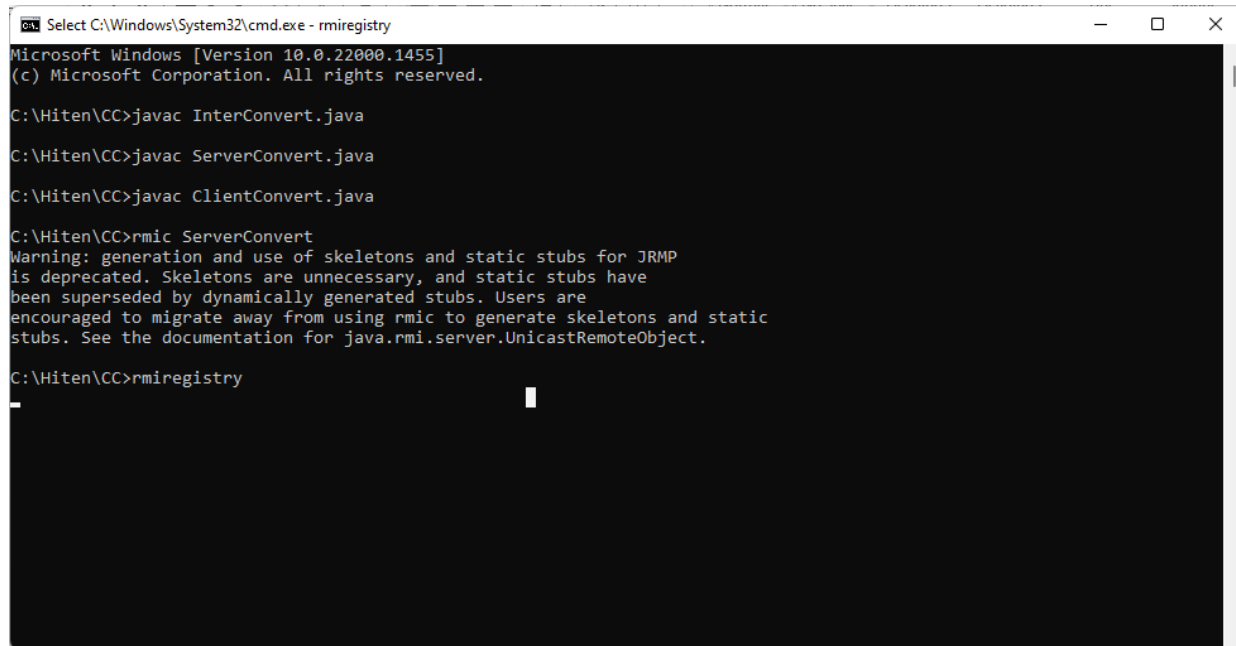
3. ClientDate.java

```
import java.rmi.*;
import java.io.*;
public class ClientConvert
{
    public static void main(String args[]) throws
    Exception {
        InterConvert h1 =
        (InterConvert)Naming.lookup("Wrd"); BufferedReader
        br = new BufferedReader(new
        InputStreamReader(System.in));
        System.out.println("Enter a number : \t"); String no = br.readLine();
```

Seat No.: 3254185

```
String ans = h1.convertDigit(no);
System.out.println("The word representation of the entered digit is : " +ans);
}
}
```

OUTPUT:



```
Select C:\Windows\System32\cmd.exe - rmiregistry
Microsoft Windows [Version 10.0.22000.1455]
(c) Microsoft Corporation. All rights reserved.

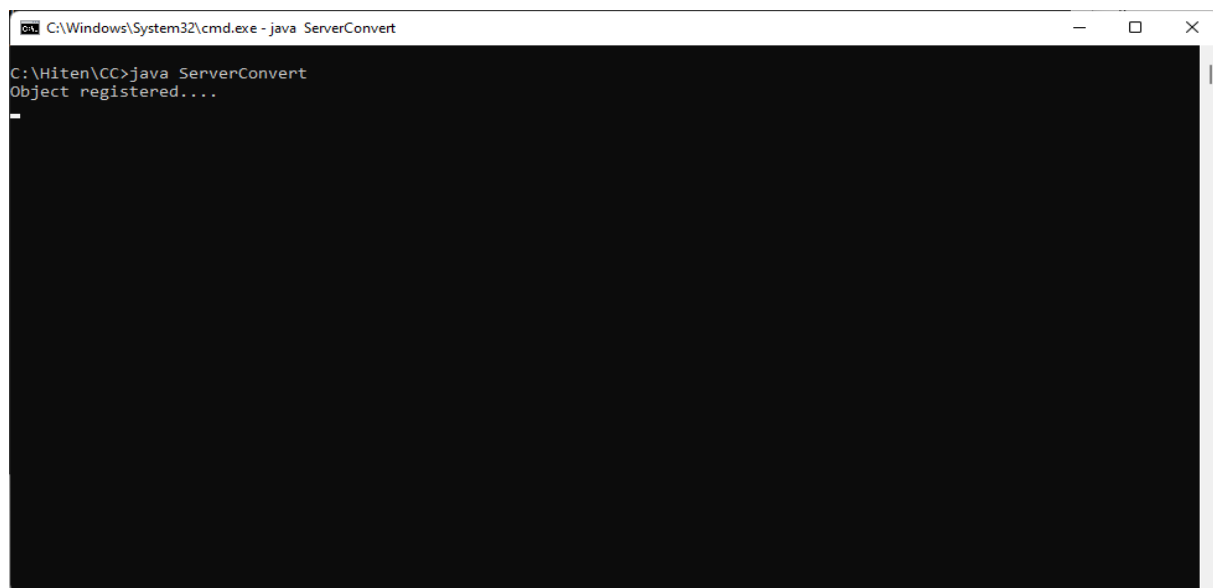
C:\Hiten\CC>javac InterConvert.java

C:\Hiten\CC>javac ServerConvert.java

C:\Hiten\CC>javac ClientConvert.java

C:\Hiten\CC>rmic ServerConvert
Warning: generation and use of skeletons and static stubs for JRMP
is deprecated. Skeletons are unnecessary, and static stubs have
been superseded by dynamically generated stubs. Users are
encouraged to migrate away from using rmic to generate skeletons and static
stubs. See the documentation for java.rmi.server.UnicastRemoteObject.

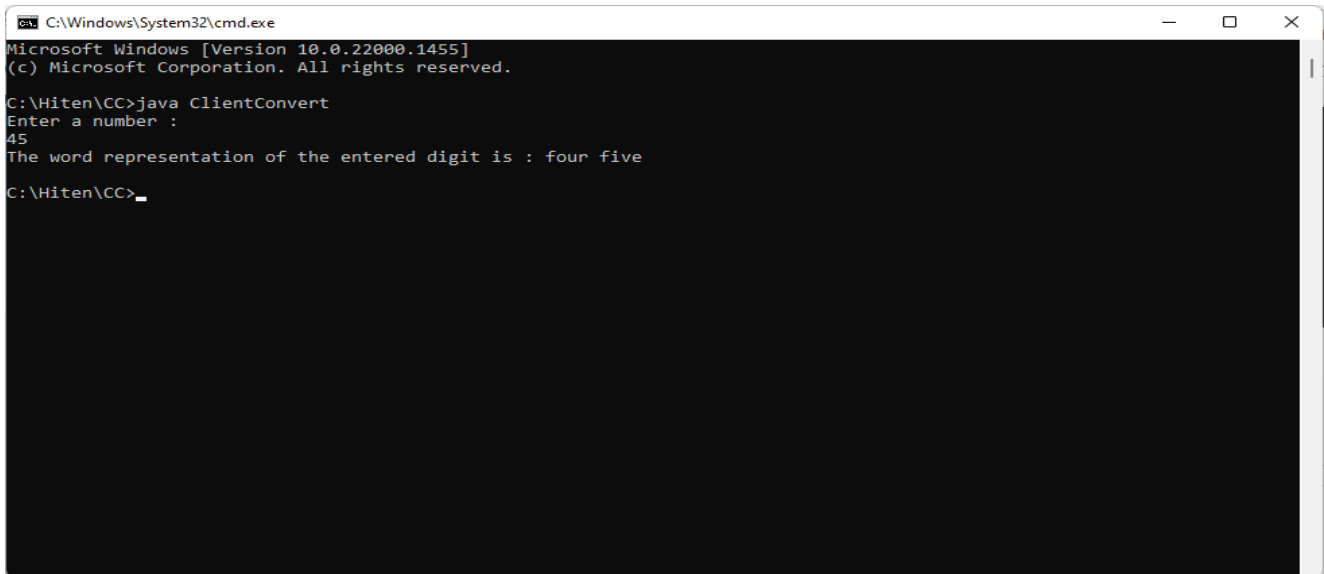
C:\Hiten\CC>rmiregistry
_
```



```
C:\Windows\System32\cmd.exe - java ServerConvert

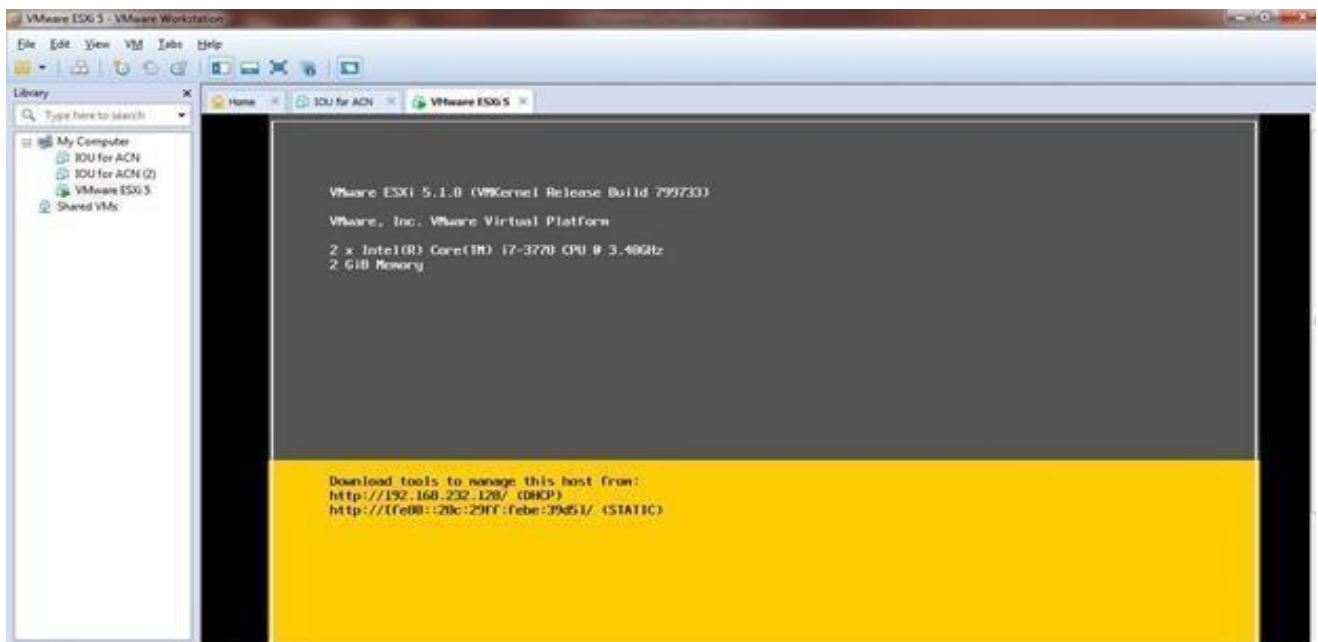
C:\Hiten\CC>java ServerConvert
Object registered....
_
```

Seat No.: 3254185



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.22000.1455]
(c) Microsoft Corporation. All rights reserved.

C:\Hiten\CC>java ClientConvert
Enter a number :
45
The word representation of the entered digit is : four five
C:\Hiten\CC>
```



PRACTICAL 7

7. Implement virtualization using VMWare ESXi Server and managing with vCenter

Aim: Implement virtualization using VMWare ESXi Server and managing with vCenter

Steps:

Install **ESXi iso** in VMWare workstation.

Install VMware vSphere Client



In vSphere create new **Virtual Machine**. Install Windows XP iso file and open it

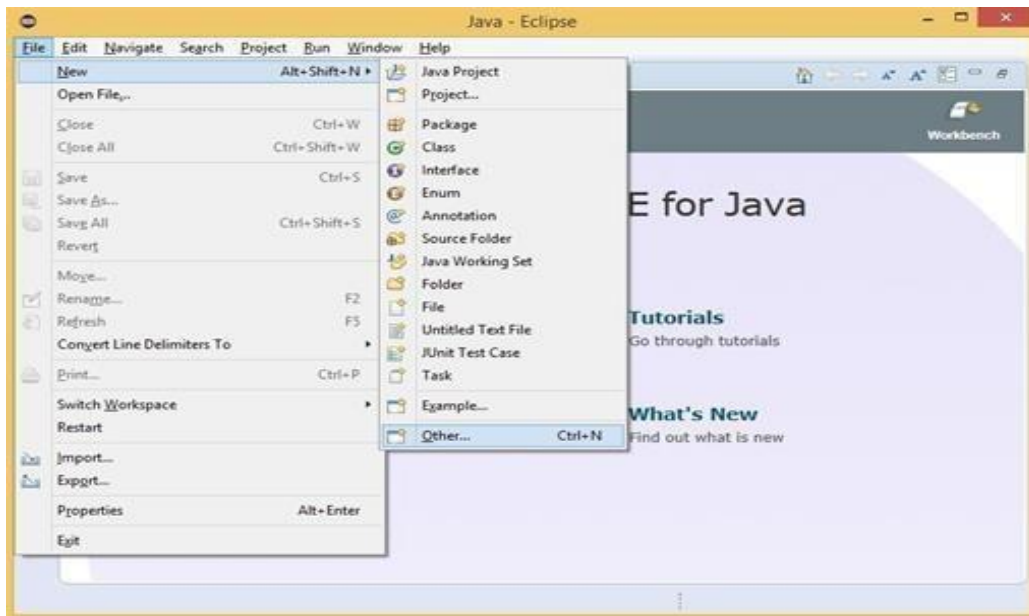


PRACTICAL 8

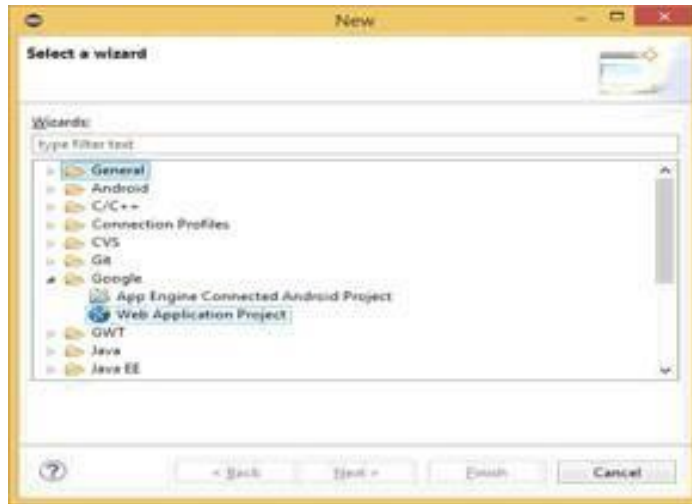
8. Develop application for Google App Engine

Aim: Develop application for Google App Engine

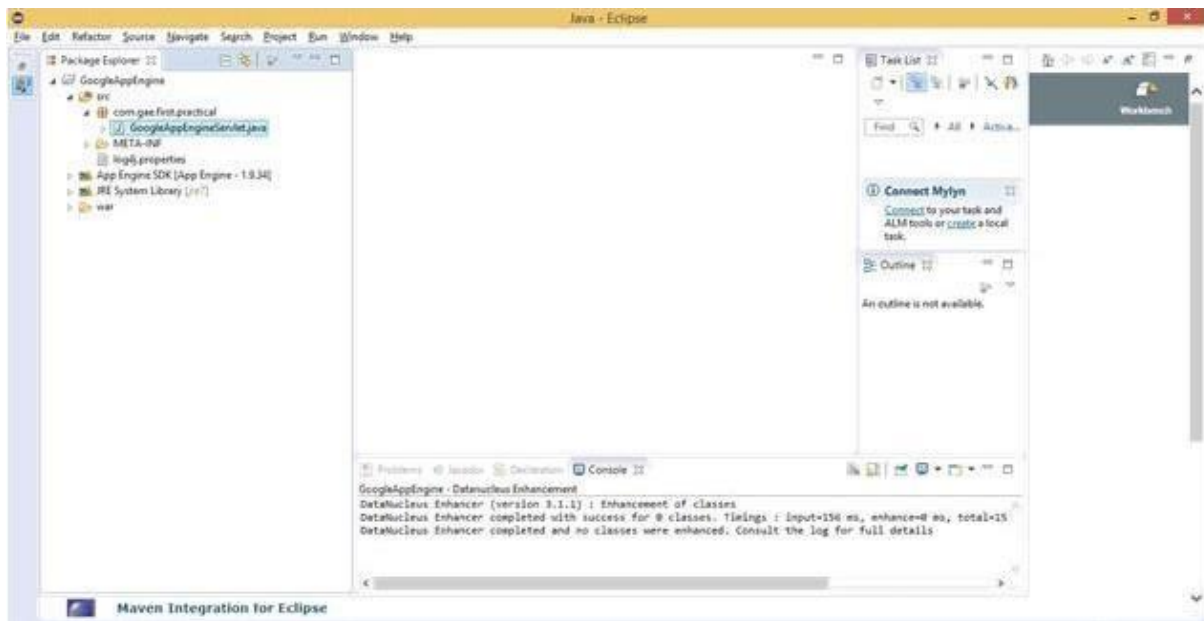
- Open Eclipse Luna. Go to **Help Menu Install New Software...**
- In **Install** window Click on the “**Add**” button besides the **Work with** textbox. **AddRepository** window appears. Enter the **Location** as “<https://dl.google.com/eclipse/plugin/4.4>” and click on “**OK**” button.
- From the available softwares select the required softwares and tools as shown in the below image for the **GAE**. Then click on the “**Next**” button.
- In the **Install Details** window click on “**Next**” button.
- In the Next Window "Review the Items to be Installed" then click on “**Next**”
- In the next window for Review Licenses select the option “**I accept.....**” and click on “**Finish**” Button.
- After Installation you will get option to “**Restart Eclipse**”, click on **Yes**. So that the software you selected gets updated...
- Now, go to **File Menu_New_Other...**



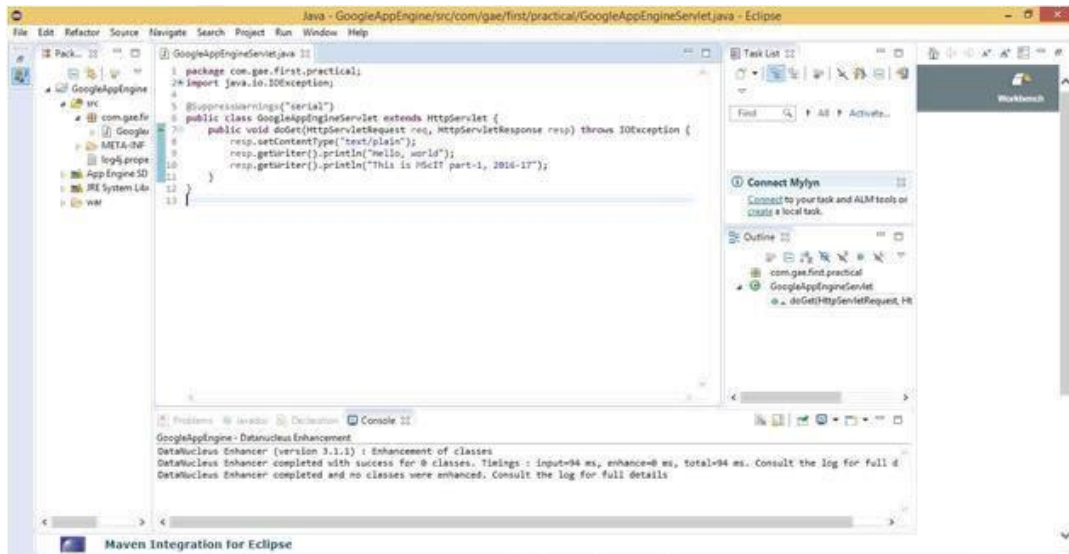
In the New window select **Google Web Application Project** and click on “**Next**” button.



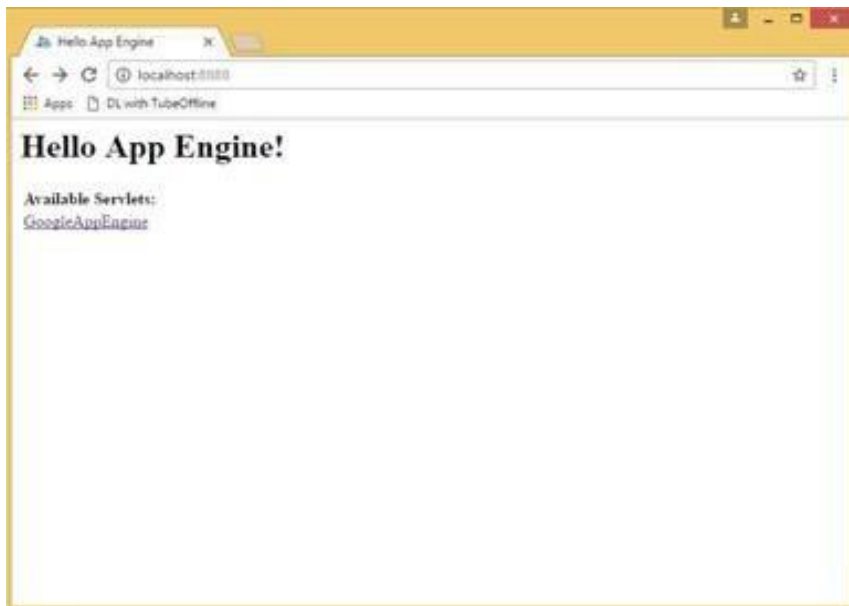
Enter the details for the new Web application project. Deselect the **Use Google Web Toolkit** option under the section **Google SDKs**. Click on the **“Finish”** button. From the **Package Explorer** open the **.java** file (Here it is **“Google_App_EngineServlet.java”**).



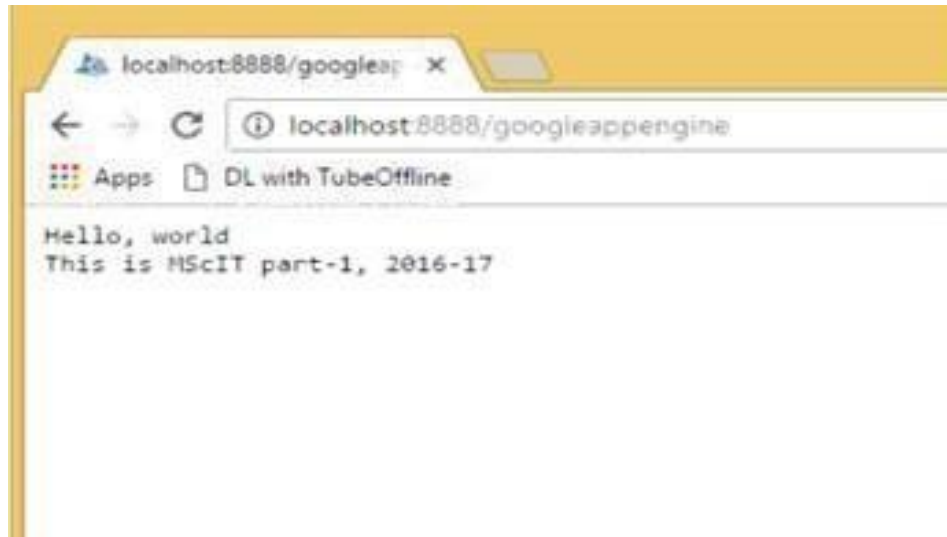
Edit the file as required (Unedited file too can be used. Here the editing is done to “what should be displayed” on the browser). **Save** the file. Click on the **Run** option available on the Tools bar.



In the browser (Here, Google Chrome) type the address as **“localhost:8888”** which is **“Default”**.



In **localhost:8888** the link to the **Google_App_EngineServlet.java** file as **Google_App_Engine** is displayed. Click on this link. It will direct you to **“localhost:8888/Google_App_Engine”**.



The **output text entered** in the **java** program is **displayed as the output** when clicked the link “Google App Engine”

SOFT COMPUTING TECHNIQUES



NURTURING POTENTIAL

SAKET GYANPEETH'S
SAKET COLLEGE OF ARTS, SCIENCE AND COMMERCE

(Permanently Affiliated to University of Mumbai)

NAAC Accredited B Grade

Saket Vidyanagari Marg, Chinchpada Road, Katemanivali, Kalyan (East) – 421306, Dist.

Department of Information Technology

Thane (MAH)

CERTIFICATE

This is to certify that

ANSHUMAN GAURISHANKAR SINGH of MSc Information Technology Part-I

Class has satisfactory carried out the required practical in the subject.

SOFT COMPUTING TECHNIQUES

For the Academic year 2022 – 2023

Practical In-Charge

Head of Department

External Examiner

INDEX

Sr. No.	Aim	Sign
1	To implement Perceptron Algorithm	
2	To implement Back Propagation Algorithm	
3	To implement Hebb's Rule in C++	
4	To implement McCulloch Pitts model for XOR gate	
5	To implement Maxnet	
6	To implement BAM using python	
7	Find the ratios using fuzzy logic	
8	Fuzzy logic for tipping	

PRACTICAL 1

AIM: To implement Perceptron Algorithm

CODE:

```
from sklearn import datasets
import matplotlib.pyplot as plt
import numpy as np
X, y = datasets.make_blobs(n_samples=150, n_features=2,
                           centers=2, cluster_std=1.05,
                           random_state=2)
X, y = datasets.make_blobs(n_samples=150, n_features=2,
                           centers=2, cluster_std=1.05,
                           random_state=2)
fig = plt.figure(figsize=(10,8))
plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], 'r^')
plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], 'bs')
plt.xlabel("feature 1")
plt.ylabel("feature 2")
plt.title('Random Classification Data with 2 classes')
```



```
def step_func(z):
    return 1.0 if (z > 0) else 0.0
# looping for every example.
for idx, x_i in enumerate(X):

    # Insering 1 fordef perceptron(X, y, lr, epochs):

    # X --> Inputs.
```

```
# y --> labels/target.
# lr --> learning rate.
# epochs --> Number of iterations.

# m-> number of training examples
# n-> number of features
m, n = X.shape

# Initializing parameters(theta) to zeros.
# +1 in n+1 for the bias term.
theta = np.zeros((n+1,1))

# Empty list to store how many examples were
# misclassified at every iteration.
n_miss_list = []

# Training.
for epoch in range(epochs):

    # variable to store #misclassified.
    n_miss = 0
    bias, X0 = 1.
    x_i = np.insert(x_i, 0, 1).reshape(-1,1)

    # Calculating prediction/hypothesis.
    y_hat = step_func(np.dot(x_i.T, theta))

    # Updating if the example is misclassified.
    if (np.squeeze(y_hat) - y[idx]) != 0:
        theta += lr*((y[idx] - y_hat)*x_i)

    # Incrementing by 1.
    n_miss += 1

    # Appending number of misclassified examples
    # at every iteration.
    n_miss_list.append(n_miss)

return theta, n_miss_list
def plot_decision_boundary(X, theta):

    # X --> Inputs
    # theta --> parameters
```

```
# The Line is  $y=mx+c$ 
# So, Equate  $mx+c = \theta_0.X_0 + \theta_1.X_1 + \theta_2.X_2$ 
# Solving we find m and c
x1 = [min(X[:,0]), max(X[:,0])]
m = -theta[1]/theta[2]
c = -theta[0]/theta[2]
x2 = m*x1 + c

# Plotting
fig = plt.figure(figsize=(10,8))
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "r^")
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "bs")
plt.xlabel("feature 1")
plt.ylabel("feature 2")
plt.title('Perceptron Algorithm')
plt.plot(x1, x2, 'y-')
theta, miss_l = perceptron(X, y, 0.5, 100)
plot_decision_boundary(X, theta)
```

OUTPUT:



CONCLUSION: Thus implemented the experiment successfully.

PRACTICAL 2

AIM: To implement Back Propagation Algorithm

CODE:

```
# Import Libraries
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load dataset
data = load_iris()

# Get features and target
X=data.data
y=data.target

# Get dummy variable
y = pd.get_dummies(y).values
y[:3]
```

Output: array([[1, 0, 0], [1, 0, 0], [1, 0, 0]], dtype=uint8)

```
#Split data into train and test data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=20, random_state=4)

# Initialize variables
learning_rate = 0.1
iterations = 5000
N = y_train.size

# number of input features
input_size = 4

# number of hidden layers neurons
hidden_size = 2

# number of neurons at the output layer
output_size = 3
results = pd.DataFrame(columns=["mse", "accuracy"])

# Initialize weights
np.random.seed(10)

# initializing weight for the hidden layer
W1 = np.random.normal(scale=0.5, size=(input_size, hidden_size))

# initializing weight for the output layer
W2 = np.random.normal(scale=0.5, size=(hidden_size , output_size))

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

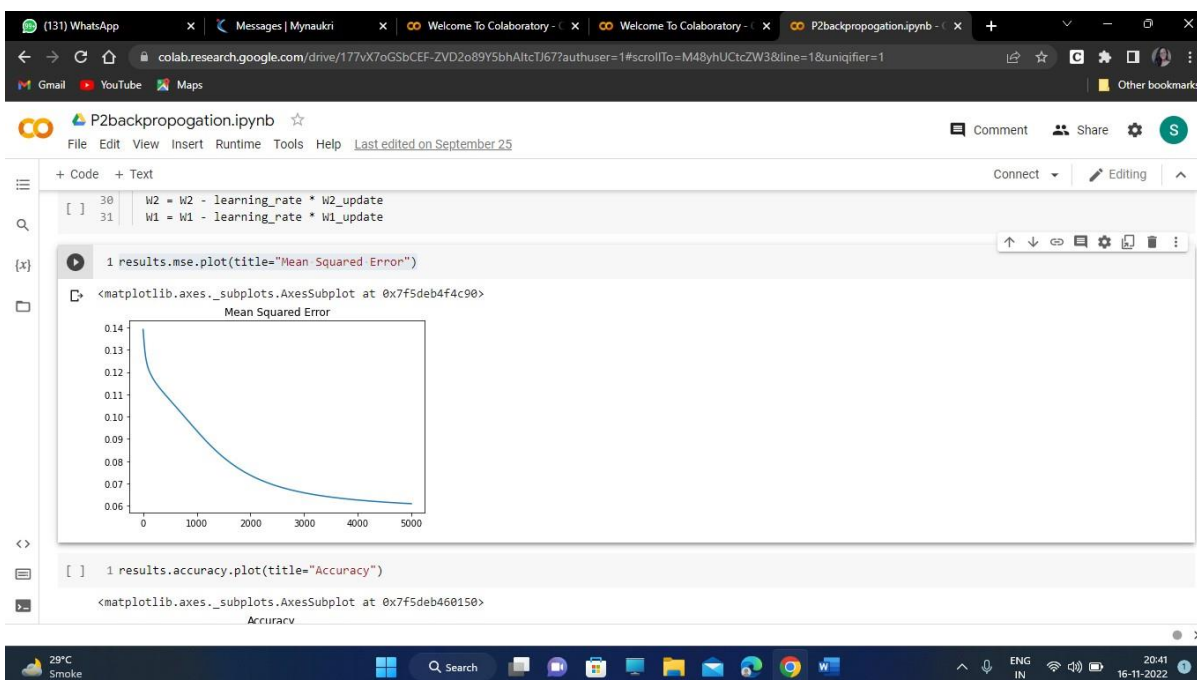
def mean_squared_error(y_pred, y_true):
    return ((y_pred - y_true)**2).sum() / (2*y_pred.size)

def accuracy(y_pred, y_true):
```

```

acc = y_pred.argmax(axis=1) == y_true.argmax(axis=1)
return acc.mean()
for itr in range(iterations):
    # feedforward propagation
    # on hidden layer
    Z1 = np.dot(X_train, W1)
    A1 = sigmoid(Z1)
    # on output layer
    Z2 = np.dot(A1, W2)
    A2 = sigmoid(Z2)
    # Calculating error
    mse = mean_squared_error(A2, y_train)
    acc = accuracy(A2, y_train)
    results.append({"mse":mse, "accuracy":acc},ignore_index=True )
    # backpropagation
    E1 = A2 - y_train
    dW1 = E1 * A2 * (1 - A2)
    E2 = np.dot(dW1, W2.T)
    dW2 = E2 * A1 * (1 - A1)
    # weight updates
    W2_update = np.dot(A1.T, dW1) / N
    W1_update = np.dot(X_train.T, dW2) / N
    W2 = W2 - learning_rate * W2_update
    W1 = W1 - learning_rate * W1_update
results.mse.plot(title="Mean Squared Error")

```

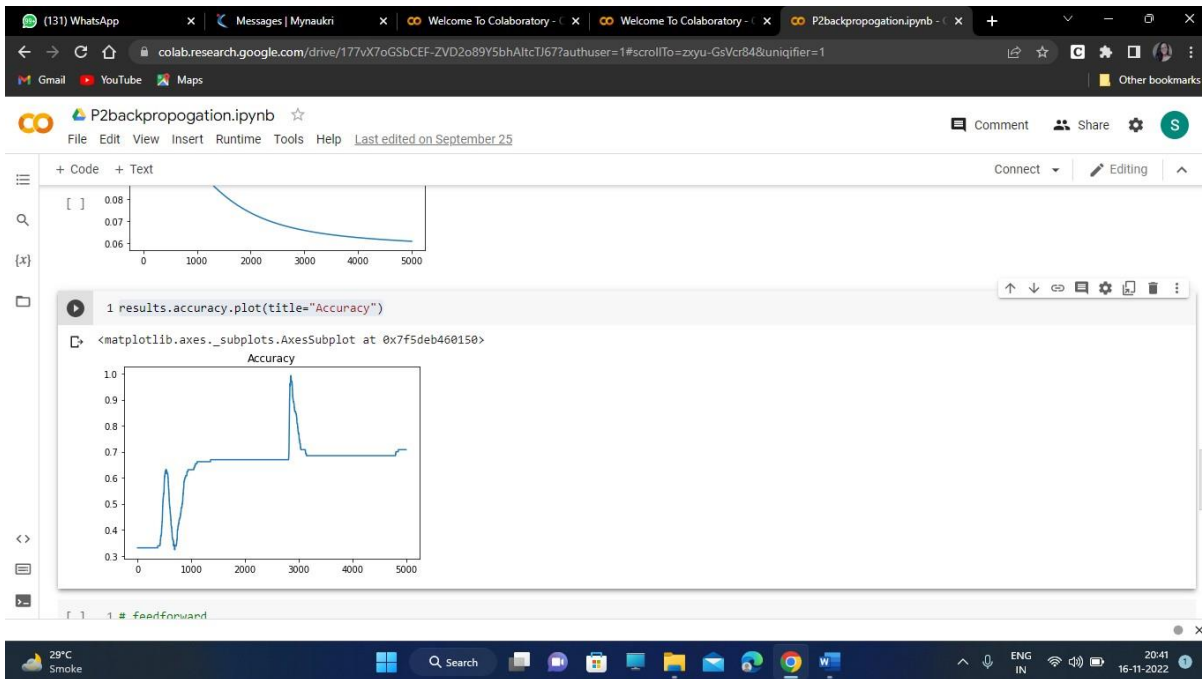


```

results.accuracy.plot(title="Accuracy")

```

Seat No.: 3254185



feedforward

$Z1 = \text{np.dot}(X_test, W1)$

$A1 = \text{sigmoid}(Z1)$

$Z2 = \text{np.dot}(A1, W2)$

$A2 = \text{sigmoid}(Z2)$

$\text{acc} = \text{accuracy}(A2, y_test)$

$\text{print}(\text{"Accuracy: " + str(acc)})$

OUTPUT: Accuracy: 0.8

CONCLUSION: Thus implemented the experiment successfully.

PRACTICAL 3**AIM:** To implement Hebb's Rule in C++**CODE:**

```

#include<iostream>
using namespace std;
int main()
{
    int m,n;
    cout<<"enter no.of features and no.of training datasets: \n";
    cin>> m>>n;
    int wt1[m], wt2[m];
    int input[n][m];
    cout<<"enter the input matrix row wise "<<endl;
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cin>>input[i][j];
        }
    }

    int target1[n], target2[n];
    cout<<" Enter the target in binary : "<<endl;
    for(int i=0;i<n;i++)
    {
        cin>>target1[i];
    }
    cout<<"Enter the target in bipolar: "<<endl;
    for(int i=0;i<n;i++)
    {
        cin>>target2[i];
    }
    for(int i=0;i<m;i++)
    {
        wt1[i]=0; //step 1: initialise all wts to 0
        wt2[i]=0;
    }
    for(int j=0;j<n;j++)
    {
        cout<<"#####j="<<j<<endl;
        for(int i=0; i<m;i++)
        {
            wt1[i]+= (input[j][i]*target1[j]);

```

```
        cout<<"weight1 at i="<<i<<" is "<<wt1[i]<<endl;
        wt2[i]+= (input[j][i]*target2[j]);
        cout<<"wt2 at i="<<i<<" is "<<wt2[i]<<endl;
    }
}
cout<<"*****OUTPUT*****\nafter 1 epoch: binary weights: "<<endl;
for (int i = 0; i < m; ++i)
{
    /* code */
    cout<<wt1[i]<<" ";
}
cout<<"\nafter 1 epoch: bipolar weights: "<<endl;
for (int i = 0; i < m; ++i)
{
    /* code */
    cout<<wt2[i]<<" ";
}
return 0;
}
```

Output:

enter no.of features and no.of training datasets:

3 4

enter the input matrix row wise

-1 -1 1

-1 1 1

1 1 -1

1 1 1

Enter the target in binary :

0

0

0

1

Enter the target in bipolar:

-1

-1

-1

1

#####j=0

weight1 at i=0 is 0

wt2 at i=0 is 1

weight1 at i=1 is 0

wt2 at i=1 is 1

weight1 at i=2 is 0

Seat No.: 3254185

```
wt2 at i=2 is -1
#####j=1
weight1 at i=0 is 0
wt2 at i=0 is 2
weight1 at i=1 is 0
wt2 at i=1 is 0
weight1 at i=2 is 0
wt2 at i=2 is -2
#####j=2
weight1 at i=0 is 0
wt2 at i=0 is 1
weight1 at i=1 is 0
wt2 at i=1 is -1
weight1 at i=2 is 0
wt2 at i=2 is -1
#####j=3
weight1 at i=0 is 1
wt2 at i=0 is 2
weight1 at i=1 is 1
wt2 at i=1 is 0
weight1 at i=2 is 1
wt2 at i=2 is 0
*****OUTPUT*****
after 1 epoch: binary weights:
1 1 1
after 1 epoch: bipolar weights:
2 0 0
```

** Process exited - Return Code: 0 **

CONCLUSION: Thus implemented the experiment successfully.

PRACTICAL 4**AIM:** To implement McCulloch Pitts model for XOR gate**CODE:**

```

# import Python Libraries
import numpy as np
from matplotlib import pyplot as plt
# Sigmoid Function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
# Initialization of the neural network parameters
# Initialized all the weights in the range of between 0 and 1
# Bias values are initialized to 0
def initializeParameters(inputFeatures, neuronsInHiddenLayers, outputFeatures):
    W1 = np.random.randn(neuronsInHiddenLayers, inputFeatures)
    W2 = np.random.randn(outputFeatures, neuronsInHiddenLayers)
    b1 = np.zeros((neuronsInHiddenLayers, 1))
    b2 = np.zeros((outputFeatures, 1))
    parameters = {"W1" : W1, "b1": b1,
                  "W2" : W2, "b2": b2}
    return parameters
# Forward Propagation
def forwardPropagation(X, Y, parameters):
    m = X.shape[1]
    W1 = parameters["W1"]
    W2 = parameters["W2"]
    b1 = parameters["b1"]
    b2 = parameters["b2"]
    Z1 = np.dot(W1, X) + b1
    A1 = sigmoid(Z1)
    Z2 = np.dot(W2, A1) + b2
    A2 = sigmoid(Z2)
    cache = (Z1, A1, W1, b1, Z2, A2, W2, b2)
    logprobs = np.multiply(np.log(A2), Y) + np.multiply(np.log(1 - A2), (1 - Y))
    cost = -np.sum(logprobs) / m
    return cost, cache, A2
# Backward Propagation
def backwardPropagation(X, Y, cache):
    m = X.shape[1]
    (Z1, A1, W1, b1, Z2, A2, W2, b2) = cache
    dZ2 = A2 - Y
    dW2 = np.dot(dZ2, A1.T) / m
    db2 = np.sum(dZ2, axis = 1, keepdims = True)
    dA1 = np.dot(W2.T, dZ2)
    dZ1 = np.multiply(dA1, A1 * (1 - A1))

```

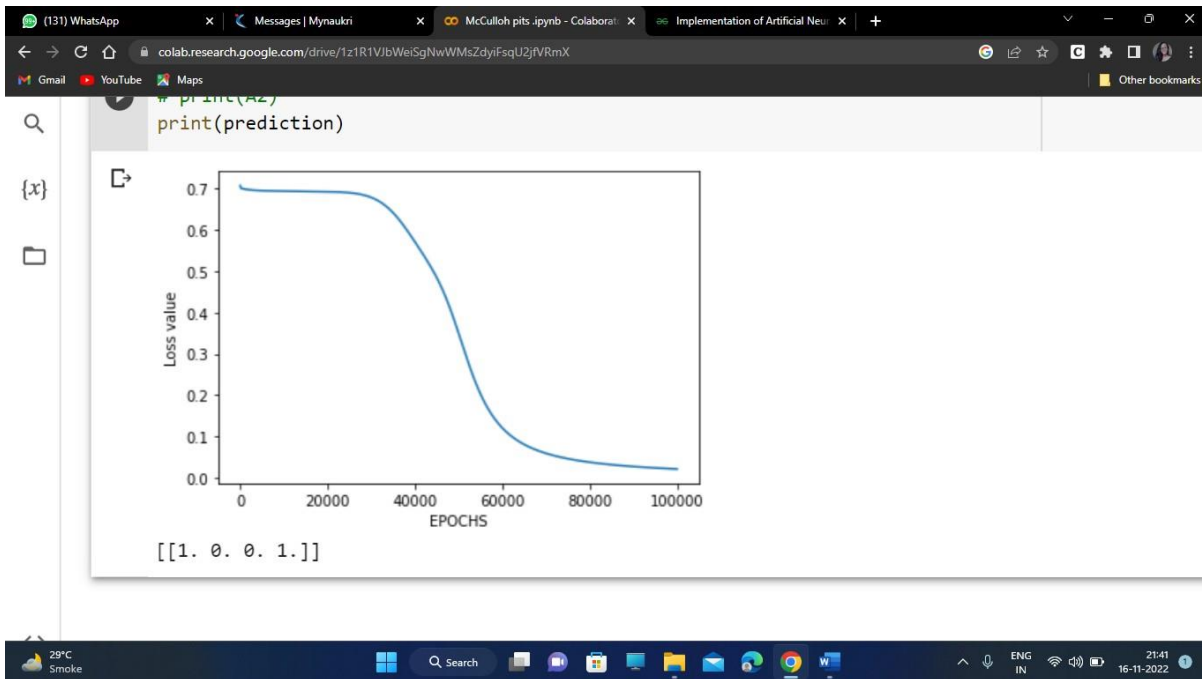
```

dW1 = np.dot(dZ1, X.T) / m
db1 = np.sum(dZ1, axis = 1, keepdims = True) / m
gradients = {"dZ2": dZ2, "dW2": dW2, "db2": db2,
             "dZ1": dZ1, "dW1": dW1, "db1": db1}
return gradients
# Updating the weights based on the negative gradients
def updateParameters(parameters, gradients, learningRate):
    parameters["W1"] = parameters["W1"] - learningRate * gradients["dW1"]
    parameters["W2"] = parameters["W2"] - learningRate * gradients["dW2"]
    parameters["b1"] = parameters["b1"] - learningRate * gradients["db1"]
    parameters["b2"] = parameters["b2"] - learningRate * gradients["db2"]
    return parameters
# Model to learn the XOR truth table
X = np.array([[0, 0, 1, 1], [0, 1, 0, 1]]) # XOR input
Y = np.array([[0, 1, 1, 0]]) # XOR output
# Define model parameters
neuronsInHiddenLayers = 2 # number of hidden layer neurons (2)
inputFeatures = X.shape[0] # number of input features (2)
outputFeatures = Y.shape[0] # number of output features (1)
parameters = initializeParameters(inputFeatures, neuronsInHiddenLayers, outputFeatures)
epoch = 100000
learningRate = 0.01
losses = np.zeros((epoch, 1))
for i in range(epoch):
    losses[i, 0], cache, A2 = forwardPropagation(X, Y, parameters)
    gradients = backwardPropagation(X, Y, cache)
    parameters = updateParameters(parameters, gradients, learningRate)
# Evaluating the performance
plt.figure()
plt.plot(losses)
plt.xlabel("EPOCHS")
plt.ylabel("Loss value")
plt.show()
# Testing
X = np.array([[1, 1, 0, 0], [0, 1, 0, 1]]) # XOR input
cost, _, A2 = forwardPropagation(X, Y, parameters)
prediction = (A2 > 0.5) * 1.0
# print(A2)
print(prediction)

```

OUTPUT:

Seat No.: 3254185



CONCLUSION: Thus implemented the experiment successfully.

PRACTICAL 5**AIM:** To implement Maxnet**CODE:**

```

def winner(mylist):
    a=[]
    for i in range(0,len(mylist)):
        if mylist[i]>0:
            a.append(i)
    return a
m=0
delta=0
yin=[0]
epoch=0
f=open("maxnet_input.txt")
linenumber=0
while True:
    line=f.readline()
    line=line.rstrip('\n')
    if len(line)==0:
        break
    linenumber+=1
    word=line.split('=')
    if linenumber==1:
        m=int(word[1])
        yin=yin*m
    if linenumber==3:
        x=word[1].split(',')
        for i in range(x.__len__()):
            yin[i]=float(x[i])
    if linenumber==2:
        delta=float(word[1])
f.close();
while True:
    epoch+=1
    yout=[]
    for i in range(0,m):
        if yin[i]>=0:
            yout.append(yin[i])
        else:
            yout.append(0)
    if len(winner(yout))==1:
        print('winner unit is : ',winner(yout)[0]+1)
        break

```

Seat No.: 3254185

```
for i in range(0,m):  
    yin[i]=yout[i]-(sum(yout)-yout[i])*delta  
if epoch==100:  
    break
```

OUTPUT: winner unit is : 4

CONCLUSION: Thus implemented the experiment successfully.

PRACTICAL 6**AIM:** To implement BAM using python**CODE:**

```

# Import Python Libraries
import numpy as np
# Take two sets of patterns:
# Set A: Input Pattern
x1 = np.array([1, 1, 1, 1, 1, 1]).reshape(6, 1)
x2 = np.array([-1, -1, -1, -1, -1, -1]).reshape(6, 1)
x3 = np.array([1, 1, -1, -1, 1, 1]).reshape(6, 1)
x4 = np.array([-1, -1, 1, 1, -1, -1]).reshape(6, 1)
# Set B: Target Pattern
y1 = np.array([1, 1, 1]).reshape(3, 1)
y2 = np.array([-1, -1, -1]).reshape(3, 1)
y3 = np.array([1, -1, 1]).reshape(3, 1)
y4 = np.array([-1, 1, -1]).reshape(3, 1)
'''
print("Set A: Input Pattern, Set B: Target Pattern")
print("\nThe input for pattern 1 is")
print(x1)
print("\nThe target for pattern 1 is")
print(y1)
print("\nThe input for pattern 2 is")
print(x2)
print("\nThe target for pattern 2 is")
print(y2)
print("\nThe input for pattern 3 is")
print(x3)
print("\nThe target for pattern 3 is")
print(y3)
print("\nThe input for pattern 4 is")
print(x4)
print("\nThe target for pattern 4 is")
print(y4)
print("\n.....")
'''
# Calculate weight Matrix: W
inputSet = np.concatenate((x1, x2, x3, x4), axis = 1)
targetSet = np.concatenate((y1.T, y2.T, y3.T, y4.T), axis = 0)
print("\nWeight matrix:")
weight = np.dot(inputSet, targetSet)
print(weight)
print("\n.....")
# Testing Phase

```

Test for Input Patterns: Set A

```
print("\nTesting for input patterns: Set A")
```

```
def testInputs(x, weight):
```

```
    # Multiply the input pattern with the weight matrix
```

```
    # (weight.T X x)
```

```
    y = np.dot(weight.T, x)
```

```
    y[y < 0] = -1
```

```
    y[y >= 0] = 1
```

```
    return np.array(y)
```

```
print("\nOutput of input pattern 1")
```

```
print(testInputs(x1, weight))
```

```
print("\nOutput of input pattern 2")
```

```
print(testInputs(x2, weight))
```

```
print("\nOutput of input pattern 3")
```

```
print(testInputs(x3, weight))
```

```
print("\nOutput of input pattern 4")
```

```
print(testInputs(x4, weight))
```

Test for Target Patterns: Set B

```
print("\nTesting for target patterns: Set B")
```

```
def testTargets(y, weight):
```

```
    # Multiply the target pattern with the weight matrix
```

```
    # (weight X y)
```

```
    x = np.dot(weight, y)
```

```
    x[x <= 0] = -1
```

```
    x[x > 0] = 1
```

```
    return np.array(x)
```

```
print("\nOutput of target pattern 1")
```

```
print(testTargets(y1, weight))
```

```
print("\nOutput of target pattern 2")
```

```
print(testTargets(y2, weight))
```

```
print("\nOutput of target pattern 3")
```

```
print(testTargets(y3, weight))
```

```
print("\nOutput of target pattern 4")
```

```
print(testTargets(y4, weight))
```

OUTPUT:

Weight matrix:

```
[[4 0 4]
```

```
 [4 0 4]
```

```
 [0 4 0]
```

```
 [0 4 0]
```

```
 [4 0 4]
```

```
 [4 0 4]]
```

Testing for input patterns: Set A

Output of input pattern 1

[[1]

[1]

[1]]

Output of input pattern 2

[[-1]

[-1]

[-1]]

Output of input pattern 3

[[1]

[-1]

[1]]

Output of input pattern 4

[[-1]

[1]

[-1]]

Testing for target patterns: Set B

Output of target pattern 1

[[1]

[1]

[1]

[1]

[1]

[1]]

Output of target pattern 2

[[-1]

[-1]

[-1]

[-1]

[-1]

[-1]]

Output of target pattern 3

[[1]

[1]

[-1]

[-1]

[1]

[1]]

Output of target pattern 4

[[-1]

[-1]

Seat No.: 3254185

[1]

[1]

[-1]

[-1]]

CONCLUSION: Thus implemented the experiment successfully.

PRACTICAL 7

AIM: Find the ratios using fuzzy logic

CODE:

```
!pip install fuzzywuzzy
!pip install python-Levenshtein
from fuzzywuzzy import fuzz
from fuzzywuzzy import process
s1 = "I love fuzzysforfuzzys"
s2 = "I am loving fuzzysforfuzzys"
print ("FuzzyWuzzy Ratio:", fuzz.ratio(s1, s2))
print ("FuzzyWuzzy PartialRatio: ", fuzz.partial_ratio(s1, s2))
print ("FuzzyWuzzy TokenSortRatio: ", fuzz.token_sort_ratio(s1, s2))
print ("FuzzyWuzzy TokenSetRatio: ", fuzz.token_set_ratio(s1, s2))
print ("FuzzyWuzzy WRatio: ", fuzz.WRatio(s1, s2), '\n\n')
# for process library,
query = 'fuzzys for fuzzys'
choices = ['fuzzy for fuzzy', 'fuzzy fuzzy', 'g. for fuzzys']
print ("List of ratios: ")
print (process.extract(query, choices), '\n')
print ("Best among the above list: ", process.extractOne(query, choices))
```

OUTPUT:

```
FuzzyWuzzy Ratio: 86
FuzzyWuzzy PartialRatio: 86
FuzzyWuzzy TokenSortRatio: 86
FuzzyWuzzy TokenSetRatio: 87
FuzzyWuzzy WRatio: 86
List of ratios:
[('g. for fuzzys', 95), ('fuzzy for fuzzy', 94), ('fuzzy fuzzy', 86)]
Best among the above list: ('g. for fuzzys', 95)
```

CONCLUSION: Thus implemented the experiment successfully.

PRACTICAL 8

AIM: Fuzzy logic for tipping

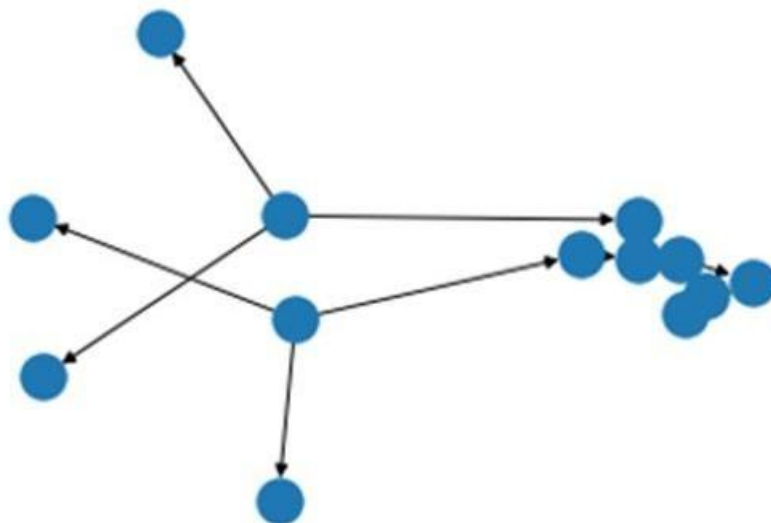
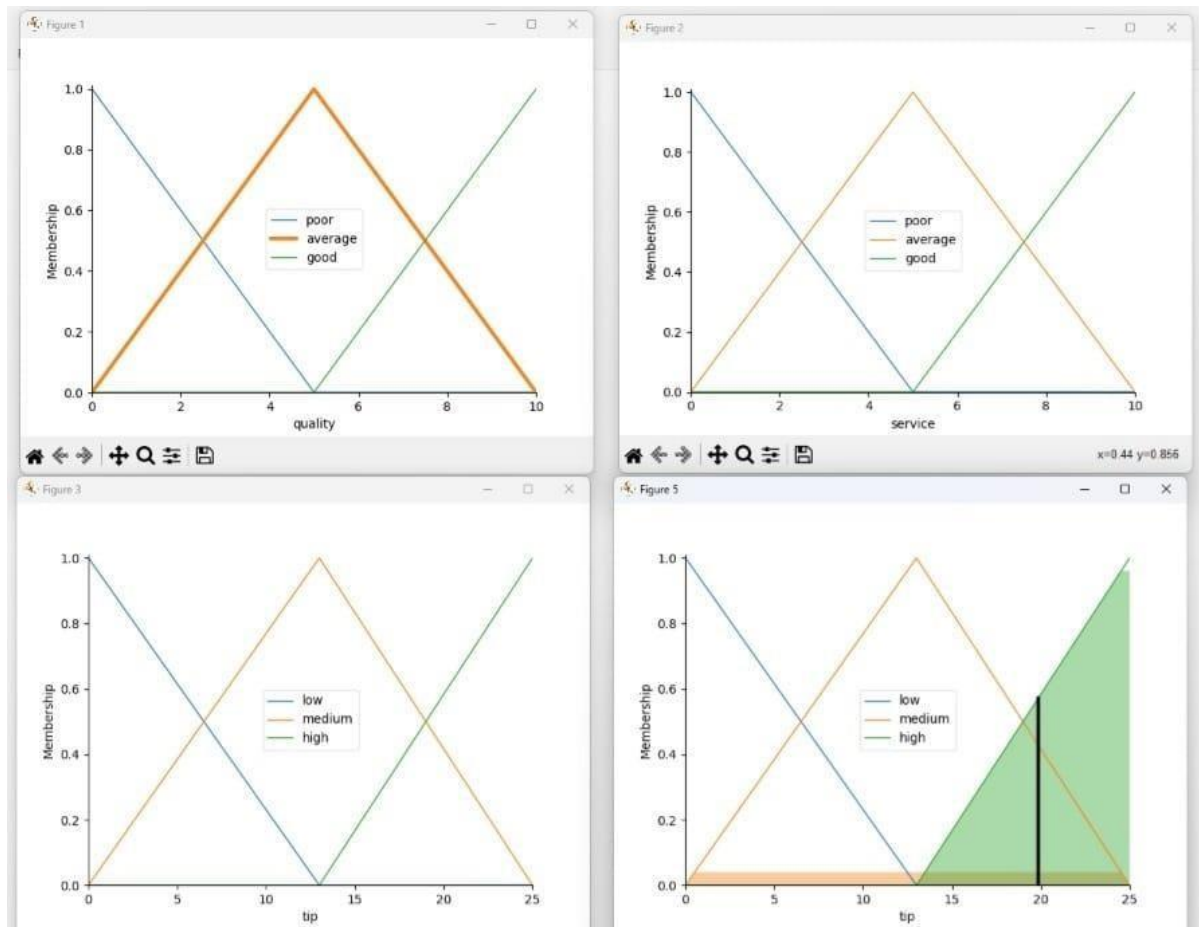
CODE:

```
!pip install scikit-fuzzy
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# New Antecedent/Consequent objects hold universe variables and membership
# functions
quality = ctrl.Antecedent(np.arange(0, 11, 1), 'quality')
service = ctrl.Antecedent(np.arange(0, 11, 1), 'service')
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')
quality.automf(3)
service.automf(3)
tip['low'] = fuzz.trimf(tip.universe, [0, 0, 13])
tip['medium'] = fuzz.trimf(tip.universe, [0, 13, 25])
tip['high'] = fuzz.trimf(tip.universe, [13, 25, 25])
quality['average'].view()
service.view()
tip.view()
rule1 = ctrl.Rule(quality['poor'] | service['poor'], tip['low'])
rule2 = ctrl.Rule(service['average'], tip['medium'])
rule3 = ctrl.Rule(service['good'] | quality['good'], tip['high'])
rule1.view()
tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
tipping = ctrl.ControlSystemSimulation(tipping_ctrl)
tipping.input['quality'] = 6.5
tipping.input['service'] = 9.8
tipping.compute()
print (tipping.output['tip'])
tip.view(sim=tipping)
```

OUTPUT:

19.847607361963192



CONCLUSION: Thus implemented the experiment successfully.